Faculty Of Technological Industry and Energy
كلية تكنولوجيا الصناعة والطاقة

جامعة الدلتا التكنولوجية
DTU DELTA TECHNOLOGICAL UNIVERSITY

Department of Information Technology
Delta Technological University

First Year

(First Semester)

Lecture Notes Digital Engineering



2026/2027

Dr.Hassanein Shaban Ahmed

Table of Contents

Chapter 1: Number Systems 6
1.1 Introduction 7
1.2 Number Systems 7
1.2.1 Decimal Number System 7
1.2.2 Binary Number System 8
1.2.3 Octal Number System 9
1.2.4 Hexadecimal Number System 9
1.3 Decimal Number to other Bases Conversion 10
1.4 Decimal to Binary Conversion 11
1.5 Decimal to Octal Conversion 12
1.6 Decimal to Hexa-Decimal Conversion 13
1.7 Binary Number to other Bases Conversion 14
1.7.1 Binary to Decimal Conversion 14
1.7.2 Binary to Octal Conversion 15
1.7.3 Binary to Hexa-Decimal Conversion 16
1.8 Octal Number to other Bases Conversion 17
1.8.1 Octal to Decimal Conversion 17
1.8.2 Octal to Binary Conversion 17
1.8.3 Octal to Hexa-Decimal Conversion 18
1.9 Hexa-Decimal Number to other Bases Conversion 18
1.9.1 Hexa-Decimal to Decimal Conversion 18
1.9.2 Hexa-Decimal to Binary Conversion 19
1.9.3 Hexa-Decimal to Octal Conversion 19
1.10 Unsigned Numbers 20
1.10.1 Representation of Un-Signed Binary Numbers 20
1.11 Signed Numbers 21
1.11.1 Representation of Signed Binary Numbers 21
1.11.2 Sign-Magnitude form 22
1.11.2 1's complement form 22
1.11.2 2's complement form 23
1.12 Basic Arithmetic Operations 23
1.12.1 Addition of two Signed Binary Numbers 24
1.12.2 Subtraction of two Signed Binary Numbers 25
1.13 Gray Code 27
1.13.1 Binary code to Gray Code Conversion 27
1.14 Binary Coded Decimal BCD 28
1.15 Text: ASCII Characters 29

Chapter 2: Boolean algebra and logic gates 30
2.1 Introduction 31
2.2 Basic Gates 31
2.2.1 AND gate 31
2.2.2 OR gate 32
2.2.3 NOT gate 33
2.3 Universal gates 34
2.3.1 NAND gate 34
2.3.2 NOR gate 35
2.4 Special Gates 35
2.4.1 Ex-OR gate 35
2.4.2 Ex-NOR gate 37
2.5 Universal Gates 38
2.5.1 NAND Gate 38
2.5.2 NOR Gate 39
2.6 NOT Gate Realization 39
2.6.1 Using NAND Gate 39
2.6.2 Using NOR Gate 40
2.7 AND Gate Realization 40
2.7.1 Using NAND gate 40
2.7.2 Using NOR Gate 40
2.8 OR Gate Realization 40
2.8.1 Using NAND Gate 40
2.8.2 Using NOR Gate 41
2.9 Boolean Algebra 41
2.9.1 Basic Laws of Boolean Algebra 42
2.9.1.1 Commutative Law 42
2.9.1.2 Associative Law 43
2.9.1.3 Distributive Law 44
2.9.2 Rules of Boolean Algebra 45
2.10 De Morgan's Theorems 48
2.11 Simplification Using Boolean Algebra 50
2.12 Converting AND to OR 53
2.13 Standard Forms 54
2.13.1 Sum-of-Products Expression 56
2.13.2 SOP with NANDs 57
2.14 Definition of Boolean Functions 57
2.14 Representation of Boolean Functions 58
2.15 Function Representation Conversion 58

2.15.1 Converting Truth Table to Boolean Expression 59
2.15.2 Converting to a Circuit 60
Chapter 3: Simplification of Boolean Functions 61
3.1 Introduction 62
3.2 Karnaugh Maps 62
3.2.1 Two-Variable K-Map 63
3.2.2 Three-Variable K-Map 64
3.2.3 Rules for K-Maps 65
3.3 Four-Variable K-Map 67
3.4 Don't-Care Conditions 70
3.5 Definition of Terms 72
Chapter4: Combinational Circuits (Analysis & Design) 73
4.1 Introduction 74
4.2 Combinational Circuits (Analysis & Design) 74
4.2.1 Combinational Circuit Analysis 75
4.2.2 Circuit Analysis steps 75
4.3 Combinational Circuit Design 77
4.4 Binary Adder 78
4.4.1 Half Adder 78
4.4.2 Full Adder 80
4.5 Four-bit Adder 83
4.6 Four-bit Subtractor 84
4.7 Four-bit Adder-Subtractor 85
4.8 Overflow in 2's Complement Addition 85
4.9 Binary Decoder 86
4.9.1 3-to-8 Binary Decoder 88
4.9.2 4 to 16 Decoder 89
4.10 Implementing Functions Using Decoders 90
4.11 Binary Encoder 91
4.11.1 4 to 2 Encoder 92
4.12 Binary Multiplexers 93
4.12.1 4x1 Multiplexer 93
4.12.1 Implementing Boolean Functions with MUXs 95
4.12.2 Implementing a 4-Input Function with a MUX 96
4.13 Three-state Gate 96
Chapter 5: Synchronous Sequential Logic 97
5.1 Introduction 98
5.2 Sequential Circuits 98
5.2.1 Types of Sequential Circuits 99

5.2.1.1 Asynchronous sequential circuits 99
5.2.1.2Synchronous sequential circuits 100
5.3 Storage Elements 100
5.4 Level-sensitive vs Edge-triggered 101
5.5 SR Latch 101
5.6 D Latch 103
5.7 D flip-flop 104
5.8 JK flip-flop 106
5.9 T Flip-Flop 107
5.10 Asynchronous Inputs 109
Chapter 6: Registers and Counters 110
6.1 Introduction 111
6.2 Simple Parallel Load Register 111
6.2.1 Parallel Load Register 113
6.3 Shift Registers 113
6.4 Counters 114
6.4.1 Binary Counter 114
6.4.2 BCD counter using a counter with parallel load 115
Chapter 7: Software Lab Manual 117
References 149

Chapter 1: Number Systems

1.1 Introduction

Consider base or radix of a number system is ‘r’, then the numbers present in that number system are ranging from zero to r-1. The total numbers present in that number system is ‘r’. So, we will get various number systems, by choosing the values of radix as greater than or equal to two.

In this chapter, let us discuss about the popular number systems and how to represent a number in the respective number system.

1.2 Number Systems

The following number systems are the most commonly used.

1. Decimal Number system
2. Binary Number system
3. Octal Number system
4. Hexadecimal Number system

1.2.1 Decimal Number System

The base or radix of Decimal number system is 10. So, the numbers ranging from 0 to 9 are used in this number system. The part of the number that lies to the left of the decimal point is known as integer part. Similarly, the part of the number that lies to the right of the decimal point is known as fractional part.

In this number system, the successive positions to the left of the decimal point having weights of 10^0, 10^1, 10^2, 10^3 and so on. Similarly, the successive positions to the right of the decimal point having weights of 10^{-1}, 10^{-2}, 10^{-3} and so on. That means, each position has specific weight, which is power of base 10

Example

Consider the decimal number 1358.246. Integer part of this number is 1358 and fractional part of this number is 0.246. The digits 8, 5, 3 and 1 have weights of 100,

101, 10^2 and 10^3respectively. Similarly, the digits 2, 4 and 6 have weights of 10^{-1}, 10^{-2} and 10^{-3} respectively.

Mathematically, we can write it as

$$1358.246 = (1 \times 10^3) + (3 \times 10^2) + (5 \times 10^1) + (8 \times 10^0) + (2 \times 10^{-1}) + (4 \times 10^{-2}) + (6 \times 10^{-3})$$

After simplifying the right hand side terms, we will get the decimal number, which is on left hand side.

1.2.2 **Binary Number System**

All digital circuits and systems use this binary number system. The **base** or radix of this number system is 2. So, the numbers 0 and 1 are used in this number system.

The part of the number, which lies to the left of the **binary point** is known as integer part. Similarly, the part of the number, which lies to the right of the binary point is known as fractional part.

In this number system, the successive positions to the left of the binary point having weights of 2^0, 2^1, 22, 2^3 and so on. Similarly, the successive positions to the right of the binary point having weights of $2^{1,}{}_2$-2, 2-3 and so on. That means, each position has specific weight, which is **power of base 2.**

Example

Consider the **binary number** 1101.011. Integer part of this number is 1101 and fractional part of this number is 0.011. The digits 1, 0, 1 and 1 of integer part have weights of 2^0, 2^1, 2^2, 2^3respectively. Similarly, the digits 0, 1 and 1 of fractional part have weights of 2^{-1}, 2-2, 2^{-3} respectively.

Mathematically, we can write it as

$$1101.011 = (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) + (0 \times 2^{-1}) + (1 \times 2^{-2}) + (1 \times 2^{-3})$$

After simplifying the right hand side terms, we will get a decimal number, which is an equivalent of binary number on left hand side.

1.2.3 Octal Number System

The **base** or radix of octal number system is 8. So, the numbers ranging from 0 to 7 are used in this number system. The part of the number that lies to the left of the **octal point** is known as integer part. Similarly, the part of the number that lies to the right of the octal point is known as fractional part.

In this number system, the successive positions to the left of the octal point having weights of 8^0, 8^1, 82, 8^3 and so on. Similarly, the successive positions to the right of the octal point having weights of 8^{-1}, 8-2, 8^{-3} and so on. That means, each position has specific weight, which is **power of base 8.**

Example

Consider the **octal number 1457.236.** Integer part of this number is 1457 and fractional part of this number is 0.236. The digits 7, 5, 4 and 1 have weights of 8^0, 81, 8^2 and 8^3respectively. Similarly, the digits 2, 3 and 6 have weights of 8^{-1}, 8-2, 8^{-3} respectively.

Mathematically, we can write it as

$$1457.236 = (1 \times 8^3) + (4 \times 8^2) + (5 \times 8^1) + (7 \times 8^0) + (2 \times 8^{-1}) + (3 \times 8^{-2}) + (6 \times 8\text{-}3)$$

After simplifying the right hand side terms, we will get a decimal number, which is an equivalent of octal number on left hand side.

1.2.4 Hexadecimal Number System

The base or radix of Hexa-decimal number system is 16. So, the numbers ranging from 0 to 9 and the letters from A to F are used in this number system. The decimal equivalent of Hexa-decimal digits from A to F are 10 to 15.

The part of the number, which lies to the left of the hexadecimal point is known as integer part. Similarly, the part of the number, which lies to the right of the Hexa-decimal point is known as fractional part.

In this number system, the successive positions to the left of the Hexa-decimal point having weights of 16^0, 16^1, 16^2, 16^3 and so on. Similarly, the successive positions to the right of the Hexa-decimal point having weights of 16^{-1}, 16^{-2}, 16^{-3} and so on. That means, each position has specific weight, which is power of base 16.

Example

Consider the Hexa-decimal number 1A05.2C4. Integer part of this number is 1A05 and fractional part of this number is 0.2C4. The digits 5, 0, A and 1 have weights of 16^0, 16^1, 16^2 and 16^3respectively. Similarly, the digits 2, C and 4 have weights of 16^{-1}, 16^{-2} and 16^{-3} respectively.

Mathematically, we can write it as

$$1A05.2C4 = (1 \times 16^3) + (10 \times 16^2) + (0 \times 16^1) + (5 \times 16^0) + (2 \times 16^{-1}) + (12 \times 16^{-2}) + (4 \times 16^{-3})$$

After simplifying the right hand side terms, we will get a decimal number, which is an equivalent of Hexa-decimal number on left hand side.

In previous chapter, we have seen the four prominent number systems. In this chapter, let us convert the numbers from one number system to the other in order to find the equivalent value.

1.3 Decimal Number to other Bases Conversion

If the decimal number contains both integer part and fractional part, then convert both the parts of decimal number into other base individually. Follow these steps for converting the decimal number into its equivalent number of any base 'r'.

- Do division of integer part of decimal number and successive quotients with base ‘r’ and note down the remainders till the quotient is zero. Consider the remainders in reverse order to get the integer part of equivalent number of base ‘r’. That means, first and last remainders denote the least significant digit and most significant digit respectively.
- Do multiplication of fractional part of decimal number and successive fractions with base ‘r’ and note down the carry till the result is zero or the desired number of equivalent digits is obtained. Consider the normal sequence of carry in order to get the fractional part of equivalent number of base ‘r’.

1.4 Decimal to Binary Conversion

The following two types of operations take place, while converting decimal number into its equivalent binary number.

- Division of integer part and successive quotients with base 2.
- Multiplication of fractional part and successive

fractions with base 2.

Example

Consider the decimal number 58.25. Here, the integer part is 58 and fractional part is 0.25.

Step 1 − Division of 58 and successive quotients with base 2.

Operation	Quotient	Remainder
58/2	29	0 (LSB)
29/2	14	1
14/2	7	0
7/2	3	1
3/2	1	1
1/2	0	1(MSB)

(58)10 = (111010)2 Therefore, the integer part of equivalent binary number is 111010.

Step 2 − Multiplication of 0.25 and successive fractions with base 2.

Operation	Result	Carry
0.25 x 2	0.5	0
0.5 x 2	1.0	1
-	0.0	-

⇒ (.25)10 = (.01)2 Therefore, the fractional part of equivalent binary number is .01

⇒ (58.25)10 = (111010.01)2

Therefore, the binary equivalent of decimal number 58.25 is 111010.01.

1.5 Decimal to Octal Conversion

The following two types of operations take place, while converting decimal number into its equivalent octal number.

- Division of integer part and successive quotients with base 8.
- Multiplication of fractional part and successive fractions with base 8.

Example

Consider the decimal number 58.25. Here, the integer part is 58 and fractional part is 0.25.

Step 1 − Division of 58 and successive quotients with base 8.

Operation	Quotient	Remainder
58/8	7	2
7/8	0	7

⇒ (58)10 = (72)8 Therefore, the integer part of equivalent octal number is 72.

Step 2 − Multiplication of 0.25 and successive fractions with base 8.

Operation	Result	Carry
0.25 x 8	2.00	2
-	0.00	-

⇒ (.25)10 = (.2)8 Therefore, the fractional part of equivalent octal number is .2

⇒ (58.25)10 = (72.2)8 Therefore, the octal equivalent of decimal number 58.25 is 72.2.

1.6 Decimal to Hexa-Decimal Conversion

The following two types of operations take place, while converting decimal number into its equivalent hexa-decimal number.

- Division of integer part and successive quotients with base 16.
- Multiplication of fractional part and successive fractions with base 16. Example

Consider the decimal number 58.25. Here, the integer part is 58 and decimal part is 0.25.

Step 1 − Division of 58 and successive quotients with base 16.

Operation	Quotient	Remainder
58/16	3	10=A
3/16	0	3

⇒(58)10 = (3A)16 Therefore, the integer part of equivalent Hexa-decimal number is 3A.

⇒ Step 2 − Multiplication of 0.25 and successive fractions with base 16.

Operation	Result	Carry
0.25 x 16	4.00	**4**
-	0.00	-

⇒ (.25)10 = (.4)16 Therefore, the **fractional part** of equivalent Hexa-decimal number is .4.

⇒ (58.25)10 = (3A.4)16

Therefore, the **Hexa-decimal equivalent** of decimal number 58.25 is 3A.4.

1.7 Binary Number to other Bases Conversion

The process of converting a number from binary to decimal is different to the process of converting a binary number to other bases. Now, let us discuss about the conversion of a binary number to decimal, octal and Hexa-decimal number systems one by one.

1.7.1 Binary to Decimal Conversion

For converting a binary number into its equivalent decimal number, first multiply the bits of binary number with the respective positional weights and then add all those products.

Example

Consider the **binary number 1101.11.**

Mathematically, we can write it as

$(1101.11)2 = \times 2^3) + \times 2^2) + (0 \times 2^1) + (1 \times + (1\ 2$

(1×2^{-2})

⇒ (1101. 8 + 4 + + 1 + 0.5 + 0.25 =

⇒ (1101. (13.75)1

Therefore, the **decimal equivalent** of binary number 1101.11 is 13.75.

1.7.2 Binary to Octal Conversion

We know that the bases of binary and octal number systems are 2 and 8 respectively. Three bits of binary number is equivalent to one octal digit, since $2^3 = 8$.

Follow these two steps for converting a binary number into its equivalent octal number.

- Start from the binary point and make the groups of 3 bits on both sides of binary point. If one or two bits are less while making the group of 3 bits, then include required number of zeros on extreme sides.
- Write the octal digits corresponding to each group of 3 bits.

Example

Consider the binary number 101110.01101.

Step 1 − Make the groups of 3 bits on both sides of binary point.

101 110.011 01

Here, on right side of binary point, the last group is having only 2 bits. So, include one zero on extreme side in order to make it as group of 3 bits.

⇒ 101 110.011 010

Step 2 − Write the octal digits corresponding to each group of 3 bits.

⇒ (101 110.011 010)2 = (56.32)8 Therefore, the octal equivalent of binary number 101110.01101 is 56.32.

1.7.3 Binary to Hexa-Decimal Conversion

We know that the bases of binary and Hexa-decimal number systems are 2 and 16 respectively. Four bits of binary number is equivalent to one Hexa-decimal digit, since $2^4 = 16$.

Follow these two steps for converting a binary number into its equivalent Hexa-decimal number.

- Start from the binary point and make the groups of 4 bits on both sides of binary point. If some bits are less while making the group of 4 bits, then include required number of zeros on extreme sides.
- Write the Hexa-decimal digits corresponding to each group of 4 bits.

Example

Consider the binary number 101110.01101

Step 1 − Make the groups of 4 bits on both sides of binary point.

10 1110.0110 1

Here, the first group is having only 2 bits. So, include two zeros on extreme side in order to make it as group of 4 bits. Similarly, include three zeros on extreme side in order to make the last group also as group of 4 bits.

⇒ 0010 1110.0110 1000

Step 2 − Write the Hexa-decimal digits corresponding to each group of 4 bits.

⇒ (0010 1110.0110 1000)2 = (2E.68)16

Therefore, the Hexa-decimal equivalent of binary number 101110.01101 is (2E.68).

1.8 Octal Number to other Bases Conversion

The process of converting a number from octal to decimal is different to the process of converting an octal number to other bases. Now, let us discuss about the conversion of an octal number to decimal, binary and Hexa-decimal number systems one by one.

1.8.1 Octal to Decimal Conversion

For converting an octal number into its equivalent decimal number, first multiply the digits of octal number with the respective positional weights and then add all those products.

Example

Consider the octal number 145.23.

Mathematically, we can write it as

$$(145.23)8 = (1 \times 8^2)+(4 \times 8^1)+(5 \times 8^0)+(2 \times 8^{-1}) +(3 \times 8^{-2})$$

$$\Rightarrow (145.23)8 = 64+32+5 + +0.05 = 101.3$$

$$\Rightarrow (145.23)8 = (101.3)10$$

Therefore, the decimal equivalent of octal number 145.23 is 101.3.

1.8.2 Octal to Binary Conversion

The process of converting an octal number to an equivalent binary number is just opposite to that of binary to octal conversion. By representing each octal digit with 3 bits, we will get the equivalent binary number.

Example

Consider the octal number 145.23.

Represent each octal digit with 3 bits.

$$(145.23)_8 = (001\ 100\ 101.010\ 011)_2$$

The value doesn't change by removing the zeros, which are on the extreme side.

$\Rightarrow (145.23)_8 = (1100101.010011)_2$ Therefore, the binary equivalent of octal number 145.23 is 1100101.010011.

1.8.3 Octal to Hexa-Decimal Conversion

Follow these two steps for converting an octal number into its equivalent Hexa-decimal number.

- Convert octal number into its equivalent binary number.
- Convert the above binary number into its equivalent Hexa-decimal number.

Example

Consider the octal number 145.23

In previous example, we got the binary equivalent of octal number 145.23 as 1100101.010011. By following the procedure of binary to Hexa-decimal conversion, we will get

$$(1100101.010011)2 = (65.4C)16$$

$\Rightarrow (145.23)8 = (65.4C)16$

Therefore, the Hexa-decimal equivalent of octal number 145.23 is 65.4C.

1.9 Hexa-Decimal Number to other Bases Conversion

The process of converting a number from Hexa-decimal to decimal is different to the process of converting Hexa-decimal number into other bases. Now, let us discuss about the conversion of Hexadecimal number to decimal, binary and octal number systems one by one.

1.9.1 Hexa-Decimal to Decimal Conversion

For converting Hexa-decimal number into its equivalent decimal number, first multiply the digits of Hexa-decimal number with the respective positional weights and then add all those products.

Example

Consider the Hexa-decimal number 1A5.2

Mathematically, we can write it as

$$(1A5.2)16 = (1 \times 16^2) + (10 \times 16^1) + (5 \times 16^0) + (2 \times 16^{-1})$$

$\Rightarrow (1A5.2)16 = 256 + 160 + 5 + 0.125 = 421.125$

$\Rightarrow (1A5.2)16 = (421.125)10$

Therefore, the decimal equivalent of Hexa-decimal number 1A5.2 is 421.125.

1.9.2 Hexa-Decimal to Binary Conversion

The process of converting Hexa-decimal number into its equivalent binary number is just opposite to that of binary to Hexa-decimal conversion. By representing each Hexa-decimal digit with 4 bits, we will get the equivalent binary number.

Example

Consider the Hexa-decimal number 65.4C

Represent each Hexa-decimal digit with 4 bits.

$$(65.4C)16 = (0110\ 0101.0100\ 1100)2$$

The value doesn't change by removing the zeros, which are at two extreme sides.

$\Rightarrow (65.4C)16 = (1100101.010011)2$

Therefore, the binary equivalent of Hexa-decimal number 65.4C is 1100101.010011.

1.9.3 Hexa-Decimal to Octal Conversion

Follow these two steps for converting Hexa-decimal number into its equivalent octal number.

- Convert Hexa-decimal number into its equivalent binary number.
- Convert the above binary number into its equivalent octal number.

Example

Consider the Hexa-decimal number 65.4C

In previous example, we got the binary equivalent of Hexa-decimal number 65.4C as 1100101.010011. By following the procedure of binary to octal conversion, we will get

$$(1100101.010011)2 = (145.23)8$$

$$\Rightarrow \quad (65.4C)16 = (145.23)$$

Therefore, the octal equivalent of Hexa-decimal number 65.4Cis 145.23.

We can make the binary numbers into the following two groups − Unsigned numbers and Signed numbers.

1.10 Unsigned Numbers

Unsigned numbers contain only magnitude of the number. They don't have any sign. That means all unsigned binary numbers are positive. As in decimal number system, the placing of positive sign in front of the number is optional for representing positive numbers. Therefore, all positive numbers including zero can be treated as unsigned numbers if positive sign is not assigned in front of the number.

1.10.1 Representation of Un-Signed Binary Numbers

The bits present in the un-signed binary number holds the magnitude of a number. That means, if the un-signed binary number contains 'N' bits, then all N bits represent the magnitude of the number, since it doesn't have any sign bit.

Example

Consider the decimal number 108. The binary equivalent of this number is 1101100. This is the representation of unsigned binary number.

$$(108)10 = (1101100)2$$

It is having 7 bits. These 7 bits represent the magnitude of the number 108.

1.11 Signed Numbers

Signed numbers contain both sign and magnitude of the number. Generally, the sign is placed in front of number. So, we have to consider the positive sign for positive numbers and negative sign for negative numbers. Therefore, all numbers can be treated as signed numbers if the corresponding sign is assigned in front of the number.

If sign bit is zero, which indicates the binary number is positive. Similarly, if sign bit is one, which indicates the binary number is negative.

1.11.1 Representation of Signed Binary Numbers

The Most Significant Bit (MSB) of signed binary numbers is used to indicate the sign of the numbers. Hence, it is also called as sign bit. The positive sign is represented by placing '0' in the sign bit. Similarly, the negative sign is represented by placing '1' in the sign bit.

If the signed binary number contains 'N' bits, then (N-1) bits only represent the magnitude of the number since one bit (MSB) is reserved for representing sign of the number.

There are three types of representations for signed binary numbers

- Sign-Magnitude form
- 1's complement form
- 2's complement form

Representation of a positive number in all these 3 forms is same. But, only the representation of negative number will differ in each form.

Example

Consider the positive decimal number +108. The binary equivalent of magnitude of this number is 1101100. These 7 bits represent the magnitude of the number 108. Since it is positive number, consider the sign bit as zero, which is placed on left most side of magnitude.

$$(+108)10 = (01101100)2$$

Therefore, the signed binary representation of positive decimal number +108 is 01101100. So, the same representation is valid in sign-magnitude form, 1's complement form and 2's complement form for positive decimal number +108.

1.11.2 Sign-Magnitude form

In sign-magnitude form, the MSB is used for representing sign of the number and the remaining bits represent the magnitude of the number. So, just include sign bit at the left most side of unsigned binary number. This representation is similar to the signed decimal numbers representation.

Example

Consider the negative decimal number -108. The magnitude of this number is 108. We know the unsigned binary representation of 108 is 1101100. It is having 7 bits. All these bits represent the magnitude.

Since the given number is negative, consider the sign bit as one, which is placed on left most side of magnitude.

$$(-108)10 = (11101100)2$$

Therefore, the sign-magnitude representation of -108 is 11101100.

1.11.2 1's complement form

The 1's complement of a number is obtained by complementing all the bits of signed binary number. So, 1's complement of positive number gives a negative number. Similarly, 1's complement of negative number gives a positive number.

That means, if you perform two times 1's complement of a binary number including sign bit, then you will get the original signed binary number.

Example

Consider the negative decimal number -108. The magnitude of this number is 108. We know the signed binary representation of 108 is 01101100.

It is having 8 bits. The MSB of this number is zero, which indicates positive number. Complement of zero is one and vice-versa. So, replace zeros by ones and ones by zeros in order to get the negative number.

$$(-108)10 = (10010011)2$$

Therefore, the 1's complement of (108)10 is (10010011)2.

1.11.2 2's complement form

The 2's complement of a binary number is obtained by adding one to the 1's complement of signed binary number. So, 2's complement of positive number gives a negative number. Similarly, 2's complement of negative number gives a positive number.

That means, if you perform two times 2's complement of a binary number including sign bit, then you will get the original signed binary number.

Example

Consider the negative decimal number -108.

We know the 1's complement of (108)10 is (10010011)2

2's compliment of (108)10 = 1's compliment of (108)10 + 1.

= 10010011 + 1

= 10010100

Therefore, the 2's complement of (108)10 is (10010100)2.

1.12 Basic Arithmetic Operations

In this chapter, let us discuss about the basic arithmetic operations, which can be performed on any two signed binary numbers using 2's complement method. The **basic arithmetic operations are** addition and subtraction.

1.12.1 Addition of two Signed Binary Numbers

Consider the two signed binary numbers A & B, which are represented in 2's complement form. We can perform the **addition** of these two numbers, which is similar to the addition of two unsigned binary numbers. But, if the resultant sum contains carry out from sign bit, then discard (ignore) it in order to get the correct value.

If resultant sum is positive, you can find the magnitude of it directly. But, if the resultant sum is negative, then take 2's complement of it in order to get the magnitude.

Example 1

Let us perform the **addition** of two decimal numbers **+7 and +4** using 2's complement method.

The **2's complement** representations of +7 and +4 with 5 bits each are shown below.

$$(+7)10 = (00111)2$$

$$(+4)10 = (00100)2$$

The addition of these two numbers is

$$(+7)10 + (+4)10 = (00111)2 + (00100)2$$

$$\Rightarrow \quad (+7)10 + (+4)10 = (01011)2.$$

The resultant sum contains 5 bits. So, there is no carry out from sign bit. The sign bit '0' indicates that the resultant sum is positive. So, the magnitude of sum is 11 in decimal number system. Therefore, addition of two positive numbers will give another positive number.

Example 2

Let us perform the addition of two decimal numbers -7 and -4using 2's complement method.

The 2's complement representation of -7 and -4 with 5 bits each are shown below.

$$(-7)10 = (11001)$$

$$(-4)10 = (11100)$$

The addition of these two numbers

$$(-7)10 + (-4)10 = (11001)2 + (11100)2$$

$$\Rightarrow \quad (-7)10 + (-4)10 = (110101)2.$$

The resultant sum contains 6 bits. In this case, carry is obtained from sign bit. So, we can remove it Resultant sum after removing carry is $(-7)10 + (-4)10 = (10101)2$.

The sign bit '1' indicates that the resultant sum is negative. So, by taking 2's complement of it we will get the magnitude of resultant sum as 11 in decimal number system. Therefore, addition of two negative numbers will give another negative number.

1.12.2 Subtraction of two Signed Binary Numbers

Consider the two signed binary numbers A & B, which are represented in 2's complement form. We know that 2's complement of positive number gives a negative number. So, whenever we have to subtract a number B from number A, then take 2's complement of B and add it to A. So, mathematically we can write it as

A-B=A+*(2's complement of B)*

Similarly, if we have to subtract the number A from number B, then take 2's complement of A and add it to B. So, mathematically we can write it as

B-A=B+*(2's complement of A)*

So, the subtraction of two signed binary numbers is similar to the addition of two signed binary numbers. But, we have to take 2's complement of the number, which is supposed to be subtracted. This is the advantage of 2's complement technique. Follow, the same rules of addition of two signed binary numbers.

Example 3

Let us perform the subtraction of two decimal numbers +7 and +4 using 2's complement method.

The subtraction of these two numbers is

$$(+7)10 - (+4)10 = (+7)10 + (-4)10.$$

The 2's complement representation of +7 and -4 with 5 bits each are shown below.

$$(+7)10 = (00111)2$$

$$(+4)10 = (11100)2$$

$$\Rightarrow \quad (+7)10 + (+4)10 = (00111)2 + (11100)2 = (00011)2$$

Here, the carry obtained from sign bit. So, we can remove it. The resultant sum after removing carry is (+7)10 + (+4)10 = (00011)2

The sign bit '0' indicates that the resultant sum is positive. So, the magnitude of it is 3 in decimal number system. Therefore, subtraction of two decimal numbers +7 and +4 is +3.

Example 4

Let us perform the subtraction of two decimal numbers +4 and +7 using 2's complement method.

The subtraction of these two numbers is

$$(+4)10 - (+7)10 = (+4)10 + (-7)10.$$

The 2's complement representation of +4 and -7 with 5 bits each are shown below.

$$(+4)10 = (00100)2$$

$$(-7)10 = (11001)2$$

$$\Rightarrow \quad (+4)10 + (-7)10 = (00100)2 + (11001)2 = (11101)2$$

Here, carry is not obtained from sign bit. The sign bit '1' indicates that the resultant sum is negative. So, by taking 2's complement of it we will get the magnitude of resultant sum as 3 in decimal number system. Therefore, subtraction of two decimal numbers +4 and +7 is -3.

1.13 Gray Code

The following table shows the 4-bit Gray codes corresponding to each 4-bit binary code.

1. Gray code is not a number system It is an alternate way to represent four-bit data
2. Only one bit changes from one decimal digit to the next
3. Useful for reducing errors in communication
4. Can be scaled to larger numbers

1.13.1 Binary code to Gray Code Conversion

Follow these steps for converting a binary code into its equivalent Gray code.

- Consider the given binary code and place a zero to the left of MSB.
- Compare the successive two bits starting from zero. If the 2 bits are same, then the output is zero. Otherwise, output is one.
- Repeat the above step till the LSB of Gray code is obtained

Decimal Number	Binary Code	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Example

From the table, we know that the Gray code corresponding to binary code 1000 is 1100. Now, let us verify it by using the above procedure.

Given, binary code is 1000.

Step 1 − By placing zero to the left of MSB, the binary code will be 01000.

Step 2 − By comparing successive two bits of new binary code, we will get the gray code as 1100.

1.14 Binary Coded Decimal BCD

- *Binary Coded Decimal* (BCD) represents each decimal digit with four bits

Ex. 0011 0010 1001 = $\mathbf{329_{BCD}}$

3 2 9

- *This* is NOT the same as $001100101001_2 = 809_{10.}$

Digit	BCD Code
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

1.15 Text: ASCII Characters

ASCII: Maps 128 characters to 7-bit code.

both printable and non-printable (ESC, DEL, …) characters

http://www.asciitable.com/

00	nul	10	dle	20	sp	30	0	40	@	50	P	60	`	70	p
01	soh	11	dc1	21	!	31	1	41	A	51	Q	61	a	71	q
02	stx	12	dc2	22	"	32	2	42	B	52	R	62	b	72	r
03	etx	13	dc3	23	#	33	3	43	C	53	S	63	c	73	s
04	eot	14	dc4	24	$	34	4	44	D	54	T	64	d	74	t
05	enq	15	nak	25	%	35	5	45	E	55	U	65	e	75	u
06	ack	16	syn	26	&	36	6	46	F	56	V	66	f	76	v
07	bel	17	etb	27	'	37	7	47	G	57	W	67	g	77	w
08	bs	18	can	28	(	38	8	48	H	58	X	68	h	78	x
09	ht	19	em	29	)	39	9	49	I	59	Y	69	i	79	y
0a	nl	1a	sub	2a	*	3a	:	4a	J	5a	Z	6a	j	7a	z
0b	vt	1b	esc	2b	+	3b	;	4b	K	5b	[	6b	k	7b	{
0c	np	1c	fs	2c	,	3c	<	4c	L	5c	\	6c	l	7c	|
0d	cr	1d	gs	2d	-	3d	=	4d	M	5d	]	6d	m	7d	}
0e	so	1e	rs	2e	.	3e	>	4e	N	5e	^	6e	n	7e	~
0f	si	1f	us	2f	/	3f	?	4f	O	5f	_	6f	o	7f	del

Chapter 2: Boolean algebra and logic gates

2.1 Introduction

Digital electronic circuits operate with voltages of two logic levels namely Logic Low and Logic High. The range of voltages corresponding to Logic Low is represented with ‘0’. Similarly, the range of voltages corresponding to Logic High is represented with ‘1’.

The basic digital electronic circuit that has one or more inputs and single output is known as Logic gate. Hence, the Logic gates are the building blocks of any digital system. We can classify these Logic gates into the following three categories.

- Basic gates
- Universal gates
- Special gates

Now, let us discuss about the Logic gates come under each category one by one.

2.2 Basic Gates

In earlier chapters, we learnt that the Boolean functions can be represented either in sum of products form or in product of sums form based on the requirement. So, we can implement these Boolean functions by using basic gates. The basic gates are AND, OR & NOT gates.

2.2.1 AND gate

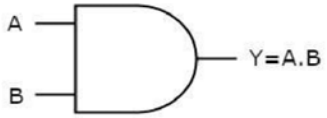
An AND gate is a digital circuit that has two or more inputs and produces an output, which is the logical AND of all those inputs. It is optional to represent the Logical AND with the symbol ‘.’.

The following table shows the **truth table** of 2-input AND gate.

A	B	Y = **A.B**
0	0	0
0	1	0
1	0	0
1	1	1

Here A, B are the inputs and Y is the output of two input AND gate. If both inputs are '1', then only the output, Y is '1'. For remaining combinations of inputs, the output, Y is '0'.

The following figure shows the **symbol** of an AND gate, which is having two inputs A, B and one output, Y.



This AND gate produces an output (Y), which is the **logical** AND of two inputs A, B. Similarly, if there are 'n' inputs, then the AND gate produces an output, which is the logical AND of all those inputs. That means, the output of AND gate will be '1', when all the inputs are '1'.

2.2.2 OR gate

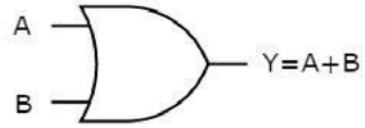
An OR gate is a digital circuit that has two or more inputs and produces an output, which is the logical OR of all those inputs. This **logical OR** is represented with the symbol '+'.

The following table shows the **truth table** of 2-input OR gate.

A	B	Y = A + B
0	0	0
0	1	1
1	0	1
1	1	1

Here A, B are the inputs and Y is the output of two input OR gate. If both inputs are '0', then only the output, Y is '0'. For remaining combinations of inputs, the output, Y is '1'.

The following figure shows the **symbol** of an OR gate, which is having two inputs A, B and one output, Y.



This OR gate produces an output (Y), which is the **logical OR** of two inputs A, B. Similarly, if there are 'n' inputs, then the OR gate produces an output, which is the logical OR of all those inputs. That means, the output of an OR gate will be '1', when at least one of those inputs is '1'.

2.2.3 NOT gate

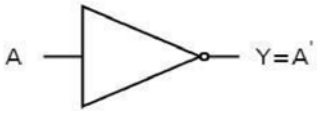
A NOT gate is a digital circuit that has single input and single output. The output of NOT gate is the **logical inversion** of input. Hence, the NOT gate is also called as inverter.

The following table shows the **truth table** of NOT gate.

A	Y = A'
0	1
1	0

Here A and Y are the input and output of NOT gate respectively. If the input, A is '0', then the output, Y is '1'. Similarly, if the input, A is '1', then the output, Y is '0'.

The following figure shows the **symbol** of NOT gate, which is having one input, A and one output, Y.



This NOT gate produces an output (Y), which is the **complement** of input, A.

2.3 Universal gates

NAND & NOR gates are called as **universal gates.** Because we can implement any Boolean function, which is in sum of products form by using NAND gates alone. Similarly, we can implement any Boolean function, which is in product of sums form by using NOR gates alone.

2.3.1 NAND gate

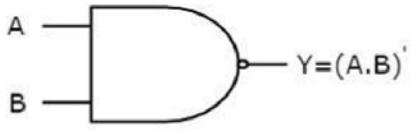
NAND gate is a digital circuit that has two or more inputs and produces an output, which is the **inversion of logical AND** of all those inputs.

The following table shows the **truth table** of 2-input NAND gate.

A	B	Y = (A.B)'
0	0	1
0	1	1
1	0	1
1	1	0

Here A, B are the inputs and Y is the output of two input NAND gate. When both inputs are '1', the output, Y is '0'. If at least one of the input is zero, then the output, Y is '1'. This is just opposite to that of two input AND gate operation.

The following image shows the **symbol** of NAND gate, which is having two inputs A, B and one output, Y.



NAND gate operation is same as that of AND gate followed by an inverter. That's why the NAND gate symbol is represented like that.

2.3.2 NOR gate

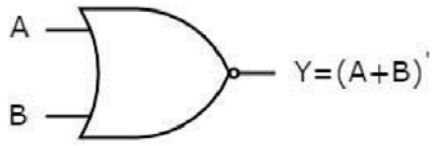
NOR gate is a digital circuit that has two or more inputs and produces an output, which is the **inversion of logical OR** of all those inputs.

The following table shows the **truth table** of 2-input NOR gate

A	B	Y = (A+B)'
0	0	1
0	1	0
1	0	0
1	1	0

Here A, B are the inputs and Y is the output. If both inputs are '0', then the output, Y is '1'. If at least one of the input is '1', then the output, Y is '0'. This is just opposite to that of two input OR gate operation.

The following figure shows the **symbol** of NOR gate, which is having two inputs A, B and one output, Y.



NOR gate operation is same as that of OR gate followed by an inverter. That's why the NOR gate symbol is represented like that.

2.4 Special Gates

Ex-OR & Ex-NOR gates are called as special gates. Because, these two gates are special cases of OR & NOR gates.

2.4.1 Ex-OR gate

The full form of Ex-OR gate is Exclusive-OR gate. Its function is same as that of OR gate except for some cases, when the inputs having even number of ones.

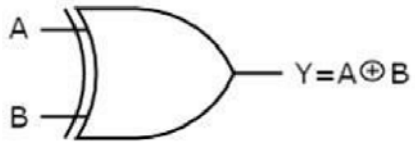
The following table shows the truth table of 2-input Ex-OR gate.

A	B	$Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Here A, B are the inputs and Y is the output of two input Ex-OR gate. The truth table of Ex-OR gate is same as that of OR gate for first three rows. The only modification is in the fourth row. That means, the output (Y) is zero instead of one, when both the inputs are one, since the inputs having even number of ones.

Therefore, the output of Ex-OR gate is '1', when only one of the two inputs is '1'. And it is zero, when both inputs are same.

Below figure shows the symbol of Ex-OR gate, which is having two inputs A, B and one output, Y.



Ex-OR gate operation is similar to that of OR gate, except for few combination(s) of inputs. That's why the Ex-OR gate symbol is represented like that. The output of Ex-OR gate is '1', when odd number of ones present at the inputs. Hence, the output of Ex-OR gate is also called as an odd function.

2.4.2 Ex-NOR gate

The full form of Ex-NOR gate is Exclusive-NOR gate. Its function is same as that of NOR gate except for some cases, when the inputs having even number of ones.

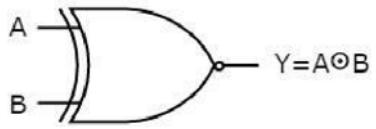
The following table shows the truth table of 2-input Ex-NOR gate.

Here A, B are the inputs and Y is the output. The truth table of Ex-NOR gate is same as that of NOR gate for first three rows. The only modification is in the fourth row. That means, the output is one instead of zero, when both the inputs are one.

A	B	$Y = A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1

Therefore, the output of Ex-NOR gate is '1', when both inputs are same. And it is zero, when both the inputs are different.

The following figure shows the symbol of Ex-NOR gate, which is having two inputs A, B and one output, Y.



Ex-NOR gate operation is similar to that of NOR gate, except for few combination(s) of inputs. That's why the Ex-NOR gate symbol is represented like that. The output of Ex-NOR gate is '1', when even

number of ones present at the inputs. Hence, the output of Ex-NOR gate is also called as an even function.

From the above truth tables of Ex-OR & Ex-NOR logic gates, we can easily notice that the Ex-NOR operation is just the logical inversion of Ex-OR operation.

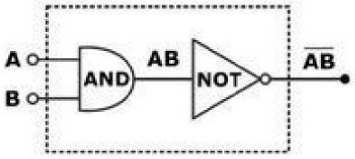
2.5 Universal Gates

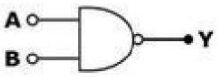
Universal gates are those gates that can perform the tasks of other gates with minor adjustments. There are two universal gates:

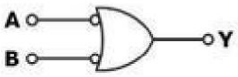
- NAND Gate
- NOR Gate

2.5.1 NAND Gate

The NAND gate is one of the universal gates. The NAND gate is a AND gate followed by a NOT gate. Thus, we can say it is a AND NOT operation. It may have two or more inputs but only one output. The logical symbols of a **NAND Gate** and the truth table are shown below.







A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

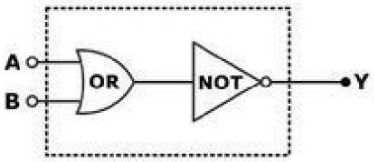
- Thus, the logical expression for the output is

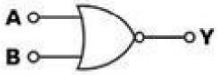
$$Y = \overline{A \cdot B} = \bar{A} + \bar{B}$$

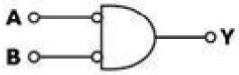
It is clear from the truth table of the two-input NAND gate that the output is 1 when either A or B or both the inputs are at logic '0'. We can say that if $\bar{A}$ = 1 = B are, both A and B are 1, and the output is 1. Therefore, the NAND gate can perform the OR function by inverting the inputs.

2.5.2 NOR Gate

The NOR gate is one of the universal gates. A NOR gate combines two basic logic gates: an OR gate and a NOT gate. So we can say it is an OR-NOT operation. It may have two or more inputs and an output. The logical symbols of the NOR Gate are shown:







A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

- It is clear from the truth table that the output is '1' only if all the inputs are at logic '0'. It can also say that if the inputs A' = B' = 1, the output Y is 1. Thus, the NOR gate is equivalent to the AND gate with inverted inputs, and it can be realized by a bubbled AND gate, as shown above.

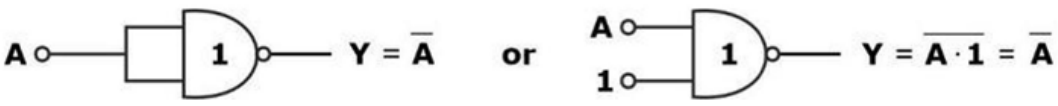
- The logical expression for the output is

$$Y = \overline{A + B} = \overline{A}\,\overline{B}$$

2.6 NOT Gate Realization

2.6.1 Using NAND Gate

For the NOT gate realization, we require 1 NAND gate, as shown in the circuit diagram:



2.6.2 Using NOR Gate

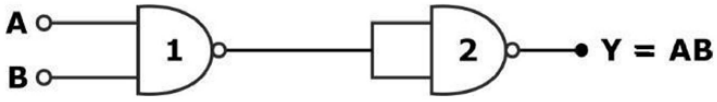
For the NOT gate realization, we require 1 NOR gate, as shown in the circuit diagram:



2.7 AND Gate Realization

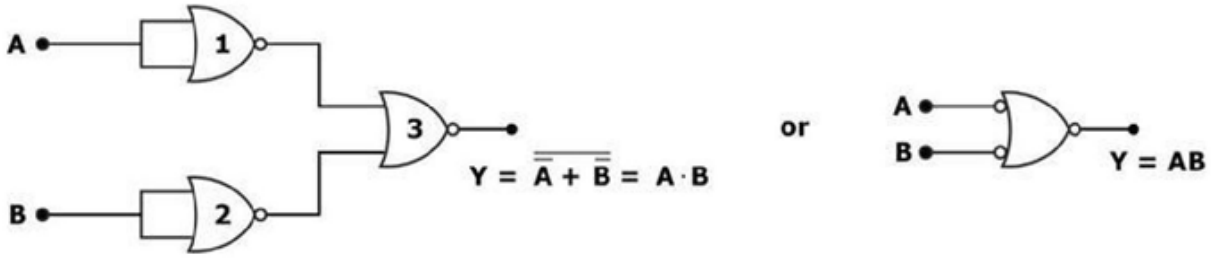
2.7.1 Using NAND gate

For the AND gate realization, we require 2 NAND gates, as shown in the circuit diagram:



2.7.2 Using NOR Gate

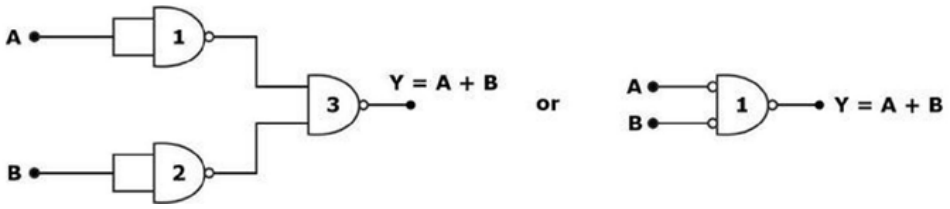
For the AND gate realization, we require 3 NOR gates if the inputs are not available in complement form, as shown in the circuit diagram:



2.8 OR Gate Realization

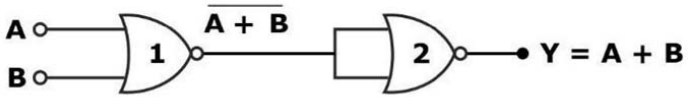
2.8.1 Using NAND Gate

For the OR gate realization, we require 3 NAND gates if the inputs are not available in complement form, as shown in the circuit diagram:



2.8.2 Using NOR Gate

For the OR gate realization, we require 2 NOR gates, as shown in the circuit diagram:



2.9 Boolean Algebra

is an algebra, which deals with binary numbers & binary variables. Hence, it is also called as Binary Algebra or logical Algebra. A mathematician, named George Boole had developed this algebra in 1854. The variables used in this algebra are also called as Boolean variables.

The range of voltages corresponding to Logic 'High' is represented with '1' and the range of voltages corresponding to logic 'Low' is represented with '0'.

Postulates and Basic Laws of Boolean Algebra

In this section, let us discuss about the Boolean postulates and basic laws that are used in Boolean algebra. These are useful in minimizing Boolean functions.

Boolean Postulates

Consider the binary numbers 0 and 1, Boolean variable (x) and its complement (x'). Either the Boolean variable or complement of it is known as literal. The four possible logical OR operations among these literals and binary numbers are shown below.

$x+0=x$

$x+1=1$

$x+x=x$

$x+x'=1$

Similarly, the four possible logical AND operations among those literals and binary numbers are shown below.

$x.1=x$

$x.0=0$

$x.x=x$

$x.x'=0$

These are the simple Boolean postulates. We can verify these postulates easily, by substituting the Boolean variable with '0' or '1'.

Note− The complement of complement of any Boolean variable is equal to the variable itself. i.e., $(x')'=x$.

2.9.1 Basic Laws of Boolean Algebra

Following are the three basic laws of Boolean Algebra.

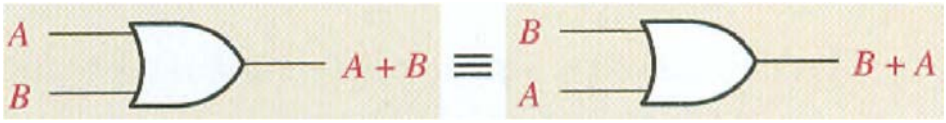
- Commutative law
- Associative law
- Distributive law

2.9.1.1 Commutative Law

If any logical operation of two Boolean variables give the same result irrespective of the order of those two variables, then that logical operation is said to be Commutative. The logical OR & logical AND operations of two Boolean Variables A & B are shown below

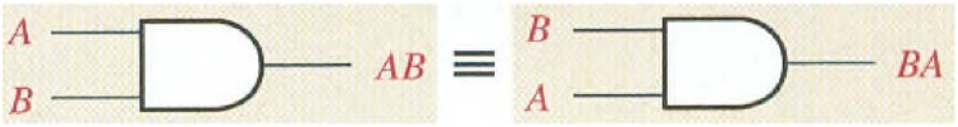
 Commutative Law of ORing:

$$A + B = B + A$$



 Commutative Law of ANDing:

$$A \cdot B = B \cdot A$$



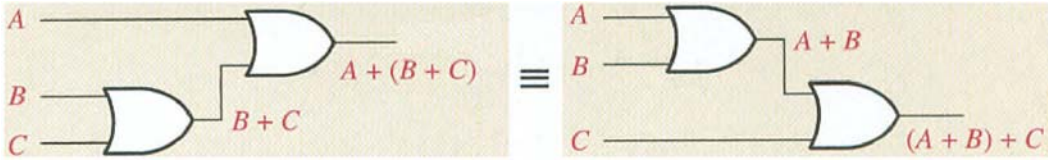
The symbol '+' indicates logical OR operation. Similarly, the symbol '.' indicates logical AND operation and it is optional to represent. Commutative law obeys for logical OR & logical AND operations.

2.9.1.2 Associative Law

If a logical operation of any two Boolean variables is performed first and then the same operation is performed with the remaining variable gives the same result, then that logical operation is said to be Associative. The logical OR & logical AND operations of three Boolean variables A, B & C are shown below.

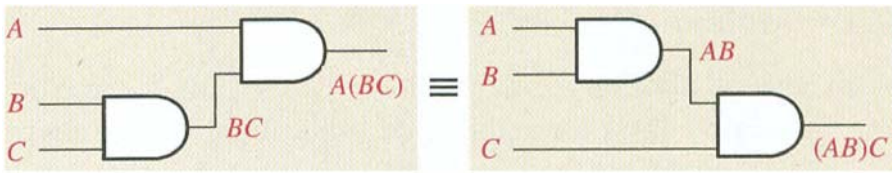
 Associative Law of ORing:

$$A + (B + C) = (A + B) + C$$



 Associative Law of ANDing:

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C$$



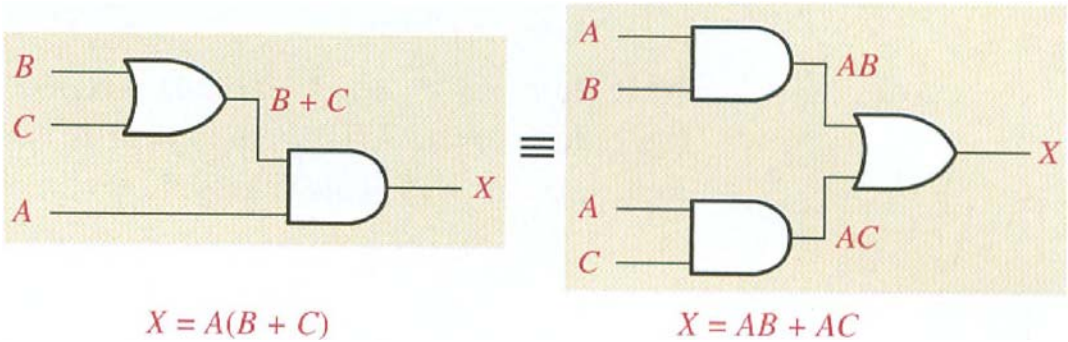
Associative law obeys for logical OR & logical AND operations.

2.9.1.3 Distributive Law

If any logical operation can be distributed to all the terms present in the Boolean function, then that logical operation is said to be Distributive. The distribution of logical OR & logical AND operations of three Boolean Variables A, B & C are shown below.

- ***Note:*** To simplify notation, the • operator is frequently omitted. When two elements are written next to each other, the AND (•) operator is implied

$$A(B + C) = AB + AC$$



Distributive law obeys for logical OR and logical AND operations.

These are the Basic laws of Boolean algebra. We can verify these laws easily, by substituting the Boolean variables with '0' or '1'.

2.9.2 Rules of Boolean Algebra

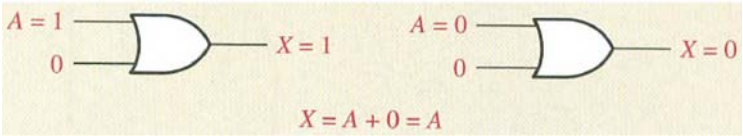
Table 2-1 lists 12 basic rules that are useful in manipulating and simplifying Boolean expressions. Rules 1 through 9 will be viewed in terms of their application to logic gates. Rules 10 through 12 will be derived in terms of the simpler rules and the laws previously

TABLE 2-1

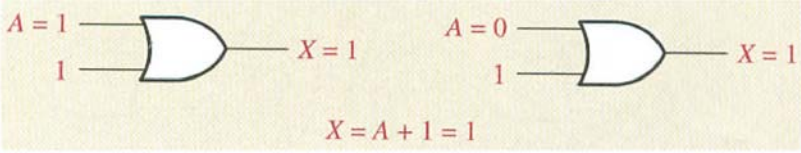
Basic rules of Boolean algebra.

1. $A + 0 = A$	7. $A \cdot A = A$
2. $A + 1 = 1$	8. $A \cdot \overline{A} = 0$
3. $A \cdot 0 = 0$	9. $\overline{\overline{A}} = A$
4. $A \cdot 1 = A$	10. $A + AB = A$
5. $A + A = A$	11. $A + \overline{A}B = A + B$
6. $A + \overline{A} = 1$	12. $(A + B)(A + C) = A + BC$

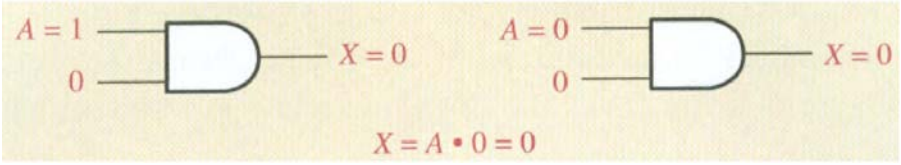
- Rule 1



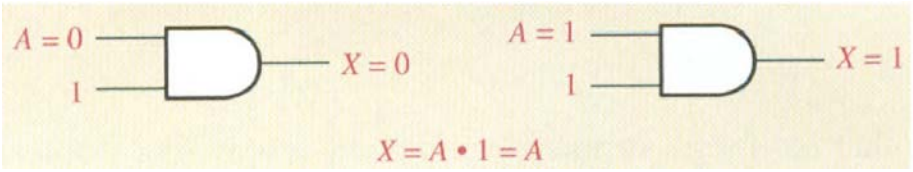
- Rule 2



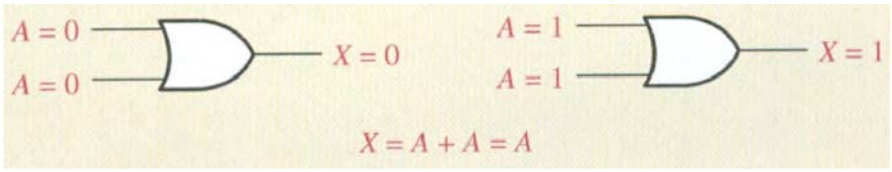
- Rule 3



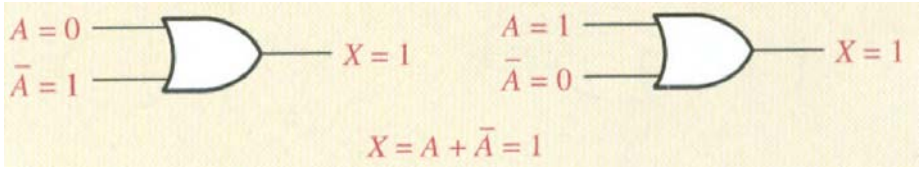
- Rule 4



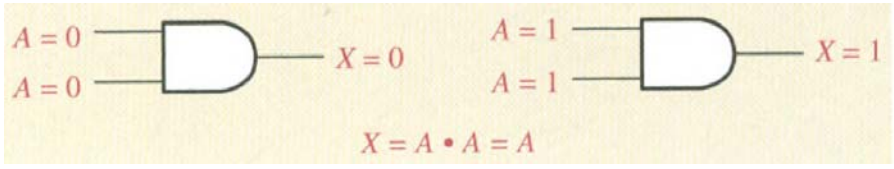
Rule 5



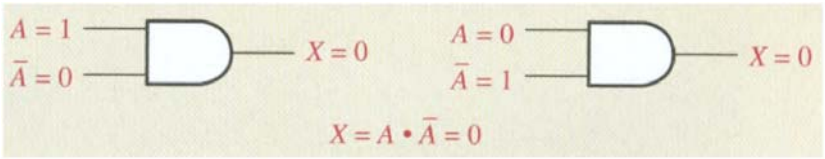
- Rule 6



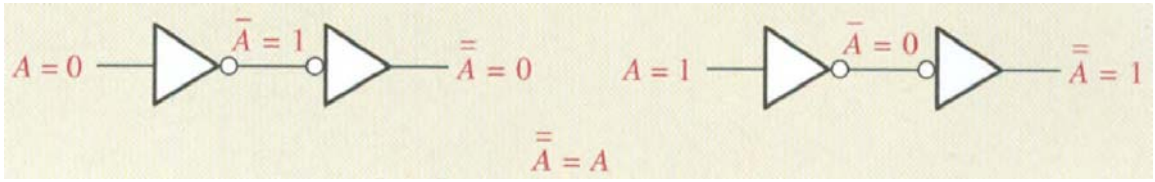
- Rule 7



- Rule 8



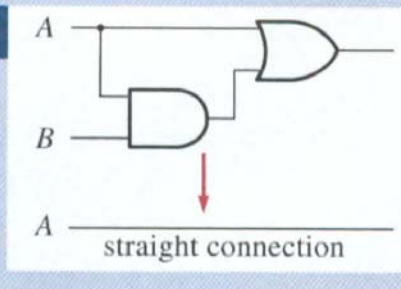
- Rule 9



Rule 10 (the absorption rule): A + AB = A

A	B	AB	A + AB
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

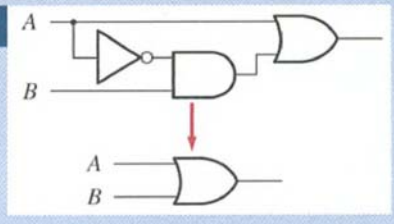
equal



- **Rule 11:** $A + \overline{A}B = A + B$

A	B	$\overline{A}B$	$A + \overline{A}B$	A + B
0	0	0	0	0
0	1	1	1	1
1	0	0	1	1
1	1	0	1	1

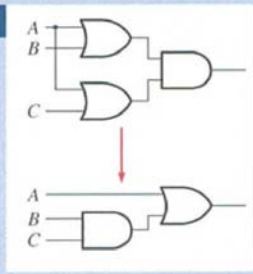
equal



 Rule 12: (A + B)(A + C) = A + BC

A	B	C	A + B	A + C	(A + B)(A + C)	BC	A + BC
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	1	1	1	0	1
1	0	1	1	1	1	0	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

equal



2.10 De Morgan's Theorems

DeMorgan, a mathematician who knew Boole, proposed two theorems that are an important part of Boolean algebra. In practical terms, DeMorgan's theorems provide mathematical verification of the equivalency of the NAND and negative-OR gates and the equivalency of the NOR and negative-AND gates.

One of DeMorgan's theorems is stated as follows:

The complement of a product of variables is equal to the sum of the complements of the variables.

Stated another way,

The complement of two or more ANDed variables is equivalent to the OR of the complements of the individual variables.

The formula for expressing this theorem for two variables is

$$\overline{XY} = \overline{X} + \overline{Y}$$

DeMorgan's second theorem is stated as follows:

The complement of a sum of variables is equal to the product of the complements of the variables.

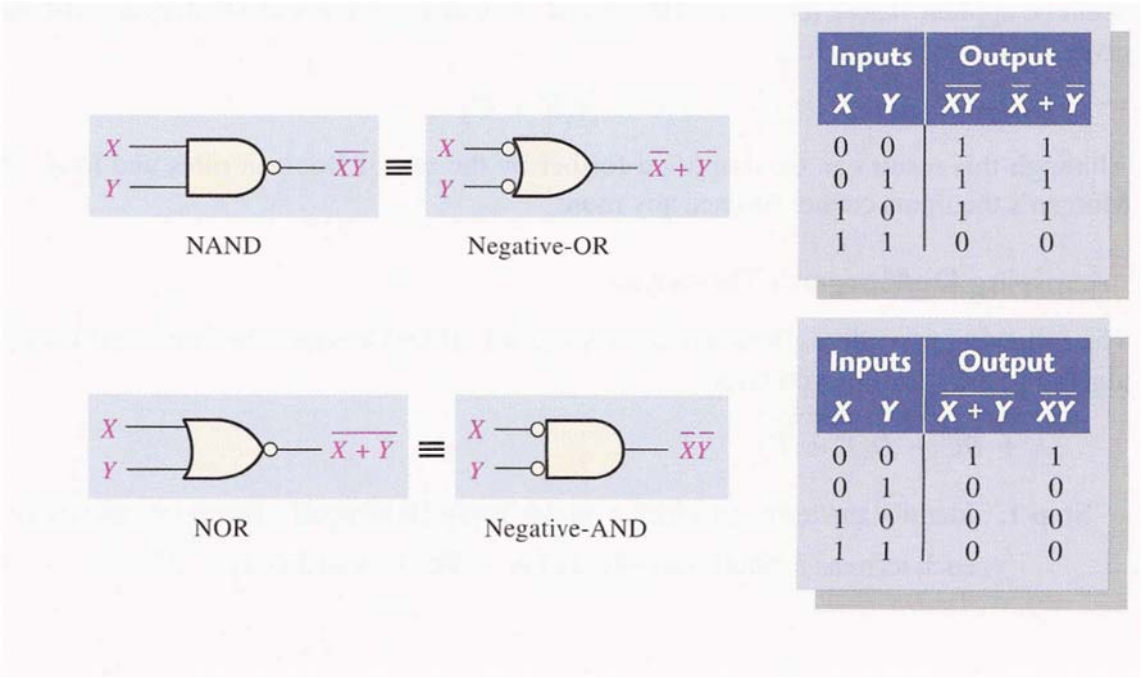
Stated another way,

The complement of two or more ORed variables is equivalent to the AND of the complements of the individual variables.

The formula for expressing this theorem for two variables is

$$\overline{X + Y} = \overline{X}\,\overline{Y}$$

The following Figure shows the gate equivalencies and truth tables for above Equations

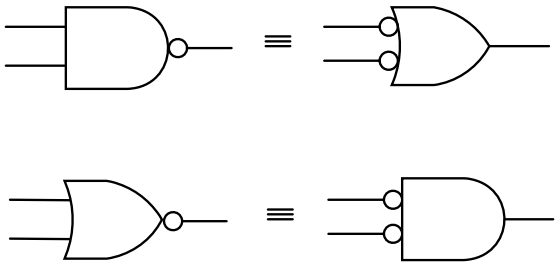


Inputs		Output	
X	Y	$\overline{XY}$	$\overline{X} + \overline{Y}$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

Inputs		Output	
X	Y	$\overline{X + Y}$	$\overline{\overline{X}\,\overline{Y}}$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

$$\overline{A.B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A}.\overline{B}$$



As stated, DeMorgan's theorems also apply to expressions in which there are more than two variables. The following examples illustrate the application of DeMorgan's theorems to 3-variable and 4-variable expressions. General:

$$\overline{A.B.C.D} = \overline{A} + \overline{B} + \overline{C} + \overline{D}$$
$$\overline{A + B + C + D} = \overline{A}.\overline{B}.\overline{C}.\overline{D}$$

De Morgan's Theorems Example

$$\overline{(A.B + C)(A + B.C)} =$$
$$\overline{(A.B + C)} + \overline{(A + B.C)} =$$
$$(\overline{A.B}.\overline{C}) + (\overline{A}.\overline{B.C}) =$$
$$(\overline{A} + \overline{B}).\overline{C} + \overline{A}.(\overline{B} + \overline{C}) =$$
$$\overline{A}.\overline{C} + \overline{B}.\overline{C} + \overline{A}.\overline{B} + \overline{A}.\overline{C} =$$
$$\overline{A}.\overline{C} + \overline{B}.\overline{C} + \overline{A}.\overline{B}$$

2.11 Simplification Using Boolean Algebra

Many times in the application of Boolean algebra, you have to reduce a particular expression to its simplest form or change its form to a more convenient one to implement the expression most efficiently.

- Apply the laws, rules, and theorems of Boolean algebra to simplify general expressions

EXAMPLE

Using Boolean algebra techniques, simplify this expression:

$$AB + A(B + C) + B(B + C)$$

Solution The following is not necessarily the only approach.

Step 1: Apply the distributive law to the second and third terms in the expression, as follows:

$$AB + AB + AC + BB + BC$$

Step 2: Apply rule 7 *(BB = B)* to the fourth term.

$$AB + AB + AC + B + BC$$

Step 3: Apply rule *5 (AB + AB = AB)* to the first two terms.

$$AB + AC + B + BC$$

Step 4: Apply rule 10 *(B + BC = B)* to the last two terms.

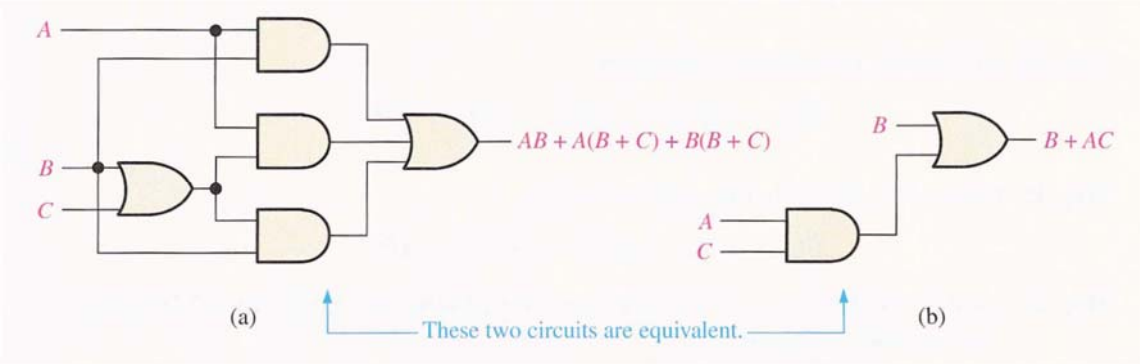
$$AB + AC + B$$

Step 5: Apply rule 10 *(AB + B = B)* to the first and third terms.

$$B + AC$$

At this point the expression is simplified as much as possible. Once you gain experience in applying Boolean algebra, you can often combine many individual steps.

The following Figures shows that the simplification process has significantly reduced the number of logic gates required to implement the expression. Part (a) shows that five gates are required to implement the expression in its original form; however, only two gates are needed for the simplified expression, shown in part (b). It is important to realize that these two gate circuits are equivalent. That is, for any combination of levels on the A, B, and C inputs, you get the same output from either circuit (Simplification means fewer gates for the same function)



EXAMPLE

Simplify the following Boolean expression:

$$[AB(C + BD) + AB]C$$

Note that brackets and parentheses mean the same thing: the term inside is multiplied
(ANDed) with the term outside.

Solution

Step 1: Apply the distributive law to the terms within the brackets.

$$(ABC + ABBD + A\ B)C$$

Step 2: Apply rule 8 $(BB = 0)$ to the second term within the parentheses.

$$(ABC + A \bullet O \bullet D + AB)C$$

Step 3: Apply rule 3 $(A \bullet 0 \bullet D = 0)$ to the second term within the parentheses.

$$(ABC + 0 + AB)C$$

Step 4: Apply rule 1 (drop the 0) within the parentheses.

$$(ABC + AB)C$$

Step 5: Apply the distributive law.

$$ABCC + ABC$$

Step **6:** Apply rule 7 $(CC = C)$ to the first term.

$$ABC + A\ BC$$

Step 7: Factor out *BC*.

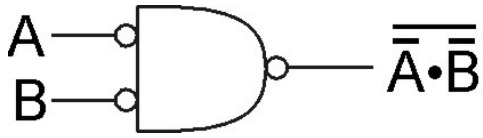
$$BC(A + A)$$

Step **8:** Apply rule 6 ($A + A = 1$).

$$BC \bullet 1$$

2.12 Converting AND to OR

- Using De Morgan's Theorems, AND can be converted to OR (with some help from NOT)
- Consider the following gate:



To convert AND to OR (or vice versa), invert inputs and output.

A	B	$\overline{A}$	$\overline{B}$	$\overline{A} \cdot \overline{B}$	$\overline{\overline{A} \cdot \overline{B}}$
0	0	1	1	1	0
0	1	1	0	0	1
1	0	0	1	0	1
1	1	0	0	0	1

Same as A+B

2.13 Standard Forms

The two canonical forms of Boolean algebra are basic forms that one obtains from reading a given function from the truth table. These forms are very seldom the ones with the least number of literals, because each minterm or maxterm must contain, by definition, *all* the variables, either complemented or uncomplemented.

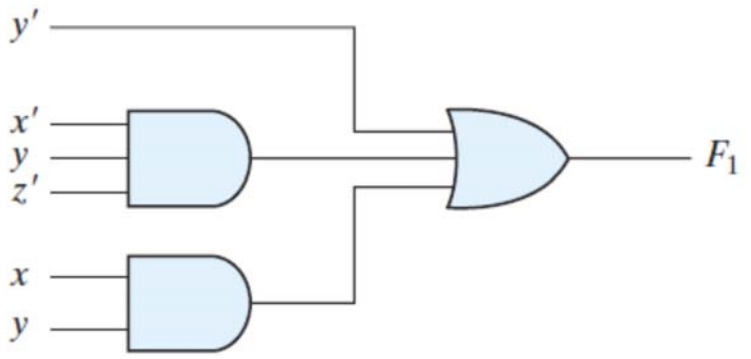
Another way to express Boolean functions is in *standard* form. In this configuration, the terms that form the function may contain one, two, or any number of literals. There are two types of standard forms: the sum of products and products of sums.

The *sum of products* is a Boolean expression containing AND terms, called *product terms,* with one or more literals each. The *sum* denotes the ORing of these terms. An example of a function expressed as a sum of products is

$$F1 = y' + xy + x'yz'$$

The expression has three product terms, with one, two, and three literals. Their sum is, in effect, an OR operation.

The logic diagram of a sum-of-products expression consists of a group of AND gates followed by a single OR gate. This configuration pattern is shown in Figure. Each product term requires an AND gate, except for a term with a single literal. The logic sum is formed with an OR gate whose inputs are the outputs of the AND gates and the single literal. It is assumed that the input variables are directly available in their complements, so inverters are not included in the diagram. This circuit configuration is referred to as a *two-level implementation*.

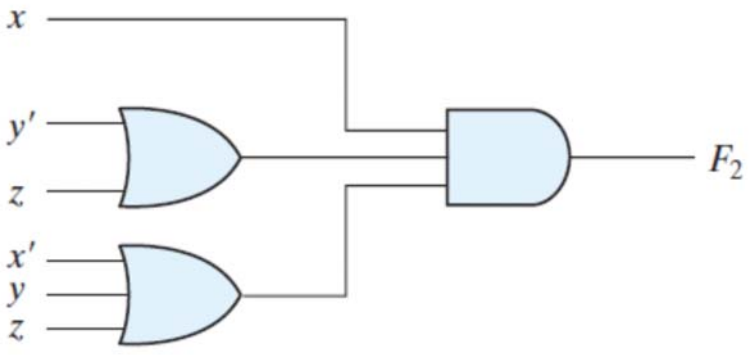


Sum of products

A *product of sums* is a Boolean expression containing OR terms, called *sum terms*. Each term may have any number of literals. The *product* denotes the ANDing of these terms. An example of a function expressed as a product of sums is

$$F2 = x(y' + z)(x' + y + z')$$

This expression has three sum terms, with one, two, and three literals. The product is an AND operation. The use of the words *product* and *sum* stems from the similarity of the AND operation to the arithmetic product (multiplication) and the similarity of the OR operation to the arithmetic sum (addition). The gate structure of the product-of-sums expression consists of a group of OR gates for the sum terms (except for a single literal), followed by an AND gate, as shown in Figure. This standard type of expression results in a two-level structure of gates**.**



Product of sums

2.13.1 Sum-of-Products Expression

- A *literal* is a variable or the complement of a variable.

  Examples: $\mathbf{X}$, $\mathbf{Y}$, $\overline{\mathbf{X}}$, $\overline{\mathbf{Y}}$

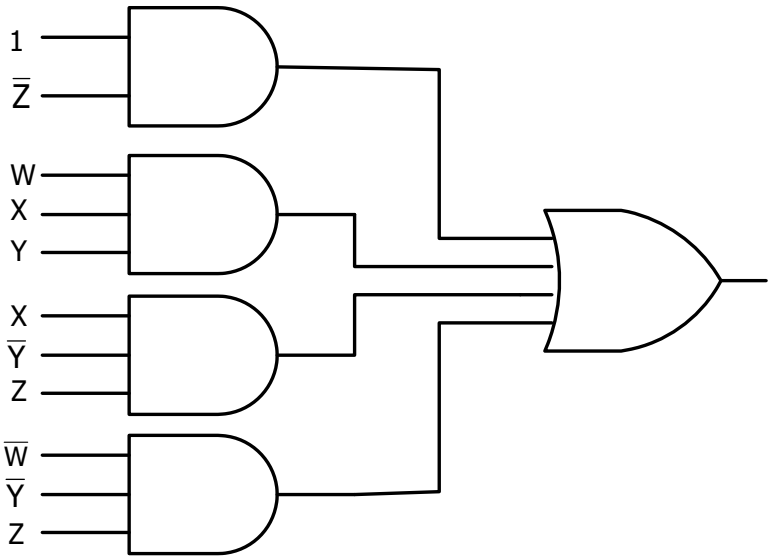
- A *product term* is a single literal or a logical product of two or more literals. Examples: $\overline{\mathbf{Z}}$, $\mathbf{W.X.Y}$, $\mathbf{X.\overline{Y}.Z}$, $\overline{\mathbf{W}}.\overline{\mathbf{Y}}.\mathbf{Z}$

- A *sum-of-products* (SOP) expression is a logical sum of product terms.

  Example: $\overline{\mathbf{Z}} + \mathbf{W.X.Y} + \mathbf{X.\overline{Y}.Z} + \overline{\mathbf{W}}.\overline{\mathbf{Y}}.\mathbf{Z}$

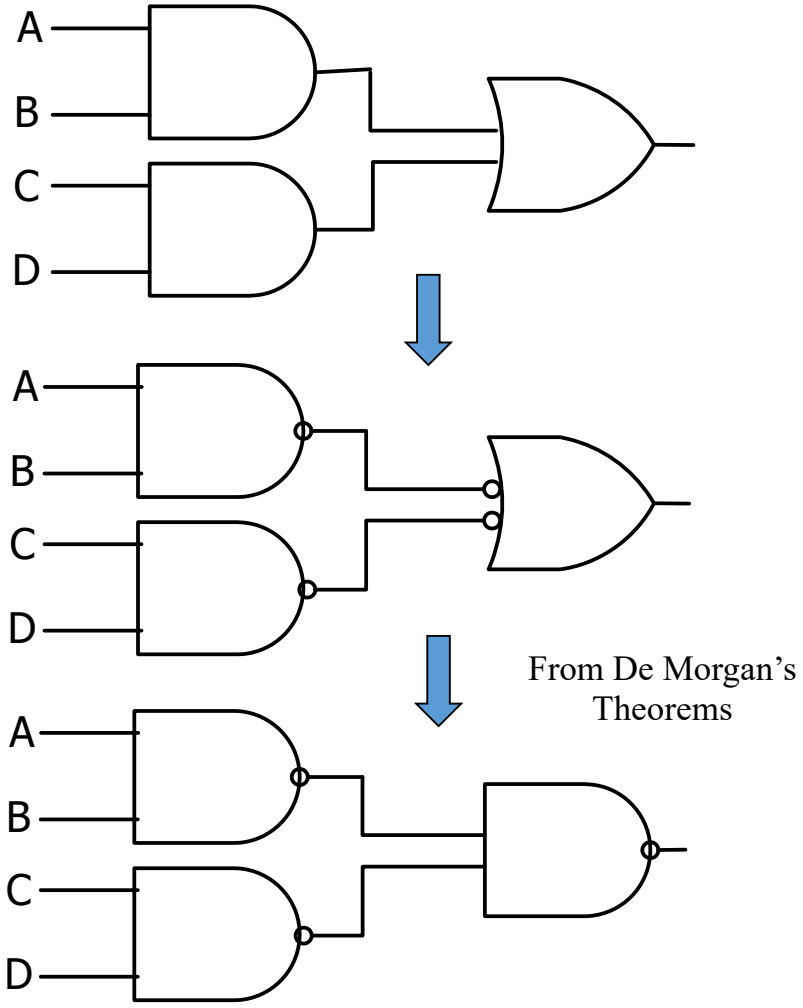
- Every SOP expression can be realized by a two-level circuit containing AND gates followed by an OR gate

-



2.13.2 SOP with NANDs

A SOP expression can be implemented with only NAND gates as shown in Figure:



2.14 Definition of Boolean Functions

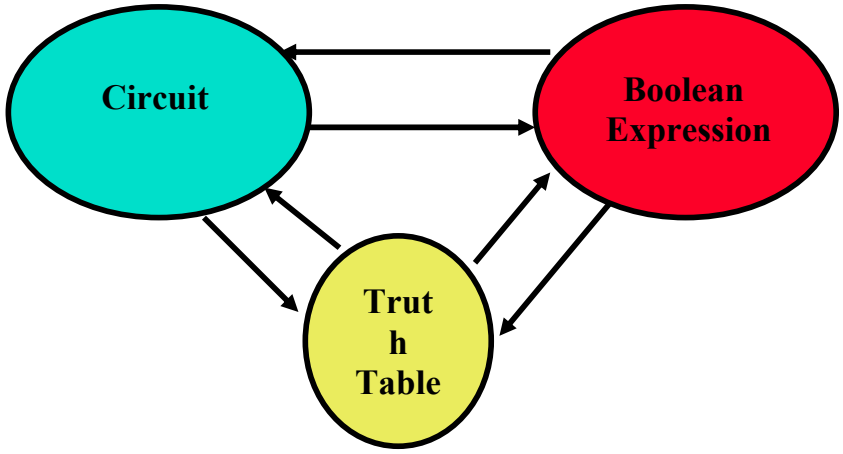
A Boolean function is a function that takes one or more Boolean inputs and produces a Boolean output. It can be represented using Boolean algebra, which involves operations such as AND, OR, NOT, NAND, NOR, XOR, and XNOR.

2.14 Representation of Boolean Functions

- **Truth Tables**: List all possible values of the variables and the corresponding function values.
- **Boolean Expressions**: Use algebraic expressions with Boolean variables and operators.
- **Circuit diagram:** A Boolean function is represented by a circuit diagram using interconnected logic gates (AND, OR, NOT, etc.).

2.15 Function Representation Conversion

- Need to transit between a Boolean expression, a truth table, and a circuit (symbols)
- All three formats are equivalent



Minterm

- For each truth-table row a product term can be defined that evaluates to 1 only when the inputs have the values listed in that row.
- If the product term contains each input variable exactly once it is called a minterm

Row	A	B	C	Minterm
0	0	0	0	$\overline{A}.\overline{B}.\overline{C}$
1	0	0	1	$\overline{A}.\overline{B}.C$
2	0	1	0	$\overline{A}.B.\overline{C}$
3	0	1	1	$\overline{A}.B.C$
4	1	0	0	$A.\overline{B}.\overline{C}$
5	1	0	1	$A.\overline{B}.C$
6	1	1	0	$A.B.\overline{C}$
7	1	1	1	$A.B.C$

Converting to a Circuit

2.15.1 Converting Truth Table to Boolean Expression

- Any logic function can be expressed algebraically by taking the **OR** of all those **minterms** corresponding to the truth-table rows for which the function produces a 1 output.
- *Example:* Majority detector.

Row	A	B	C	Minterm	R
0	0	0	0	A.B.C	0
1	0	0	1	A.B.C	0
2	0	1	0	A.B.C	0
3	0	1	1	A.B.C	1
4	1	0	0	A.B.C	0
5	1	0	1	A.B.C	1
6	1	1	0	A.B.C	1
7	1	1	1	A.B.C	1

$$R=\bar{A}.B.C+ A.\bar{B}.C+ A.B.\bar{C}+ A.B.C$$

Alternate forms:

$$R = m_3 + m_5 + m_6 + m_7$$

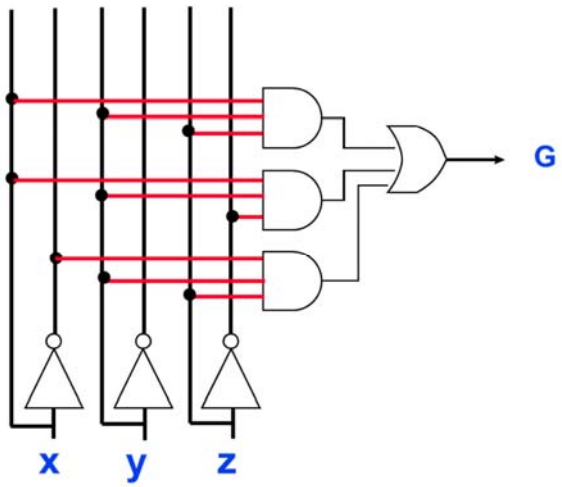
$$R = \sum (3, 5, 6, 7)$$

2.15.2 Converting to a Circuit

- Number of 1's in truth table output column equals AND terms for Sum-of-Products (SOP)

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

G = xyz + xyz' + x'yz



Chapter 3: Simplification of Boolean Functions

3.1 Introduction

Gate-level minimization is the design task of finding an optimal gate-level implementation of the Boolean functions describing a digital circuit. The complexity of the digital logic gates that implement a Boolean function is directly related to the complexity of the algebraic expression from which the function is implemented. Boolean expressions may be simplified by Boolean algebraic means as discussed in previous chapter. However, this minimization method is difficult because it has no clear rules to guide each step. The map method presented here provides a simple, straightforward procedure for minimizing Boolean functions. This method may be regarded as a pictorial form of a truth table. The map method is also known as the *Karnaugh map* or *K-map*.

3.2 Karnaugh Maps

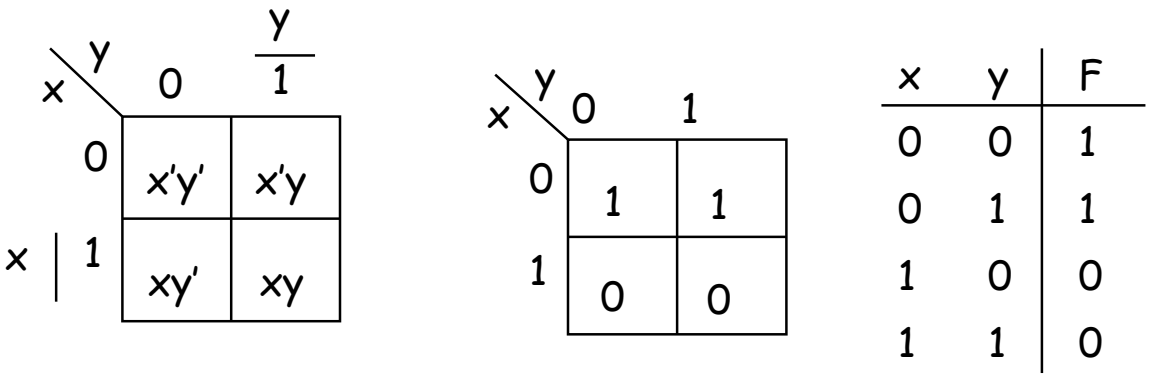
A K-map is a diagram made up of squares, with each square representing one minterm of the function that is to be minimized. Since any Boolean function can be expressed as a sum of minterms, it follows that a Boolean function is recognized graphically in the map from the area enclosed by those squares whose minterms are included in the function.

- Karnaugh Maps are an alternate way of representing Boolean functions
  - Each row in the truth table is represented by a square
  - Each square represents a *minterm*
- Easy to convert between truth table, K-map, and SOP as shown in figure
  - Un-optimized form: number of 1's in K-map equals number of minterms (products) in SOP
  - Optimized form: reduced number of minterms

- A Karnaugh map is a graphical tool for assisting in the general simplification procedure

Example represent the following Boolean expression using Karnaugh Maps

$$F = \Sigma(m_0,m_1) = x'y' + x'y$$



3.2.1 Two-Variable K-Map

The two-variable map is shown in Fig 3.1(a). There are four minterms for two variables; hence, the map consists of four squares, one for each minterm. The map is redrawn in (b) to show the relationship between the squares and the two variables *x* and *y*. The 0 and 1 marked in each row and column designate the values of variables. Variable *x* appears primed in row 0 and unprimed in row 1. Similarly, *y* appears primed in column 0 and unprimed in column 1.

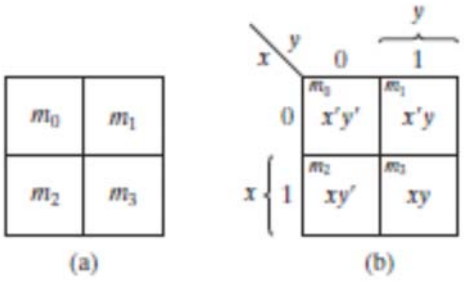


Figure 3.1 Two-variable K-map

If we mark the squares whose minterms belong to a given function, the two-variable map becomes another useful way to represent any one of the Boolean functions of two variables. As an example, the function *xy* is shown in Fig. 3.2(a). Since *xy* is equal to *m*3, a 1 is placed inside the square that belongs to *m*3. Similarly, the function $x + y$ is represented in the map of Fig. 3.2 (b) by three squares marked with 1's. These squares are found from the minterms of the function:

$$m1 + m2 + m3 = x'y + xy' + xy = x + y$$

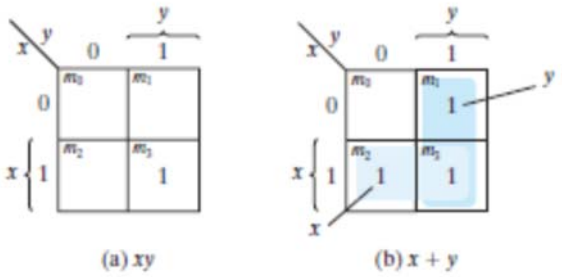


Figure 3.2 Representation of functions in the map

3.2.2 Three-Variable K-Map

A three-variable K-map is shown in Fig. 3.3. There are eight minterms for three binary variables; therefore, the map consists of eight squares. Note that the minterms are arranged, not in a binary sequence, but in a sequence similar to the Gray code. The characteristic of this sequence is that only one bit changes in value from one adjacent column to the next. The map drawn in part (b) is marked with numbers in each row and each column to show the relationship between the squares and the three variables. For example, the square assigned to m5 corresponds to row 1 and column 01. When these two numbers are concatenated, they give the binary number 101, whose decimal equivalent is 5. Each cell of the map corresponds to a unique minterm, so another way of looking at square m5 = xy'z is to consider it to be in the row marked x and the column belonging to y'z (column 01).

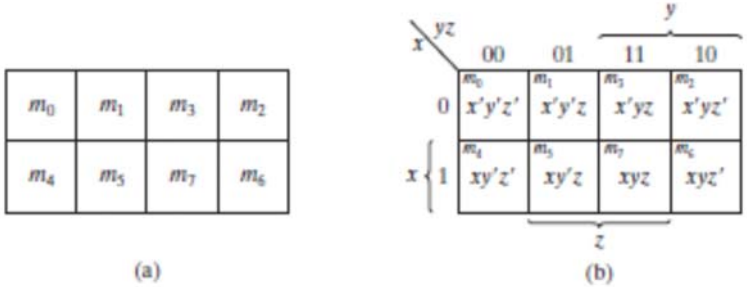
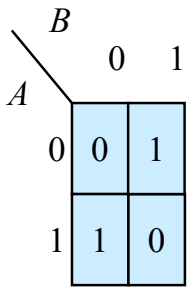


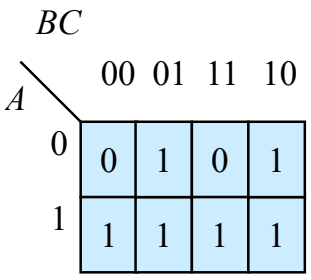
Figure 3.3 Three-variable K-map

Example: Express this function as a sum of minterms.

- Two variable maps



$$F=\overline{A}\,B + A\,\overline{B}$$



$$F=\overline{A}\,\overline{B}\,C + \overline{A}\,B\,\overline{C} + A\,\overline{B}\,\overline{C} + A\,\overline{B}\,C + A\,B\,C + A\,B\,\overline{C}$$

3.2.3 Rules for K-Maps

- We can reduce functions by circling 1's in the K-map
- Each circle represents minterm reduction
- After circling, deduce a minimized AND-OR form

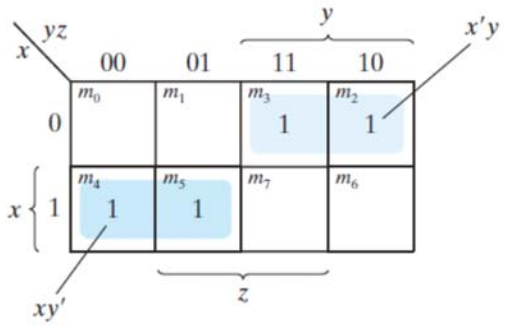
Rules to consider

- Every cell containing a 1 must be included at least once

- The largest possible “power of 2 rectangle” should be used
- Use the smallest possible number of rectangles

Example 1: Simplify the Boolean function using K-map

$$F(x, y, z) = \Sigma(2, 3, 4, 5)$$

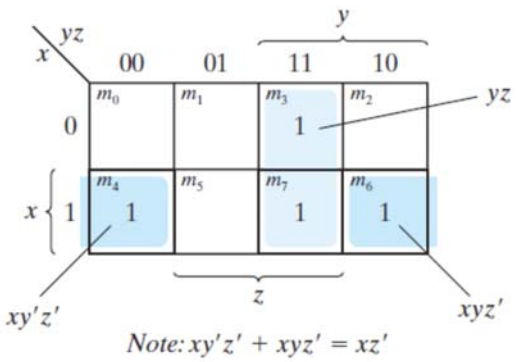


The simplified function then becomes

$$F(x, y, z) = \Sigma(2, 3, 4, 5) = \boldsymbol{x'y} + \boldsymbol{xy'}$$

Example 2: Simplify the Boolean function using K-map

$$F(x, y, z) = \Sigma(3, 4, 6, 7)$$



Then, the simplified function then becomes

$$F(x, y, z) = \Sigma(3, 4, 6, 7) = \boldsymbol{yz} + \boldsymbol{xz'}$$

3.3 Four-Variable K-Map

The map for Boolean functions of four binary variables (w, x, y, z) is shown in Fig. 3.8. In Fig. 3.8(a) are listed the 16 minterms and the squares assigned to each. In Fig. 3.8(b), the map is redrawn to show the relationship between the squares and the four variables. The rows and columns are numbered in a Gray code sequence, with only one digit changing value between two adjacent rows or columns. The minterm corresponding to each square can be obtained from the concatenation of the row number with the column number. For example, the numbers of the third row (11) and the second column (01), when concatenated, give the binary number 1101, the binary equivalent of decimal 13. Thus, the square in the third row and second column represents minterm m13

The map minimization of four-variable Boolean functions is similar to the method used to minimize three-variable functions.

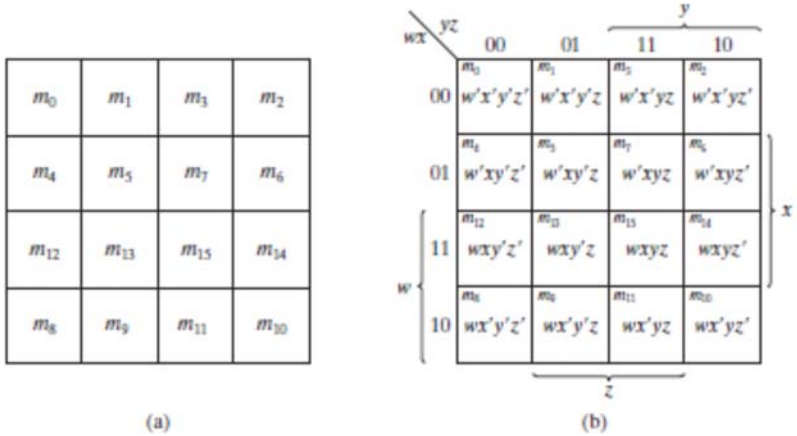


Figure 3.3 Four-variable map

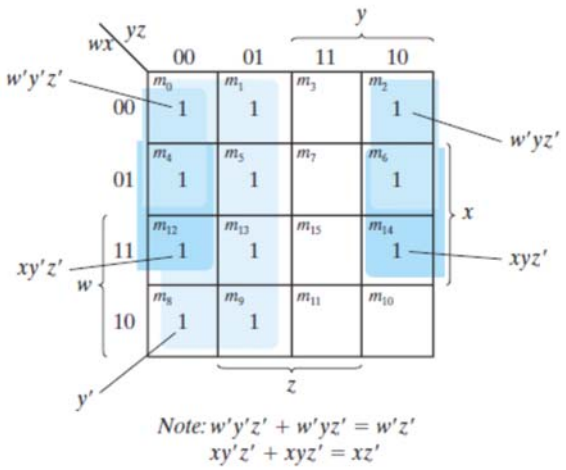
The combination of adjacent squares that is useful during the simplification process is easily determined from inspection of the four-variable map:

1. One square represents one minterm, giving a term with four literals.
2. Two adjacent squares represent a term with three literals.
3. Four adjacent squares represent a term with two literals.
4. Eight adjacent squares represent a term with one literal.

5. Sixteen adjacent squares produce a function that is always equal to 1.

Example 1: Simplify the Boolean function using K-map

$$F(w, x, y, z) = \Sigma\,(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$

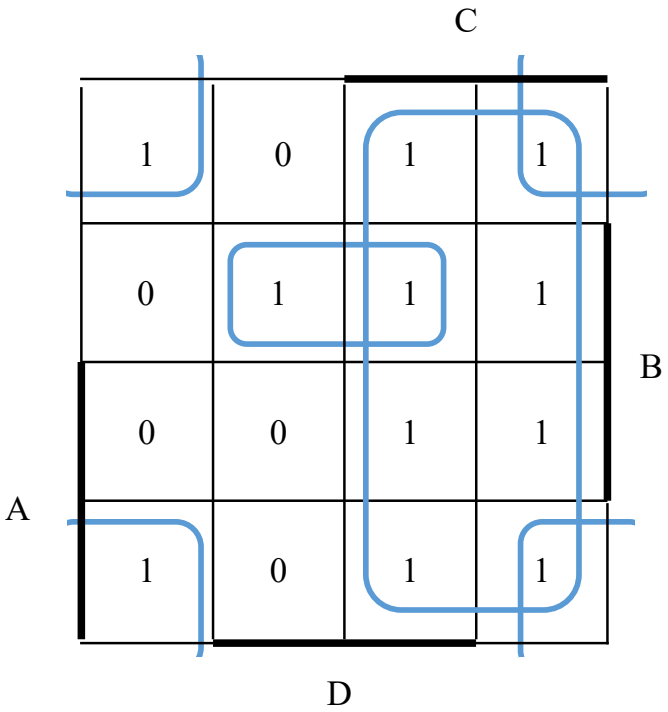


Then, the simplified function then becomes

$$F(w, x, y, z) = \Sigma\,(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14) = \quad \boldsymbol{y' + w'z' + xz'}$$

Example 2: Simplify the Boolean function using K-map

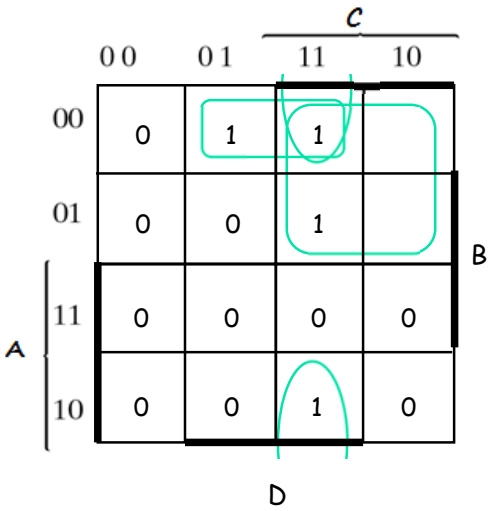
$$F(A, B, C, D) = \Sigma\,(0, 2, 3, 5, 6, 7, 8, 10, 11, 14, 15)$$



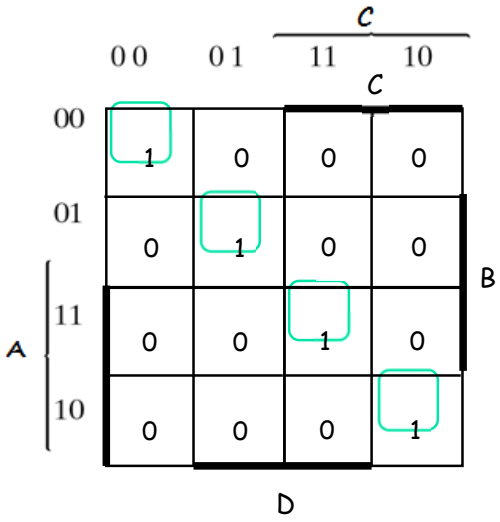
Then, the simplified function then becomes

F (A, B, C, D) = C + A'BD + B' D'

K-Map Design Examples: Simplify the following Boolean functions using K-map and the draw the physical implementation with gates the EQ Boolean function



K-map for LT



K-map for EQ

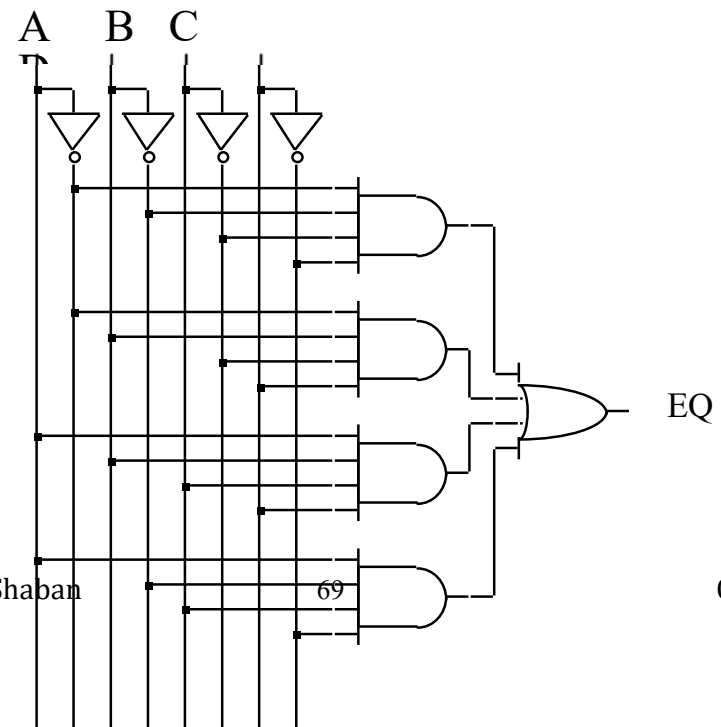
Physical Implementation

Step 1: Truth table

Step 2: K-map

Step 3: Minimized sum-of-products

Step 4: Physical implementation with gates



3.4 Don't-Care Conditions

The logical sum of the minterms associated with a Boolean function specifies the conditions under which the function is equal to 1. The function is equal to 0 for the rest of the minterms. This pair of conditions assumes that all the combinations of the values for the variables of the function are valid. In practice, in some applications the function is not specified for certain combinations of the variables. As an example, the four-bit binary code for the decimal digits has six combinations that are not used and consequently are considered to be unspecified. Functions that have unspecified outputs for some input combinations are called *incompletely specified functions* . In most applications, we simply don't care what value is assumed by the function for the unspecified minterms. For this reason, it is customary to call the unspecified minterms of a function *don't-care conditions* . These don't-care conditions can be used on a map to provide further simplification of the Boolean expression.

Example 3: Simplify the Boolean function using K-map

$$F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$$

which has the don't-care conditions

$$d(w, x, y, z) = \Sigma(0, 2, 5)$$

The minterms of F are the variable combinations that make the function equal to 1. The minterms of d are the don't-care minterms that may be assigned either 0 or 1. The map simplification is shown in Fig. 3.15 . The minterms of F are marked by 1's, those of d are marked by X's, and the remaining squares are filled with 0's. To get the simplified expression in sum-of-products form, we must include all five 1's in the map, but we may or may not include any of the X's, depending on the way the function is simplified. The term yz covers the four minterms in the third column. The remaining minterm, $m1$, can be combined with minterm $m3$ to give the three-literal term $w'x'z$. However, by including one or two adjacent X's we can combine four adjacent squares to give a two-literal term. In Fig. 3.4(a),

don’t-care minterms 0 and 2 are included with the 1’s, resulting in the simplified function

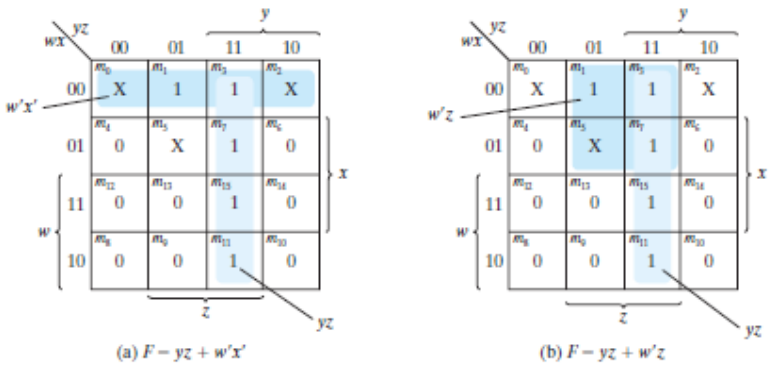


Figure 3.4 Example with don’t-care conditions

$$F = yz + w'x'$$

In Fig. 3.4(b), don’t-care minterm 5 is included with the 1’s, and the simplified function is now

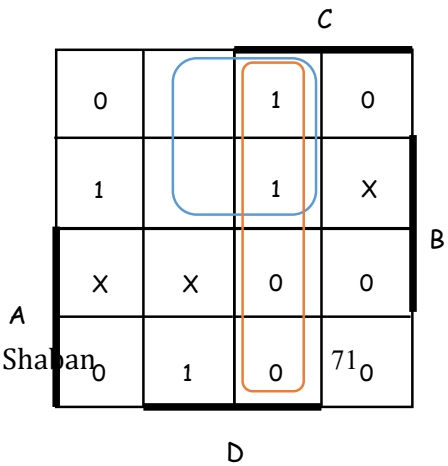
$$F = yz + w'z$$

Either one of the preceding two expressions satisfies the conditions stated for this example.

Don’t Cares Examples

f(A,B,C,D) = Σ (1,3,5,7,9) + d Σ (6,12,13)

- f = A'D + B'C'D without don't cares
- f = A'D + C'D with don't cares



Notice that: don't cares can be treated as 1s or 0s depending on which is more advantageous

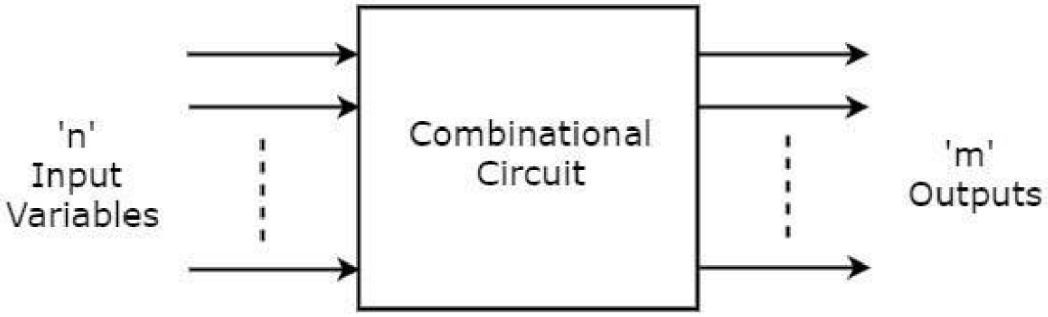
3.5 Definition of Terms

- Implicant
  - Single product term of the ON-set (terms that create a logic 1)
- Prime implicant
  - Implicant that can't be combined with another to form an implicant with fewer literals
- Essential prime implicant
  - Prime implicant is essential if it alone covers a minterm in the K-map
  - Remember that all squares marked with 1 must be covered
- Objective:
  - Group implicants into prime implicants (minimize literals per term)
  - Cover the K-map with as few prime implicants as possible (minimize number of product terms)

Chapter4: Combinational Circuits (Analysis & Design)

4.1 Introduction

Combinational circuits consist of Logic gates. These circuits operate with binary values. The output(s) of combinational circuit depends on the combination of present inputs. The following figure shows the block diagram of combinational circuit. combinational circuit also called Memoryless circuits.



This combinational circuit has ‘n’ input variables and ‘m’ outputs. Each combination of input variables will affect the output(s).

4.2 Combinational Circuits (Analysis & Design)

The combinational Circuits can be classified as:

- Combinational Circuit analysis:
  - In the analysis we are interested in determining the function of a given combination circuit.
- Combinational Circuit design:
  - In the design we are interested in developing a combinational circuit based on a given function.

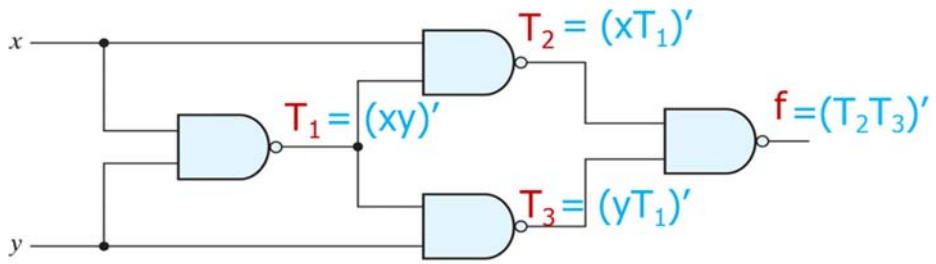
4.2.1 Combinational Circuit Analysis

Combinational Circuit analysis can be achieved by either one of the following two methods:

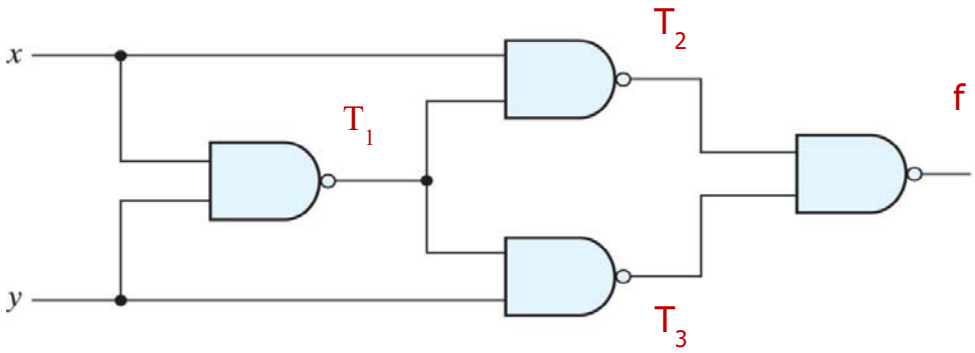
- Determining the output functions as algebraic expressions.
- Determining the truth table of the output functions.

4.2.2 Circuit Analysis steps

1. Label all gate outputs with symbols.
2. Determine Boolean function at the output of each gate.
3. Express functions in terms of input variables



Example 1: Find the output function of the following circuit.



Step1 : put a label T1, T2, T3, and f

Step2 : $T_1 = (xy)'$, $T_2 = (xT1)'$, $T_3 = (yT_1)'$, $f = (T_2T_3)'$

Step3 :

$$f = (T_2T_3)'$$

$$= ((xT_1)'\,(yT_1)')'$$

$= (xT_1)+(yT_1)$

$= x(xy)'+y(xy)'$

$= x(x'+y')+y(x'+y')$

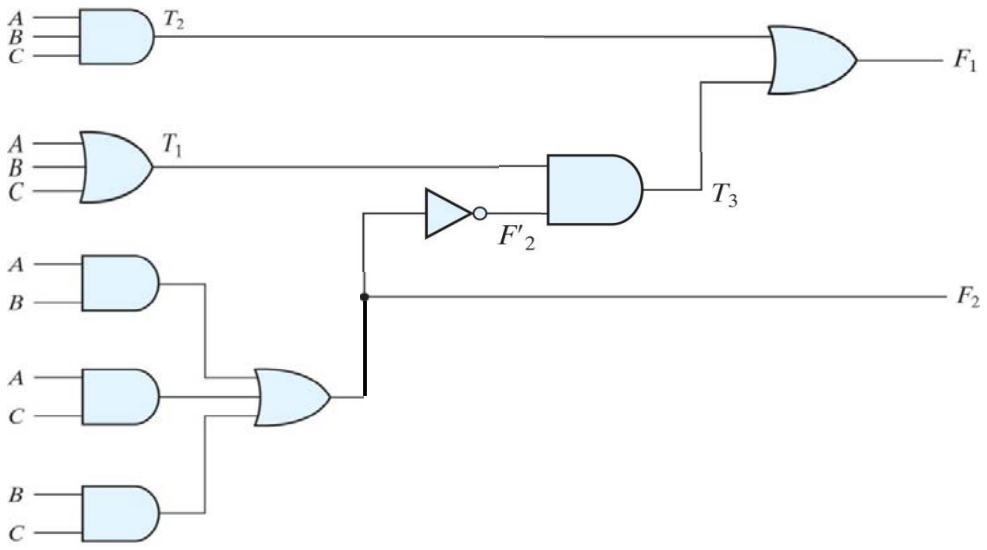
$= xy' + x'y$

$= x \oplus y$

Solution using Truth Table

x	y	$T_1=(xy)'$	$T_2=(xT_1)'$	$T_3=(yT_1)'$	$f=(T_2T_3)'$
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	0

Example 2: Find the functions of the following Circuit using Truth Table



A	B	C	F_2	F'_2	T_1	T_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

4.3 Combinational Circuit Design

Four main steps in designing a combinational circuit:

1. Determine the number of inputs and outputs and give them symbols.
2. Derive the related Truth Table. If there are 'n' input variables, then there will be 2^n possible combinations. For each combination of input, find the output values.
3. Simplify each output. Then, find the **Boolean expressions** for each output
4. Draw logic diagram and verify correctness. Implement the above Boolean expressions corresponding to each output by using **Logic gates.**

Example 1: Design a combinational logic circuit that detects odd inputs which are in the range from 0 to 7.

Solution:

1. Step 1: 3 inputs (A, B, and C) and one output (F).
2. Step 2: Truth table

3. Step 3: Simplify

A\BC	00	01	11	10
0	0	1	1	0
1	0	1	1	0

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

4. Step 4: F = C

4.4 Binary Adder

- The most basic arithmetic operation is addition. The circuit, which performs the addition of two binary numbers is known as **Binary adder.** First, let us implement an adder, which performs the addition of two bits.

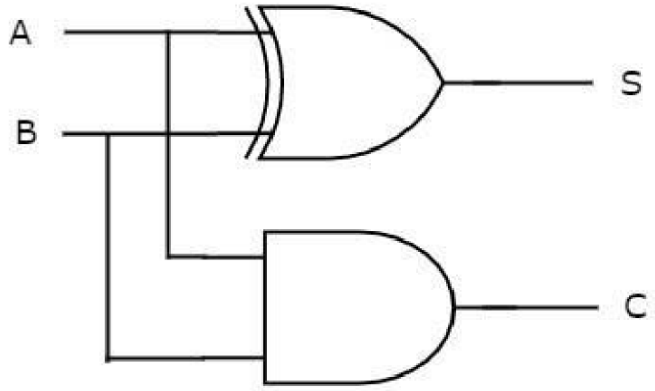
4.4.1 Half Adder

- Half adder is a combinational circuit, which performs the addition of two binary numbers A and B are of **single bit.** It produces two outputs sum, S & carry, C.
- The **Truth table** of Half adder is shown below.
- When we do the addition of two bits, the resultant sum can have the values ranging from 0 to 2 in decimal. We can represent the decimal digits 0 and 1 with single bit in binary. But, we can't represent decimal digit 2 with single bit in binary. So, we require two bits for representing it in binary.

Inputs		Outputs	
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Let, sum, S is the Least significant bit and carry, C is the Most significant bit of the resultant sum. For first three combinations of inputs, carry, C is zero and the value of S will be either zero or one based on the **number of ones** present at the inputs. But, for last combination of inputs, carry, C is one and sum, S is zero, since the resultant sum is two.

- From Truth table, we can directly write the **Boolean functions**for each output as
    - $S = A \oplus B = A \oplus B$
    - C= AB
- We can implement the above functions with 2-input Ex-OR gate & 2-input AND gate. The **circuit diagram** of Half adder is shown in the following figure.



- In the above circuit, a two input Ex-OR gate & two input AND gate produces sum, S & carry, C respectively. Therefore, Half-adder performs the addition of two bits.

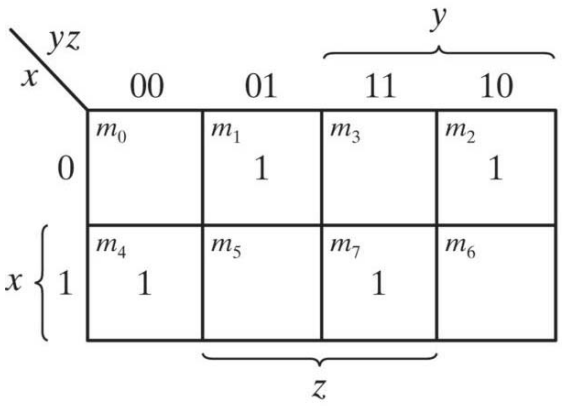
4.4.2 Full Adder

- Full adder is a combinational circuit, which performs the **addition of three bits** x, y and carry bit z. Where, x & y are the two parallel significant bits and z is the carry bit, which is generated from previous stage. This Full adder also produces two outputs sum, S & carry, Cout, which are similar to Half adder.
- The **Truth table** of Full adder is shown below.

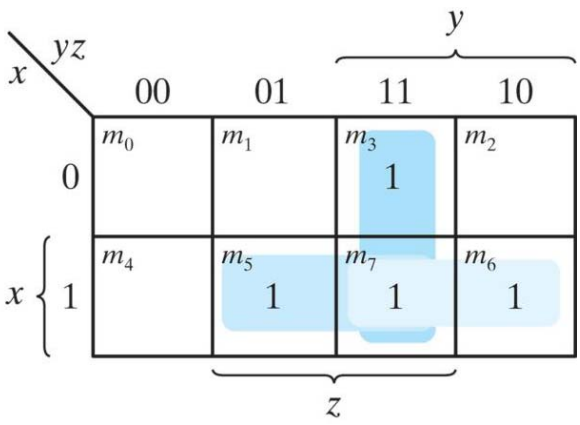
Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

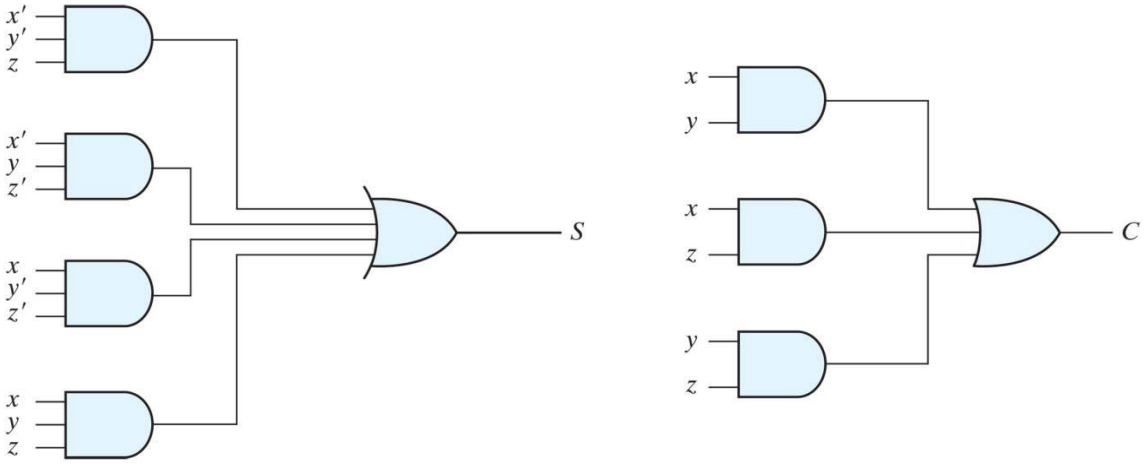
- We will get the following **Boolean functions** for each output after simplification using k-map.



$$S = x'y'z + x'yz' + xy'z' + xyz$$



$$C = xy + xz + yz$$



For further simplification, The AND/OR representations of *S* and *C* can be reduced into XORs as follows:

$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$= z\,(x'\,y' + x\,y) + z'\,(x'\,y + x\,y')$$

$$= z\,(x \odot y) + z'\,(x \oplus y)$$

$$= z\,(x \oplus y)' + z'\,(x \oplus y)$$

$$= z \oplus x \oplus y$$

$$C = xy + x'yz + xy'z$$

$$= xy + z\,(x'y + xy')$$

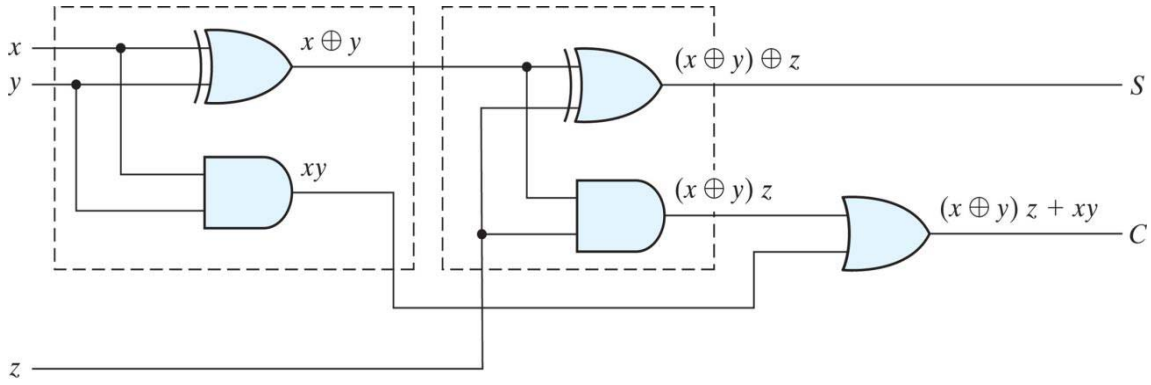
$$= xy + z\,(x \oplus y)$$

Therefore, the final equations are:

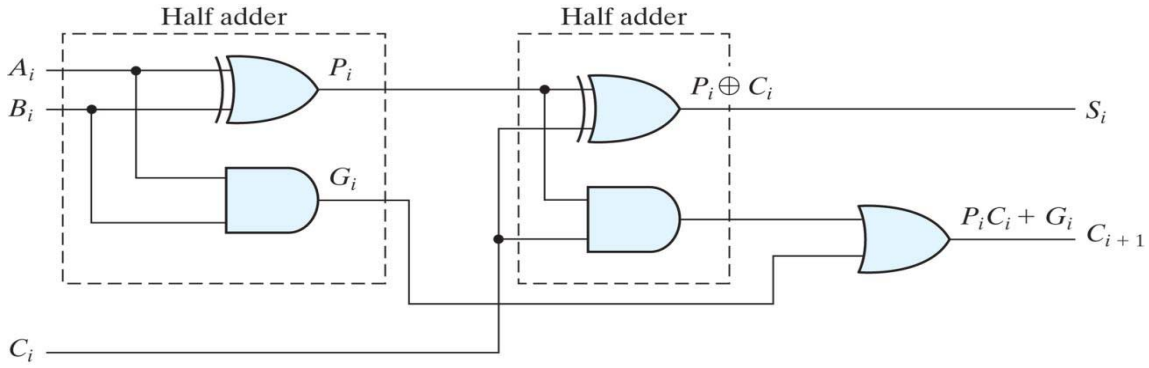
$$S = z \oplus x \oplus y$$

$$C = xy + z\,(x \oplus y)$$

- Then, the **circuit diagram** of Full adder is shown in the following figure.



Therefore, the full adder can be constructed from two half adders and an OR gate.

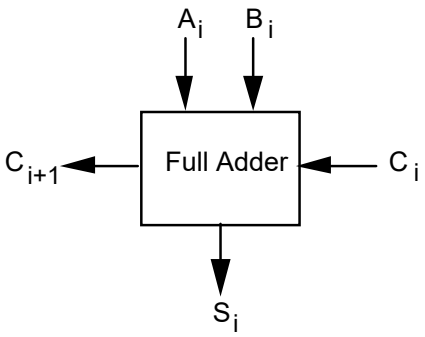


4.5 Four-bit Adder

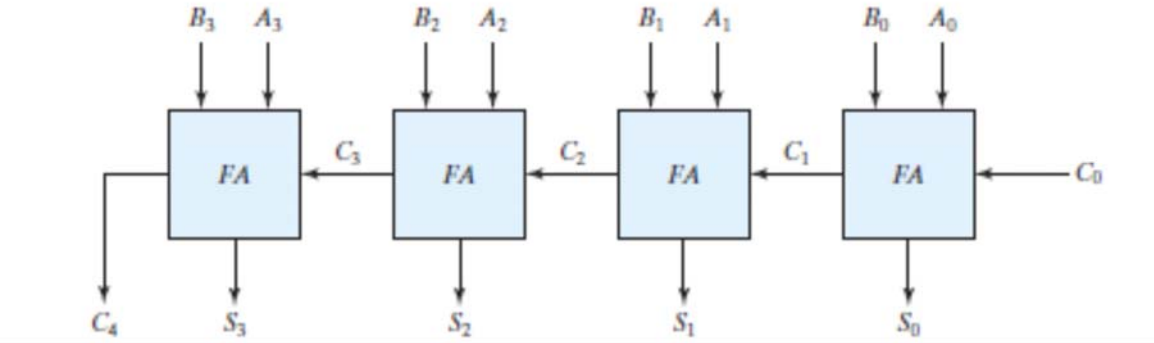
The four-bit adder is a typical example of a standard component. It can be used in many applications involving arithmetic operations. Observe that the design of this circuit

Putting it all together as Block Diagram as shown in the following figure

- Single-bit full adder
- Common piece of computer hardware



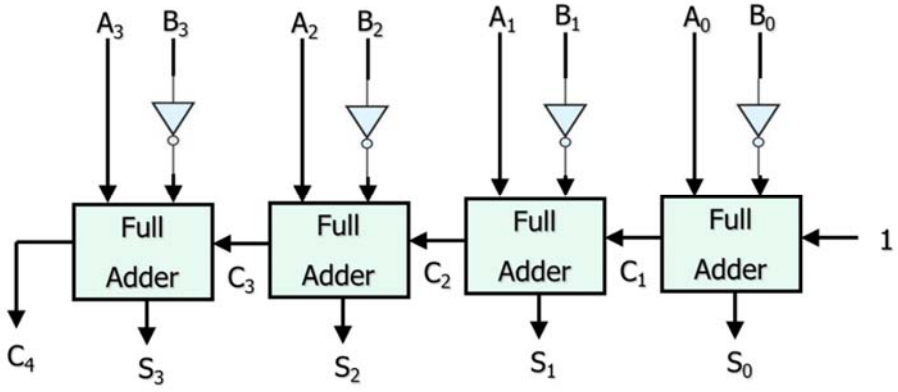
Accordingly, 4-bit Adder can be constructed as the following figure:



4.6 Four-bit Subtractor

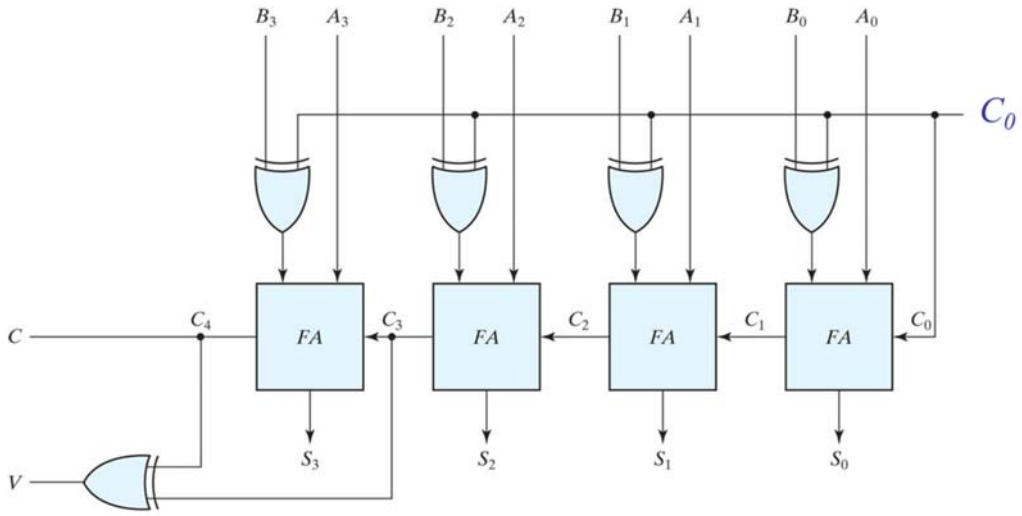
The subtraction of unsigned binary numbers can be done most conveniently by means of complements. Remember that the subtraction *A* - *B* can be done by taking the 2's complement of *B* and adding it to *A*. The 2's complement can be obtained by taking the 1's complement and adding 1 to the least significant pair of bits. The 1's complement can be implemented with inverters, and a 1 can be added to the sum through the input carry.

- Subtracting a number is similar
  - Perform 2's complement
  - Perform addition
- Can we augment the adder with 2's complement hardware?



4.7 Four-bit Adder-Subtractor

The addition and subtraction operations can be combined into one circuit with one common binary adder by including an exclusive-OR gate with each full adder. A four-bit adder–subtractor circuit (with overflow detection) as shown in following figure.



4.8 Overflow in 2's Complement Addition

When two values of the same sign are added:

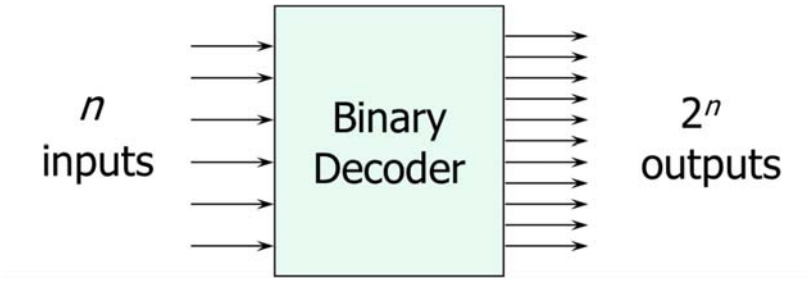
- Result won't fit in the number of bits provided
- Result has the opposite sign

00	01	11	10	00	11
0010	0011	1110	1101	0010	1110
0011	0110	1101	1010	1100	0100
0101	1001	1011	0111	1110	0010
+2	+3	-2	-3	2	-2
+3	+6	-3	-6	-4	+4
5	-7	-5	7	-2	2
	OFL		OFL		

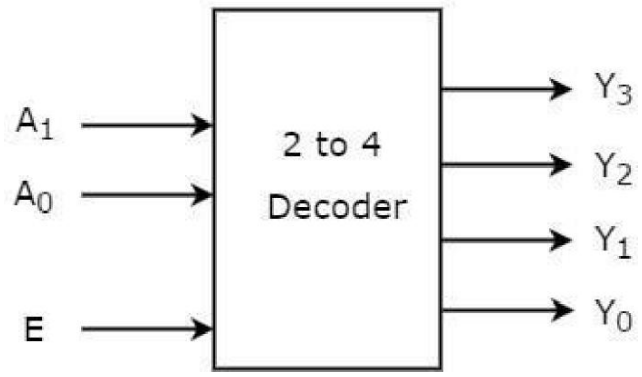
4.9 Binary Decoder

The binary decoder as shown in figure has the following characteristics

- Black box with *n* input lines and 2^n output lines.
- Only one output is a 1 for any given input.



Let 2 to 4 Decoder has two inputs A1 & A0 and four outputs Y3, Y2, Y1 & Y0. The block diagram of 2 to 4 decoder is shown in the following figure.



One of these four outputs will be '1' for each combination of inputs when enable, E is '1'. The Truth table of 2 to 4 decoder is shown below.

Enable	Inputs		Outputs			
E	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

From Truth table, we can write the Boolean functions for each output as

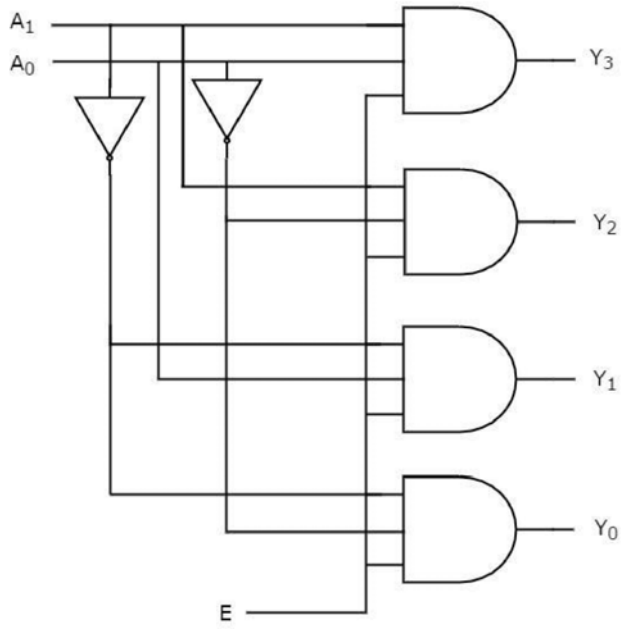
$$Y3=E.A1.A0$$

$$Y2=E.A1.A0'$$

$$Y1=E.A1'.A0$$

$$Y0=E.A1'.A0'$$

Each output is having one product term. So, there are four product terms in total. We can implement these four product terms by using four AND gates having three inputs each & two inverters. The circuit diagram of 2 to 4 decoder is shown in the following figure.

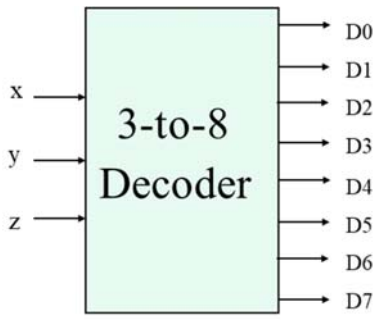


Therefore, the outputs of 2 to 4 decoder are nothing but the min terms of two input variables A1 & A0, when enable, E is equal to one. If enable, E is zero, then all the outputs of decoder will be equal to zero.

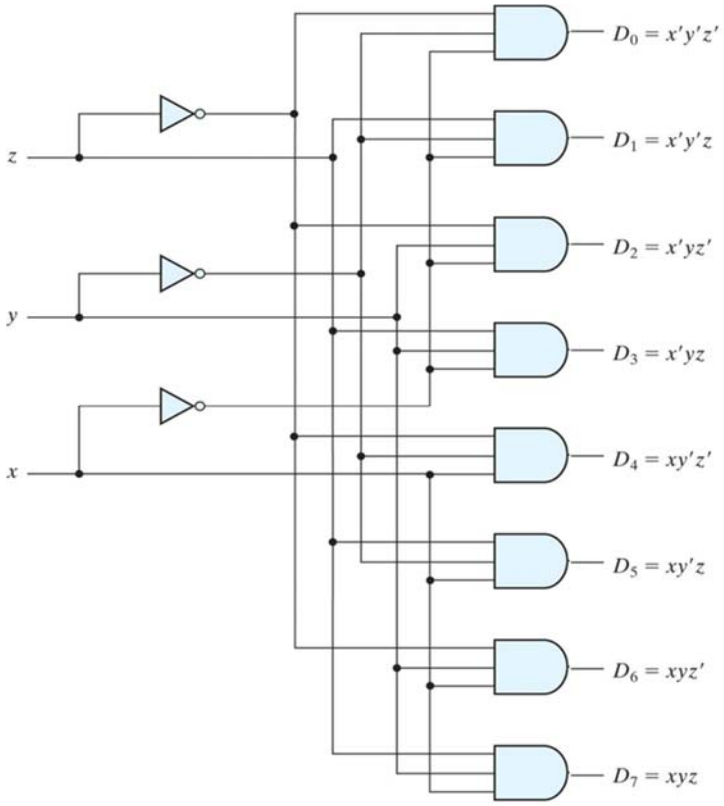
Similarly, 3 to 8 decoder produces eight min terms of three input variables A2, A1 & A0 and 4 to 16 decoder produces sixteen min terms of four input variables A3, A2, A1 & A0.

4.9.1 3-to-8 Binary Decoder

In this section, let us implement **3 to 8 decoder using 2 to 4 decoders.** We know that 2 to 4 Decoder has two inputs, A1 & A0 and four outputs, Y3 to Y0. Whereas, 3 to 8 Decoder has three inputs A2, A1 & A0 and eight outputs, Y7 to Y0. The following figure represent 3 to 8 Decoder has three inputs x, y & z and eight outputs, D7 to D0.



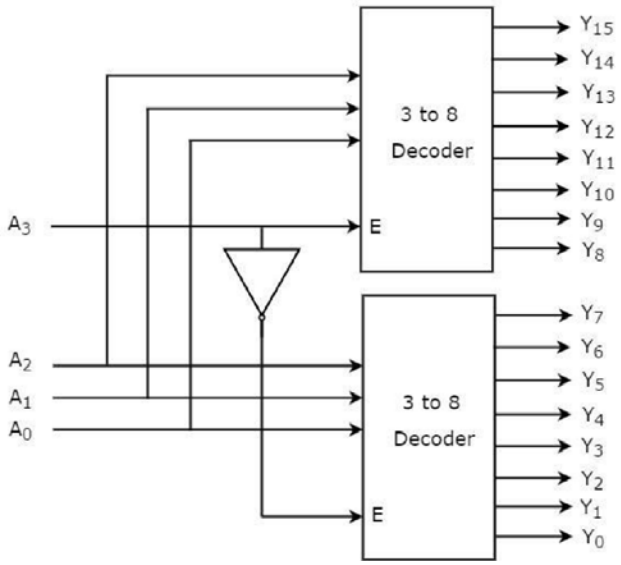
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1



4.9.2 4 to 16 Decoder

In this section, let us implement **4 to 16 decoder using 3 to 8 decoders.** We know that 3 to 8 Decoder has three inputs A2, A1& A0 and eight outputs, Y7 to Y0. Whereas, 4 to 16 Decoder has four inputs A3, A2, A1 & A0 and sixteen outputs, Y15 to Y0

Therefore, we require two 3 to 8 decoders for implementing one 4 to 16 decoder. The **block diagram** of 4 to 16 decoder using 3 to 8 decoders is shown in the following figure.



The parallel inputs A2, A1 & A0 are applied to each 3 to 8 decoder. The complement of input, A3 is connected to Enable, E of lower 3 to 8 decoder in order to get the outputs, Y7 to Y0. These are the lower eight min terms. The input, A3 is directly connected to Enable, E of upper 3 to 8 decoder in order to get the outputs, Y15 to Y8. These are the higher eight min terms.

4.10 Implementing Functions Using Decoders

- Any n-variable logic function can be implemented using a single *n-to-*2^n decoder to generate the minterms.
  - OR gate forms the sum
  - The output lines of the decoder corresponding to the minterms of the function are used as inputs to the OR gate
- Any combinational circuit with n inputs and m outputs can be implemented with an *n-to-*2^n decoder with *m* OR gates.
- Suitable when a circuit has many outputs, and each output function is expressed with few minterms.

Example

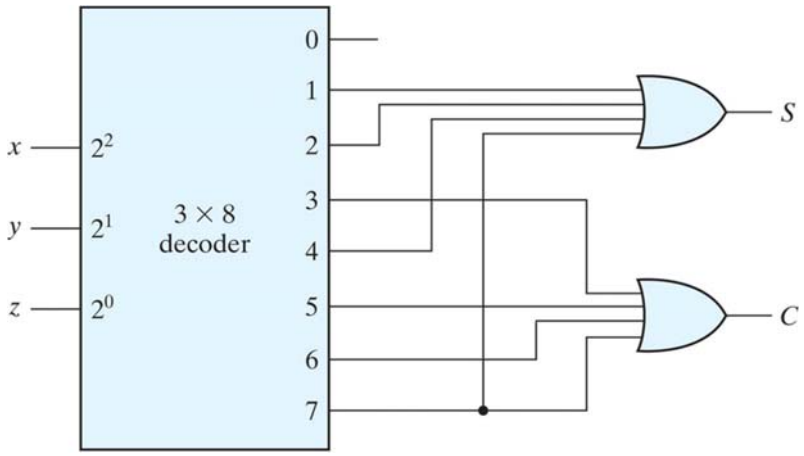
Implement the function of the Full adder using a decoder.

Sum: Σ (x, y, z) = Σ (1,2,4,7)

Carry: Σ (x, y, z) = Σ (3,5,6,7)

Solution

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

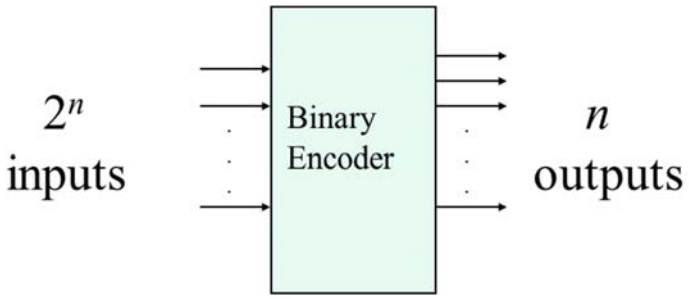


4.11 Binary Encoder

An **Encoder** is a combinational circuit that performs the reverse operation of Decoder. It has maximum of 2^n input lines and 'n' output lines. It will produce a binary code equivalent to the input, which is active High. Therefore, the encoder encodes 2^ninput lines with 'n' bits. It is optional to represent the enable signal in encoders.

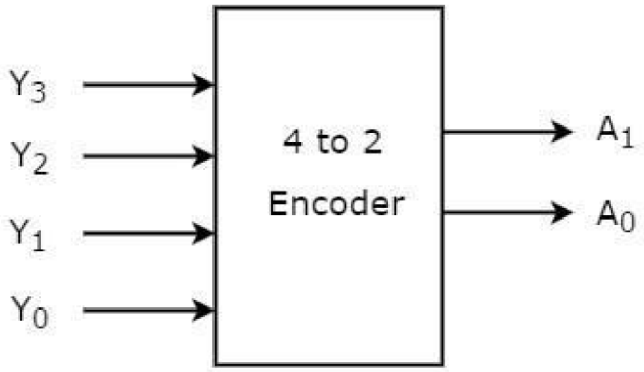
- If a decoder's output code has fewer bits than the input code, the device is usually called an encoder.
  - e.g. 2^n-to-n
- The simplest encoder is a 2^n-to-n binary encoder
  - One of 2^n inputs = 1

- Output is an n-bit binary number



4.11.1 4 to 2 Encoder

Let 4 to 2 Encoder has four inputs Y3, Y2, Y1 & Y0 and two outputs A1 & A0. The **block diagram** of 4 to 2 Encoder is shown in the following figure.



At any time, only one of these 4 inputs can be '1' in order to get the respective binary code at the output. The **Truth table** of 4 to 2 encoder is shown below.

Inputs				Outputs	
Y3	Y2	Y1	Y0	A1	A0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

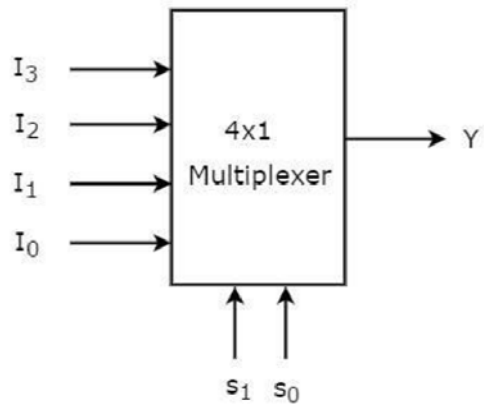
4.12 Binary Multiplexers

Multiplexer is a combinational circuit that has maximum of 2^n data inputs, 'n' selection lines and single output line. One of these data inputs will be connected to the output based on the values of selection lines.

Since there are 'n' selection lines, there will be 2^n possible combinations of zeros and ones. So, each combination will select only one data input. Multiplexer is also called as **Mux**.

4.12.1 4x1 Multiplexer

4x1 Multiplexer has four data inputs I3, I2, I1 & I0, two selection lines s1 & s0 and one output Y. The block diagram of 4x1 Multiplexer is shown in the following figure.



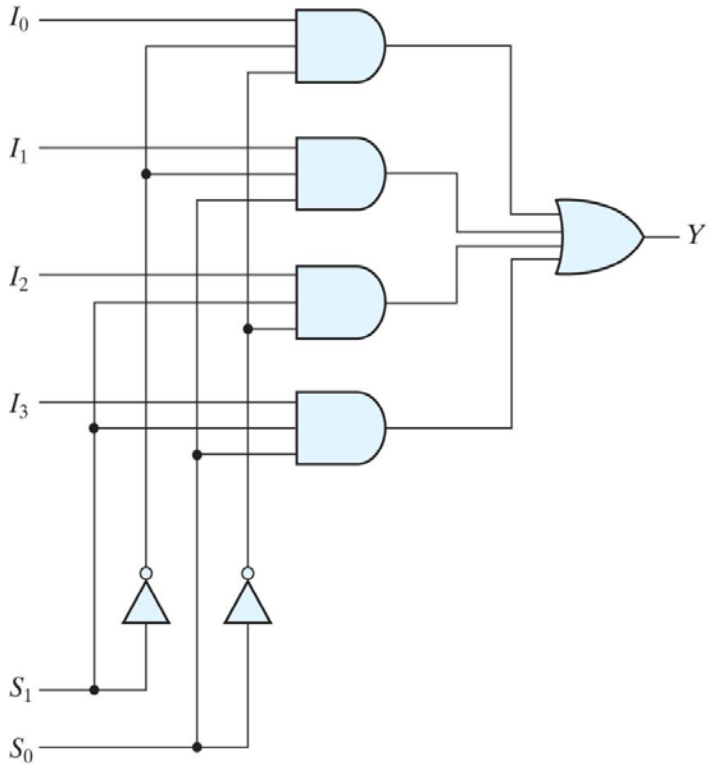
One of these 4 inputs will be connected to the output based on the combination of inputs present at these two selection lines. Truth table of 4x1 Multiplexer is shown below.

Selection Lines		Output
S_1	S_0	V
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

From Truth table, we can directly write the Boolean function for output, Y as

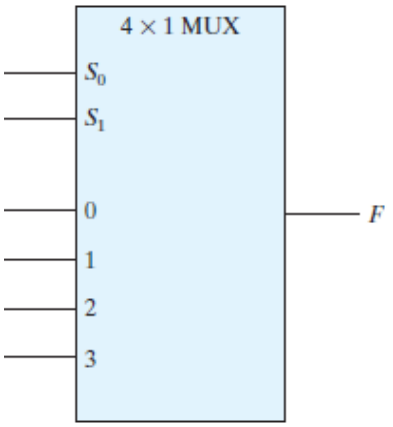
$$Y=S1'S0'I0+S1'S0I1+S1S0'I2+S1S0I3Y=S1'S0'I0+S1'S0I1+S1S0'I2+S1S0I3$$

We can implement this Boolean function using Inverters, AND gates & OR gate. The circuit diagram of 4x1 multiplexer is shown in the following figure.



4.12.1 Implementing Boolean Functions with MUXs

Example: Implement the Boolean Function F (x, y, z) = Σ (1, 2, 6, 7) using 4 x 1 MUX.

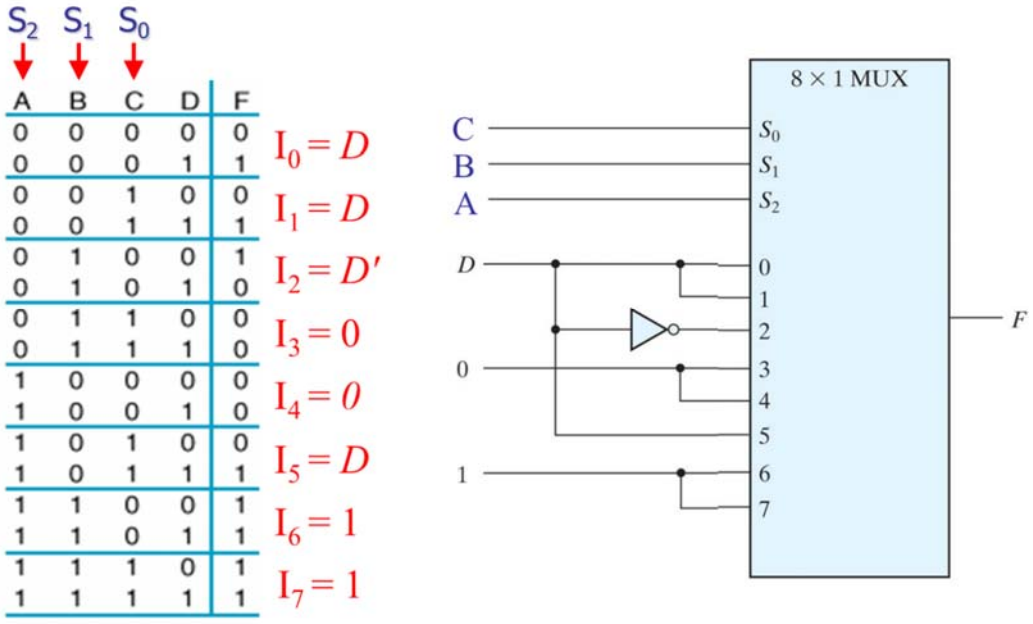


S_1	S_0			
x	y	z	F	
0	0	0	0	$I_0 = z$
0	0	1	1	
0	1	0	1	$I_1 = z'$
0	1	1	0	
1	0	0	0	$I_2 = 0$
1	0	1	0	
1	1	0	1	$I_3 = 1$
1	1	1	1	

- Set the inputs of multiplexer.
- Connect input variable to “select” inputs
- Set “data” input lines of multiplexer to function values

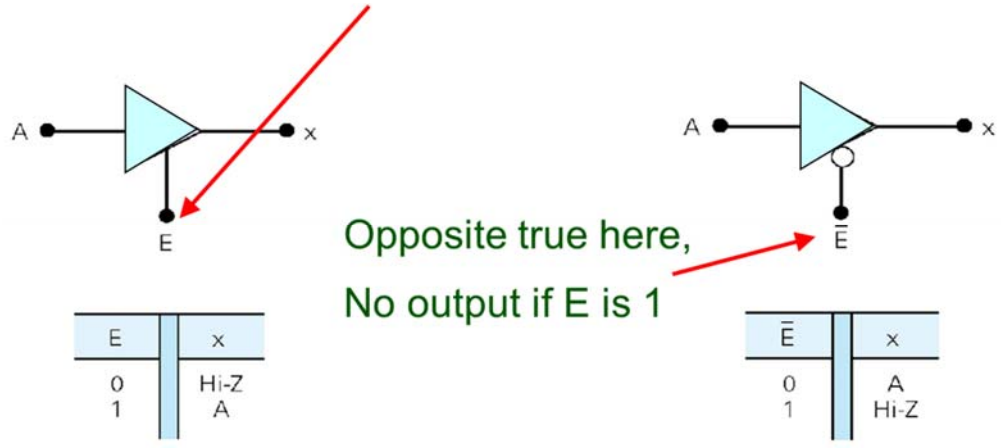
4.12.2 Implementing a 4-Input Function with a MUX

- Example: Implement the Boolean function F(A, B, C, D) = ∑(1, 3, 4, 11, 12, 13, 14, 15) using 8 x 1 MUX.



4.13 Three-state Gate

- Output: 0, 1, and high-impedance (open circuit)
- If the select input (E) is 0, the three-state gate has no output



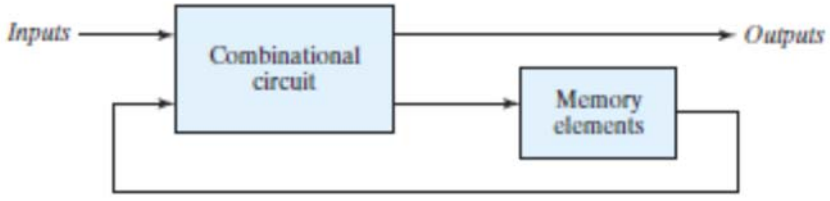
Chapter 5: Synchronous Sequential Logic

5.1 Introduction

This sequential circuit contains a set of inputs and output(s). The output(s) of sequential circuit depends not only on the combination of present inputs but also on the previous output(s). Previous output is nothing but the **present state.** Therefore, sequential circuits contain combinational circuits along with memory (storage) elements. Some sequential circuits may not contain combinational circuits, but only memory elements.

5.2 Sequential Circuits

A block diagram of a sequential circuit is shown in following Figure. It consists of a combinational circuit to which storage elements are connected to form a feedback path. The block diagram demonstrates that the outputs in a sequential circuit are a function not only of the inputs, but also of the present state of the storage elements. The next state of the storage elements is also a function of external inputs and the present state. Thus, a sequential circuit is specified by a time sequence of inputs, outputs, and internal states. In contrast, the outputs of combinational logic depend only on the present values of the inputs.



There are two main types of sequential circuits, and their classification is a function of the timing of their signals. A synchronous sequential circuit is a system whose behavior can be defined from the knowledge of its signals at discrete instants of time. The behavior of an asynchronous sequential circuit depends upon the input signals at any instant of time and the order in which the inputs change.

Following table shows the differences between combinational circuits and sequential circuits.

Combinational Circuits	Sequential Circuits
Outputs depend only on present inputs.	Outputs depend on both present inputs
Feedback path is not present.	Feedback path is present.
Memory elements are not required.	Memory elements are required.
Clock signal is not required.	Clock signal is required.
Easy to design.	Difficult to design.

5.2.1 Types of Sequential Circuits

Following are the two types of sequential circuits –

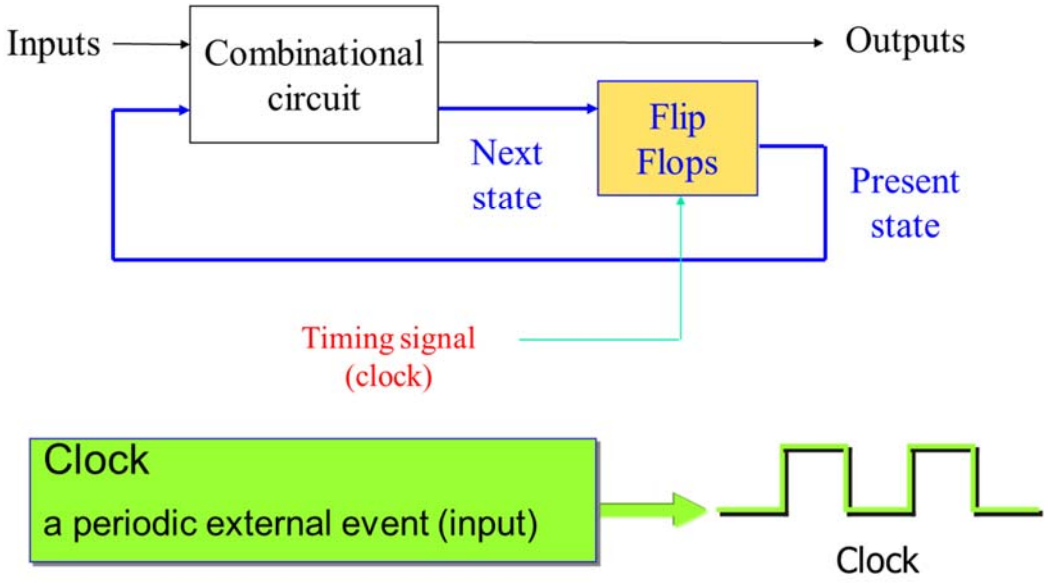
- Asynchronous sequential circuits
- Synchronous sequential circuits

5.2.1.1 Asynchronous sequential circuits

If some or all the outputs of a sequential circuit do not change (affect) with respect to active transition of clock signal, then that sequential circuit is called as Asynchronous sequential circuit. That means, all the outputs of asynchronous sequential circuits do not change (affect) at the same time. Therefore, most of the outputs of asynchronous sequential circuits are not in synchronous with either only positive edges or only negative edges of clock signal.

5.2.1.2Synchronous sequential circuits

If all the outputs of a sequential circuit change (affect) with respect to active transition of clock signal, then that sequential circuit is called as Synchronous sequential circuit. That means, all the outputs of synchronous sequential circuits change (affect) at the same time. Therefore, the outputs of synchronous sequential circuits are in synchronous with either only positive edges or only negative edges of clock signal.



5.3 Storage Elements

There are two types of memory elements based on the type of triggering that is suitable to operate it.

- Latches
- Flip-flops

- Latch = level-sensitive device
  - Control signal: Enable
  - State changes with input when enabled (e.g., when Enable = 1)
  - Holds last input value when disabled (when Enable = 0)

- Flip-flop = edge-triggered device
  - Control signal: periodic Clock
  - State of flip-flop can only change during clock transition
  - Example: Flip-flops change on rising/falling edge of clock

5.4 Level-sensitive vs Edge-triggered

- Latches are level-sensitive



(a) Response to positive level

- Flip-flops are edge-sensitive



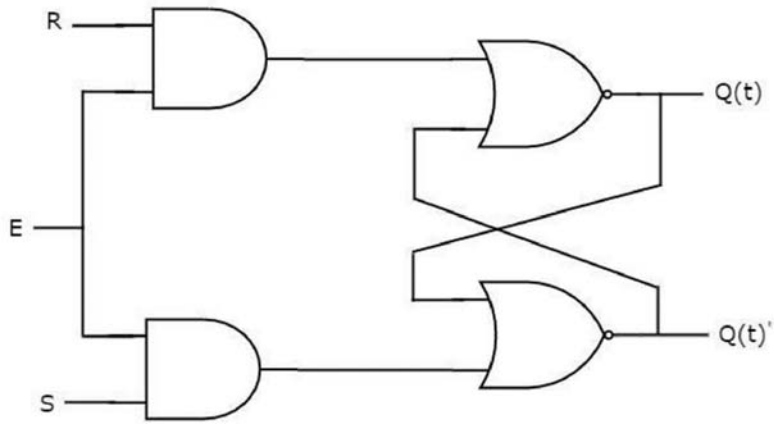
(b) Positive-edge response



(c) Negative-edge response

5.5 SR Latch

SR Latch is also called as Set Reset Latch. This latch affects the outputs as long as the enable, E is maintained at '1'. The circuit diagram of SR Latch is shown in the following figure.



This circuit has two inputs S & R and two outputs Q(t) & Q(t)'. The **upper NOR gate** has two inputs R & complement of present state, Q(t)' and produces next state, Q(t+1) when enable, E is '1'.

Similarly, the **lower NOR gate** has two inputs S & present state, Q(t) and produces complement of next state, Q(t+1)' when enable, E is '1'.

We know that a **2-input NOR gate** produces an output, which is the complement of another input when one of the input is '0'. Similarly, it produces '0' output, when one of the input is '1'.

- If S = 1, then next state Q(t + 1) will be equal to '1' irrespective of present state, Q(t) values.
- If R = 1, then next state Q(t + 1) will be equal to '0' irrespective of present state, Q(t) values.

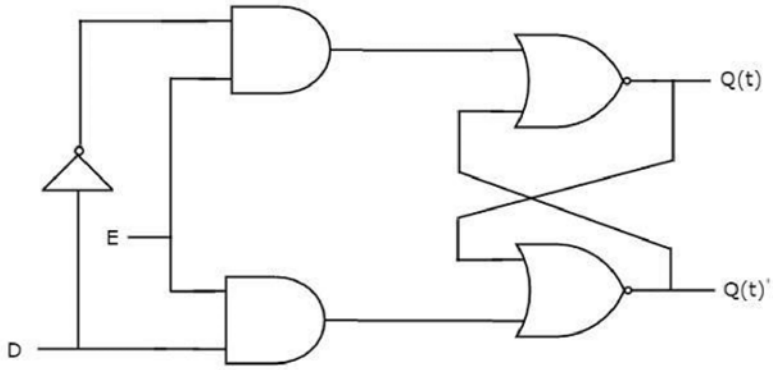
At any time, only of those two inputs should be '1'. If both inputs are '1',
then the next state Q(t + 1) value is undefined.
The following table shows the **state table** of SR latch.

S	R	Q(t + 1)
0	0	Q(t)
0	1	0
1	0	1
1	1	-

Therefore, SR Latch performs three types of functions such as Hold, Set & Reset based on the input conditions.

5.6 D Latch

There is one drawback of SR Latch. That is the next state value can't be predicted when both the inputs S & R are one. So, we can overcome this difficulty by D Latch. It is also called as Data Latch. The circuit diagram of D Latch is shown in the following figure.



This circuit has single input D and two outputs Q(t) & Q(t)'. D Latch is obtained from SR Latch by placing an inverter between S amp;& R inputs and connect D input to S. That means we eliminated the combinations of S & R are of same value.

- If D = 0 → S = 0 & R = 1, then next state Q(t + 1) will be equal to '0' irrespective of present state, Q(t) values. This is corresponding to the second row of SR Latch state table.
- If D = 1 → S = 1 & R = 0, then next state Q(t + 1) will be equal to '1' irrespective of present state, Q(t) values. This is corresponding to the third row of SR Latch state table.

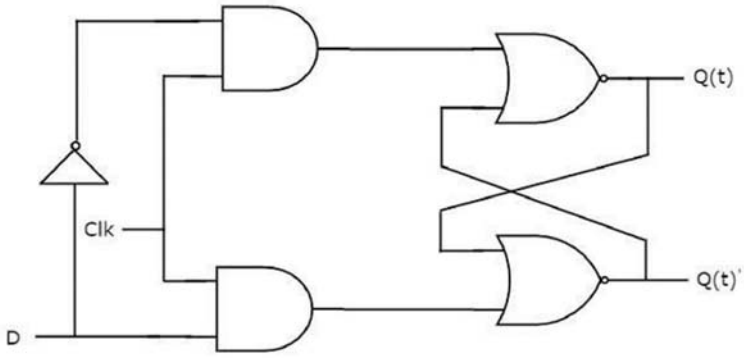
The following table shows the state table of D latch.

D	Q(t + 1)
0	0
1	1

Therefore, D Latch Hold the information that is available on data input, D. That means the output of D Latch is sensitive to the changes in the input, D as long as the enable is High.

5.7 D flip-flop

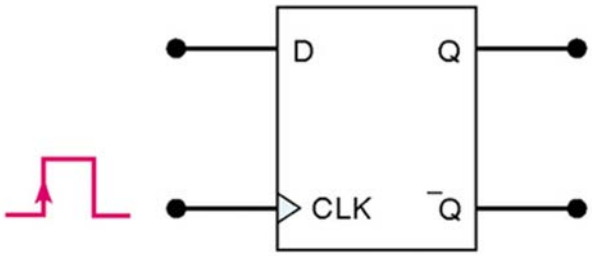
D flip-flop operates with only positive clock transitions or negative clock transitions. Whereas, D latch operates with enable signal. That means, the output of D flip-flop is insensitive to the changes in the input, D except for active transition of the clock signal. The **circuit diagram** of D flip-flop is shown in the following figure.



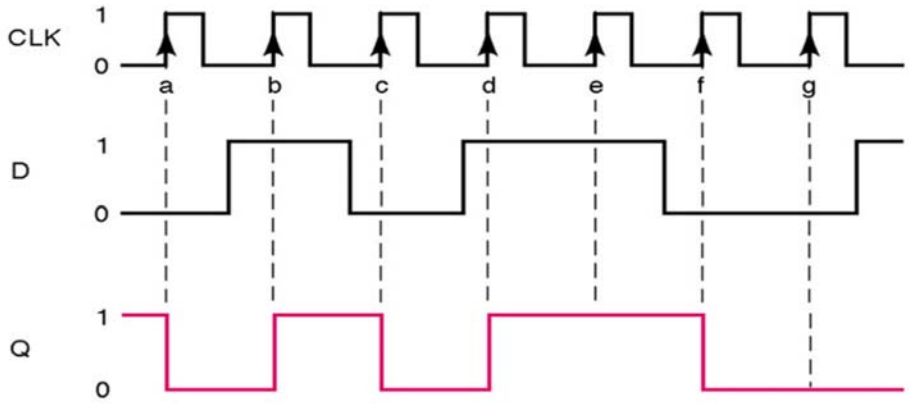
This circuit has single

input D and two outputs Q(t) & Q(t)'. The operation of D flip-flop is similar to D Latch. But, this flip-flop affects the outputs only when positive transition of the clock signal is applied instead of active enable.

The following table shows the **state table** of D flip-flop.



D	CLK	Q
0	↑	0
1	↑	1



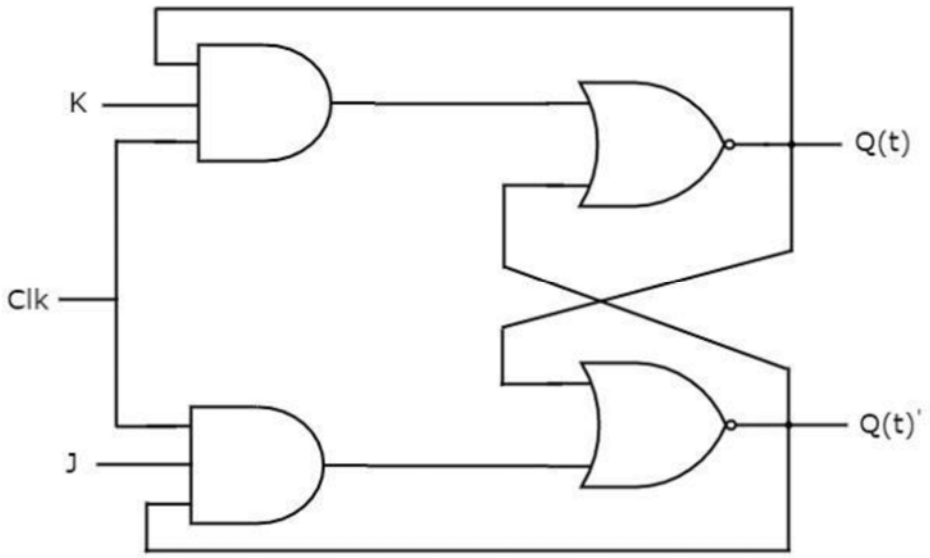
Therefore, D flip-flop always Hold the information, which is available on data input, D of earlier positive transition of clock signal. From the above state table, we can directly write the next state equation as

$$Q(t + 1) = D$$

Next state of D flip-flop is always equal to data input, D for every positive transition of the clock signal. Hence, D flip-flops can be used in registers, **shift registers** and some of the counters.

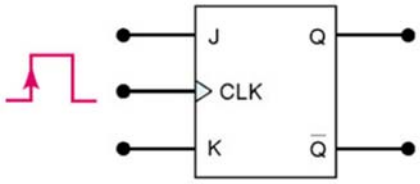
5.8 JK flip-flop

JK flip-flop is the modified version of SR flip-flop. It operates with only positive clock transitions or negative clock transitions. The **circuit diagram** of JK flip-flop is shown in the following figure.

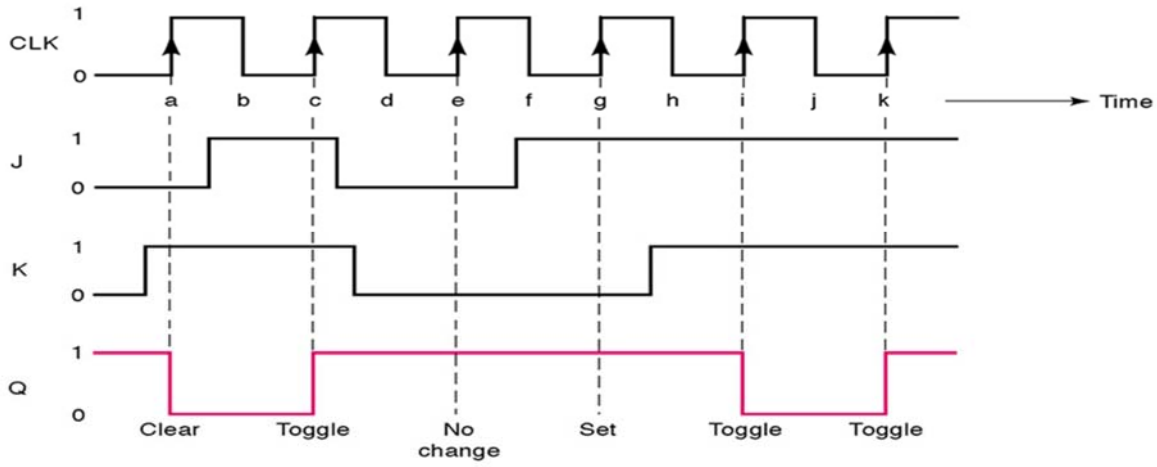


This circuit has two inputs J & K and two outputs Q(t) & Q(t)'. The operation of JK flip-flop is similar to SR flip-flop. Here, we considered the inputs of SR flip-flop as S = **J** Q(t)' and R = KQ(t) in order to utilize the modified SR flip-flop for 4 combinations of inputs.

J: set, K: reset, if J=K=1 then toggle output

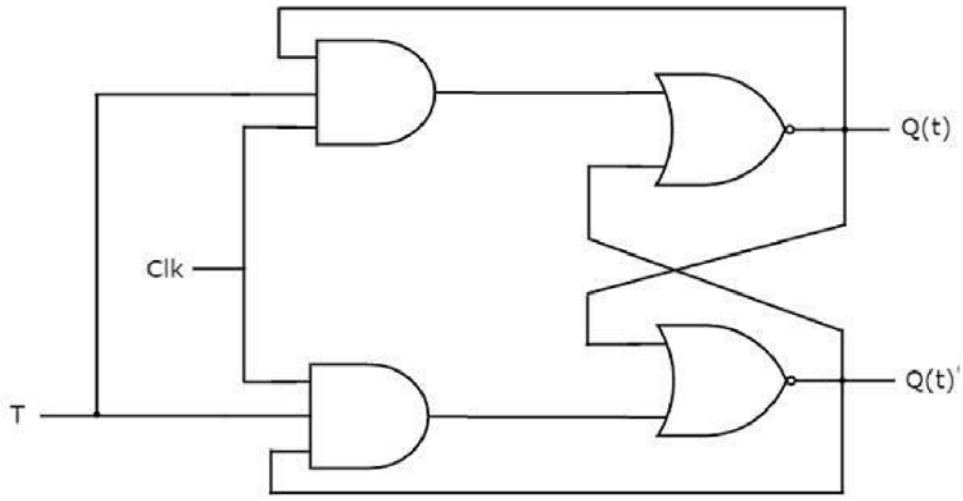


J	K	CLK	Q
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	$\overline{Q_0}$ (toggles)



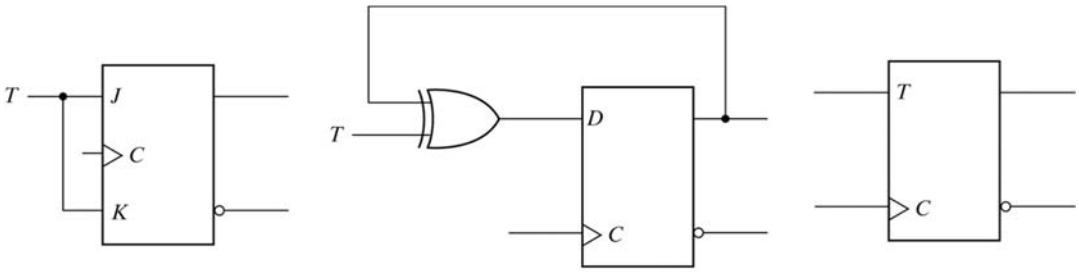
5.9 T Flip-Flop

T flip-flop is the simplified version of JK flip-flop. It is obtained by connecting the same input 'T' to both inputs of JK flip-flop. It operates with only positive clock transitions or negative clock transitions. The circuit diagram of T flip-flop is shown in the following figure.



This circuit has single input T and two outputs Q(t) & Q(t)'. The operation of T flip-flop is same as that of JK flip-flop. Here, we considered the inputs of JK flip-flop as J = T and K = T in order to utilize the modified JK flip-flop for 2 combinations of inputs. So, we eliminated the other two combinations of J & K, for which those two values are complement to each other in T flip-flop.

- Created from JK or D F.F.
- T=0 → No change
- T=1 → invert Q



(a) From *JK* flip-flop (b) From *D* flip-flop (c) Graphic symbol

The following table shows the state table of T flip-flop.

Flip-Flop Characteristic Tables

JK Flip-Flop

J	K	Q(t + 1)	
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(t)$	Complement

D Flip-Flop

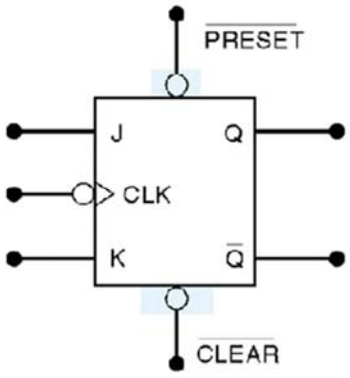
D	Q(t + 1)	
0	0	Reset
1	1	Set

T Flip-Flop

T	Q(t + 1)	
0	$Q(t)$	No change
1	$Q'(t)$	Complement

5.10 Asynchronous Inputs

- J, K are *synchronous* inputs
  - Effects on the output are synchronized with the *CLK* input
- *Asynchronous* inputs operate independently of the clock and synchronous inputs
  - Set/reset the FF to 1/0 states *at any time*



PRESET	CLEAR	FF response
1	1	Clocked operation*
0	1	Q = 1 (regardless of CLK)
1	0	Q = 0 (regardless of CLK)
0	0	Not used

*Q will respond to J, K, and CLK

Chapter 6: Registers and Counters

6.1 Introduction

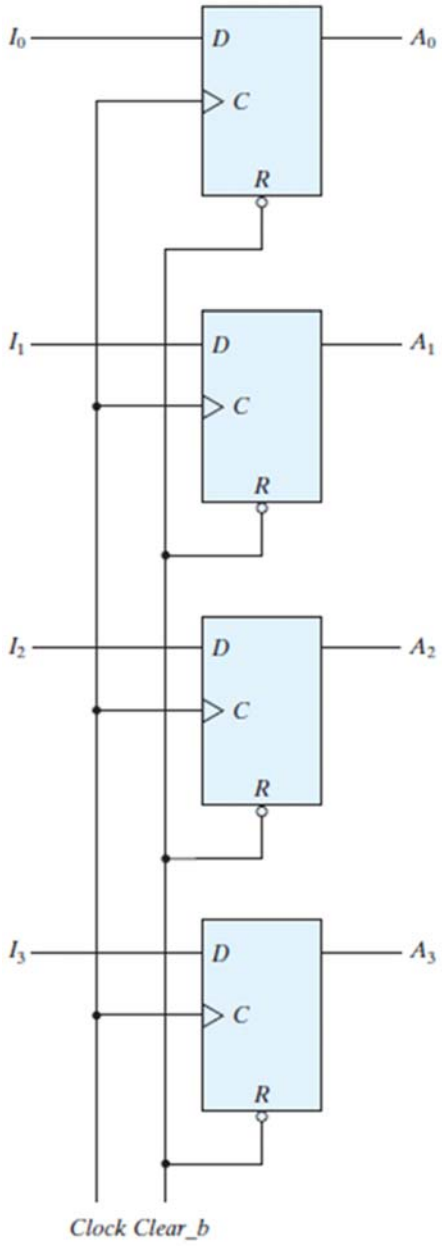
A *register* is a group of flip-flops, each one of which shares a common clock and is capable of storing one bit of information. An n -bit register consists of a group of n flip-flops capable of storing n bits of binary information. In addition to the flip-flops, a register may have combinational gates that perform certain data-processing tasks. In its broadest definition, a register consists of a group of flip-flops together with gates that affect their operation. The flip-flops hold the binary information, and the gates determine how the information is transferred into the register.

A *counter* is essentially a register that goes through a predetermined sequence of binary states. The gates in the counter are connected in such a way as to produce the prescribed sequence of states. Although counters are a special type of register, it is common to differentiate them by giving them a different name

6.2 Simple Parallel Load Register

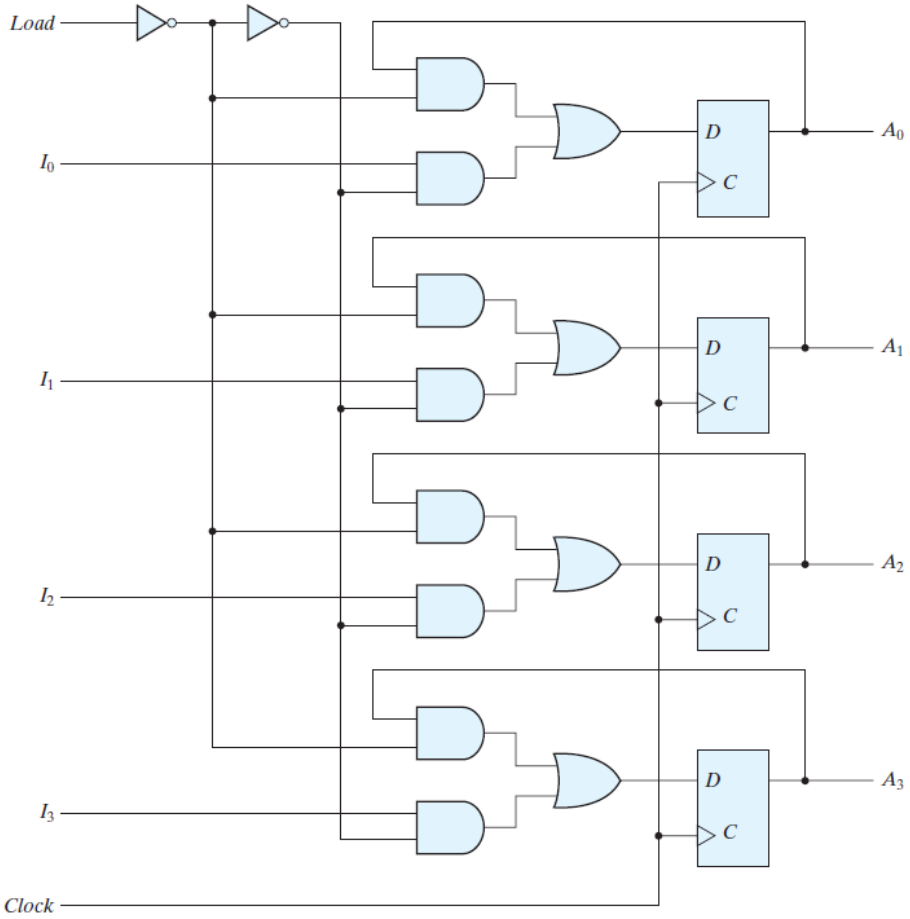
Various types of registers are available commercially. The simplest register is one that consists of only flip-flops, without any gates. The following Figure shows such a register constructed with four D -type flip-flops to form a four-bit data storage register. The common clock input triggers all flip-flops on the positive edge of each pulse, and the binary data available at the four inputs are transferred into the register. The value of ($I3$, $I2$, $I1$, $I0$) immediately before the clock edge determines the value of ($A3$, $A2$, $A1$, $A0$) after the clock edge. The four outputs can be sampled at any time to obtain the binary information stored in the register. The input *Clear_b* goes to the active-low R (reset) input of all four flip-flops. When this input goes to 0, all flip-flops are reset

asynchronously. The *Clear_b* input is useful for clearing the register to all 0's prior to its clocked operation. The *R* inputs must be maintained at logic 1 (i.e., de-asserted) during normal clocked operation. Note that, depending on the flip-flop, either *Clear*, *Clear_b*, *reset*, or *reset_b* can be used to indicate the transfer of the register to an all 0's state.



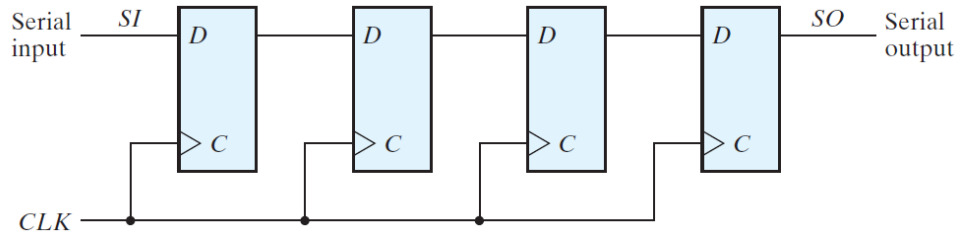
6.2.1 Parallel Load Register

Registers with parallel load are a fundamental building block in digital systems. A four-bit data-storage register with a load control input that is directed through gates and into the *D* inputs of the flip-flops is shown in the following Figure.



6.3 Shift Registers

- A register capable of shifting the binary information held in each cell to its neighboring cell, in a selected direction, is called a *shift register.*
- The shown shift register is unidirectional (left-to-right).
- Each clock pulse shifts the contents of the register one bit position to the right.



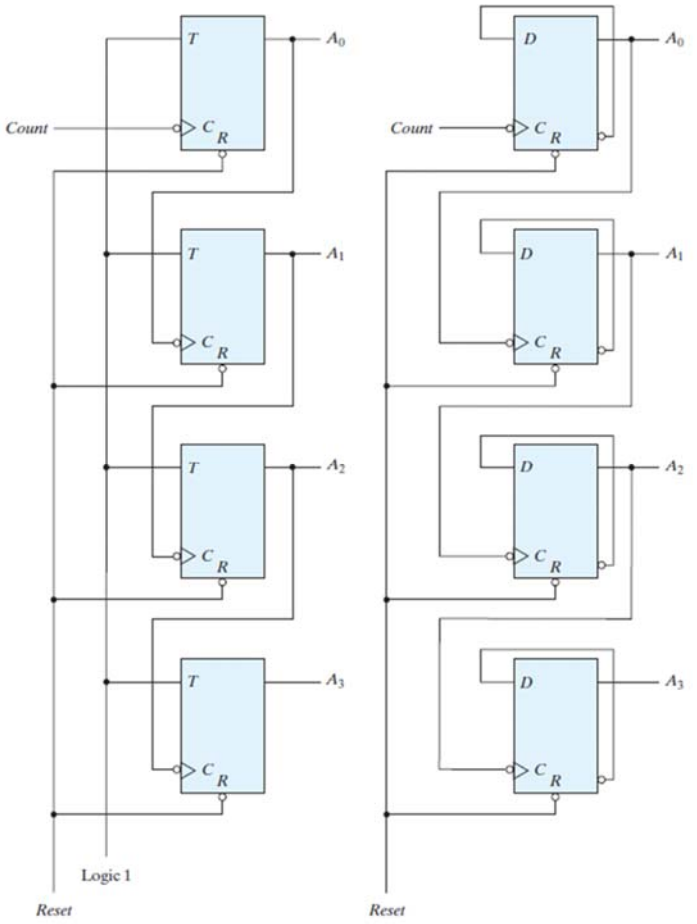
6.4 Counters

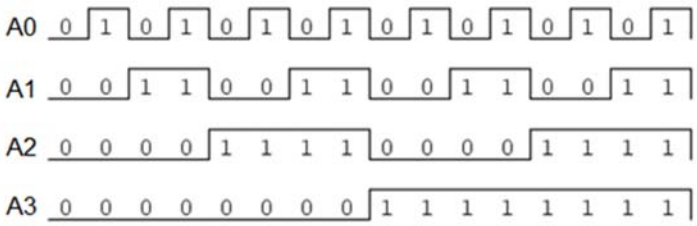
A register that goes through a prescribed sequence of states upon the application of input pulses is called a *counter*

- A counter is a register that goes through a predetermined sequence of states upon application of input pulses (clock)
- Example: n-bit binary counter
  - Counts from 0 through 2^n -1
  - Ripple counters (asynchronous)
  - Synchronous counters
- Counters can count up or down.

6.4.1 Binary Counter

Four-bit binary ripple counter as shown in the following figure. The count starts with binary 0 and increments by 1 with each count pulse input. After the count of 15, the counter goes back to 0.



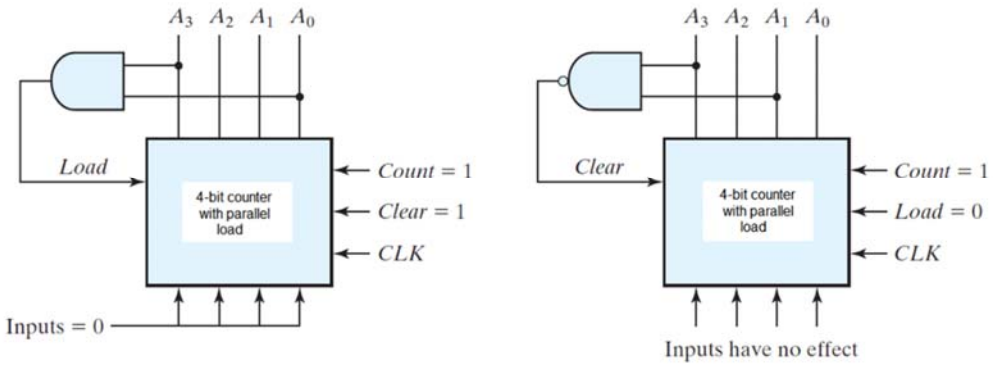


6.4.2 BCD counter using a counter with parallel load

Two ways to achieve a BCD counter using a counter with parallel load as shown in the following figure and has the following characteristic

- There are two ways in which a counter with a parallel load is used to generate the BCD count.

- The first way uses an AND gate to detect the occurrence of state 1001, which activates the *Load* input; therefore, on the next clock edge the register does not count, but is loaded from its four inputs with zeros.
- The second way uses a NAND gate to detect the count of 1010, but as soon as this count occurs, the register is cleared. The count 1010 has no chance of staying on for any appreciable time, because the register goes immediately to 0.



Chapter 7: Software Lab Manual

DELTA TECHNOLOGICAL UNVERISITY

Faculty of Technological Industry and Energy

Digital Engineering

Student Lab Manual

Student Name: .....

Student ID: .....

Section: ..... **Group:** .....

Session (Fall / Spring / Summer): .....

TABLE OF CONTENTS

Lab 1: Basic Digital Simulation.....
Lab 2: Logic Gates.....
Lab 3: Gate-Level Minimization .....
Lab 4a: Functions of Combinational Logic .....
Lab 4b: Functions of Combinational Logic .....
Lab 6a: Registers.....
Lab 6b: Counters.....
Lab 6c: Counters and Decoders .....
LAB 7: Memory and Programmable Logic .....

Lab 1: Basic Digital Simulation

Objectives:

- To learn the basic digital simulation and verify selected circuits using digital circuit simulator called "Deeds" (Digital Electronics Education and Design Suite) simulator.
- In this introductory exercise you will test the simple logic network represented in the figure below and, at the same time, you will gain confidence with the Digital Circuit Simulator (d-DcS) of the Deeds.
- Note: you can try the different inputs and output devices in the simulator.

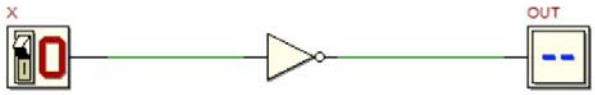
Requirements:

- A desktop or laptop computer.
- Download the Deeds (Digital Electronics Education and Design Suite) software.

Background:

Deeds: Digital Electronics Education and Design Suite is a set of educational tools for Digital Electronics. It covers various areas of digital electronics including combinational logic networks, sequential logic networks, custom circuit blocks design, micro-computer programming. The program helps students learn to design and test electronic systems and digital systems on FPGA boards.

1: Use the simulator to draw the following circuit.



2: check the behavior of the network

Start the functional simulation (Interactive Animation) of the network by clicking, on the DcS toolbar, the command 

Now the input switch can be toggled and the gate's output OUT will change accordingly.

3: Fill the output (OUT) column with the data resulting from the simulation.

X	OUT
0	
1	

4: timing simulation of the same network

Start the timing simulation of the network by clicking, on the d-DcS toolbar, the command 

The input values must be drawn directly on the timing diagram window. You should define the values versus time of the input as shown in the figure above. Then draw the resulted output on the following figure.



Lab 2: Logic Gates

Objectives

- To learn the basic logic gates and verify selected circuits using digital circuit simulator called "**Deeds**" (Digital Electronics Education and Design Suite) simulator.
- You are required to work individually and answer the following questions.

Requirements:

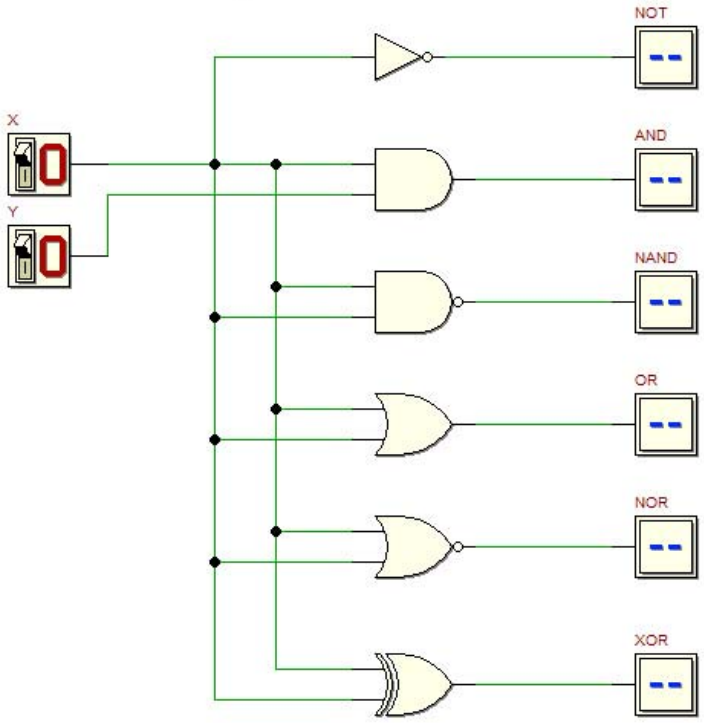
- A desktop or laptop computer.
- Download the Deeds (Digital Electronics Education and Design Suite) software.

Background :

Try out the following exercises (M. Mano, 5th Ed) before your lab:
2-2 (a, f), 2-4 (a, e), 2-5, 2-8, 2-10 , 2-12, 2-13, 2-14, 2-15, 2-18, 2-19, 2-24, 2-27

Simulation:
Simulate the circuits of logic gates (NOT, AND, NAND, OR, NOR, and XOR) to verify their truth tables.

1- Use the simulator to draw the following circuit.

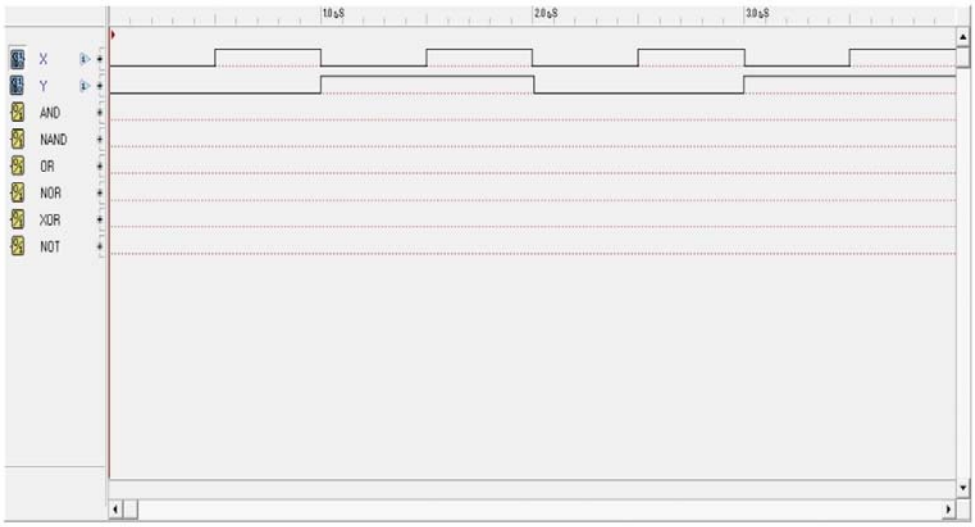


2- Start the **functional simulation (Interactive Animation)** of the network by clicking, on the **d-DcS toolbar**, the command  . Now the two input switches **X** and **Y** can be toggled and the gates' outputs NOT, AND, NAND, OR, NOR, and XOR will be changed accordingly.

3- According to the simulation results, fulfil the following truth table.

X	Y	NOT	AND	NAND	OR	NOR	XOR
0	0						
0	1						
1	0						
1	1						

4- Check the **timing simulation** of the circuit by clicking, on the **d-DcS toolbar**, the command  . The input values must be drawn directly on the timing diagram window. You should define the values versus time of the two inputs, such as all the possible combinations of **X** and **Y** are tested as shown in the following figure.



Reflection Questions

Student is required to submit the following:

- Answers to the exercises **2-14, 2-18, and 2-27** above.
- Deadline: at the beginning of the next Lab session.

Student answer to the reflection question:

Lab 3: Gate-Level Minimization

Objectives

- To learn the basics of Boolean Algebra and simplification using Boolean Algebra and Karnaugh map

Requirements:

- A desktop or laptop computer.
- Download the Deeds (Digital Electronics Education and Design Suite) software.

Background :

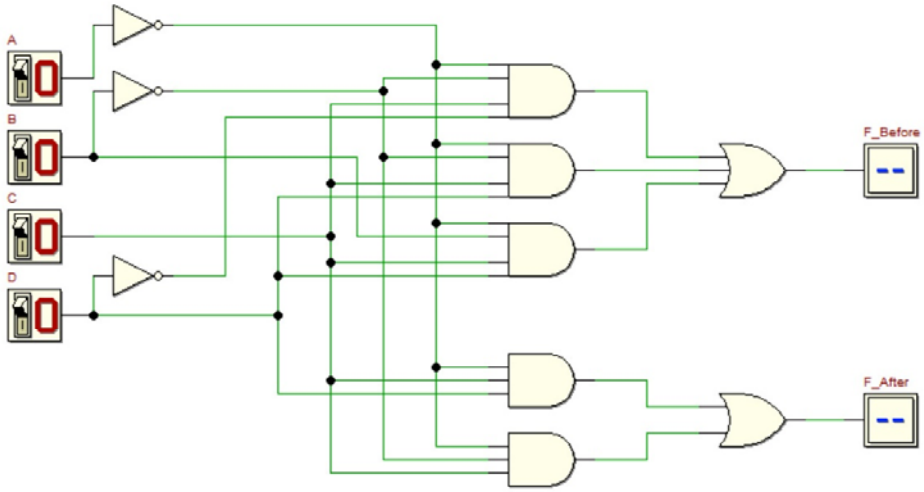
Try out the following exercises (M. Mano 5Ed) before your lab.
3-2 (c, d), 3-4 (a, b), 3-6 (a), 3-7, 3-9, 3-15 (a, b), 3-16 (a, d), 3-17, 3-18, 3-21.

Simulation

Simulate the logic diagram that implements the following function before and after the k-map reduction and fill-up the truth table for both circuits:

$$F(A, B, C, D) = \sum (2, 3, 7)$$

1- Use the simulator to draw the following circuit.

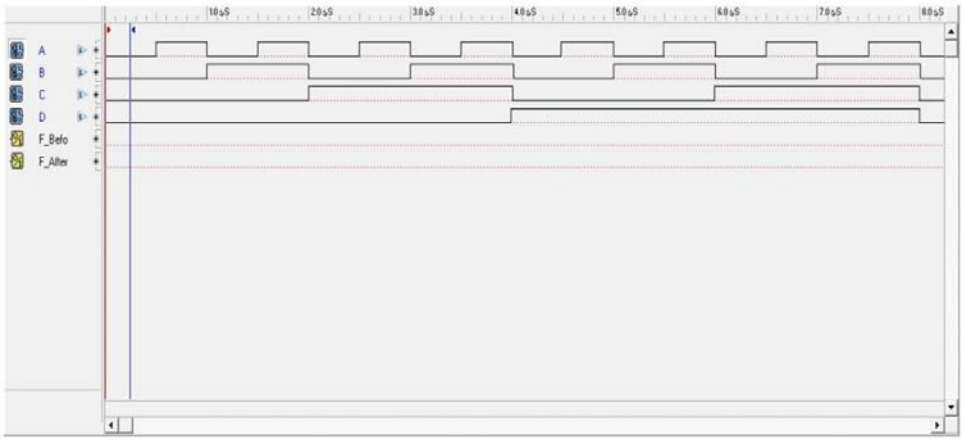


2- Start the **functional simulation (Interactive Animation)** of the circuit by clicking, on the **d-DcS toolbar**, the command  . Now the four input switches **A, B, C,** and **D** can be toggled and the outputs F_Before and F_After will be changed accordingly.

3- According to the simulation results, fulfil the following truth table. Compare the results before and after simulation.

X	Y	C	D	F_Before	F_After
0	0	0	0		
0	0	0	1		
0	0	1	0		
0	0	1	1		
0	1	0	0		
0	1	0	1		
0	1	1	0		
0	1	1	1		
1	0	0	0		
1	0	0	1		
1	0	1	0		
1	0	1	1		
1	1	0	0		
1	1	0	1		
1	1	1	0		
1	1	1	1		

4- Check the **timing simulation** of the circuit by clicking, on the **d-DcS toolbar**, the command  . The input values must be drawn directly on the timing diagram window. You should define the values versus time of the two inputs, such as all the possible combinations of **A, B, C,** and **D** are tested as shown in the following figure. Also draw the corresponding outputs on the following figure.



Reflection Questions

Student is required to submit the following:

- Answers to the exercises **3-4 (a, b) and 3-17** above.
- Deadline: at the beginning of the next Lab session.

Student answer to the reflection question:

Lab 4a: Functions of Combinational Logic

Objectives:

- To learn basic adders, parallel binary adders, ripple carry and look-ahead carry adders, and comparators.
- You are required to work individually and answer the following questions.

Requirements:

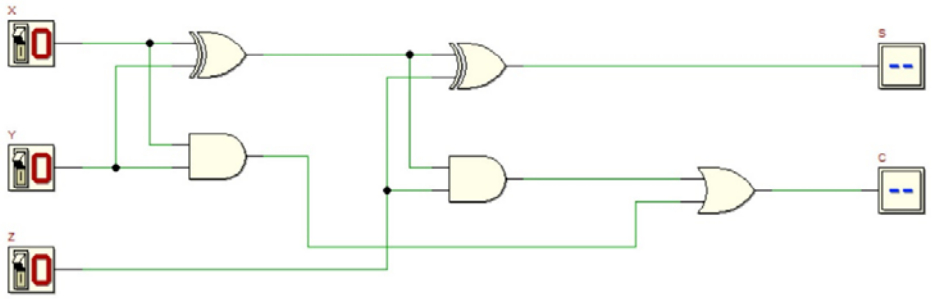
- A desktop or laptop computer.
- Download the Deeds (Digital Electronics Education and Design Suite) software.

Background:

Try out the following exercises before your lab: 4.1(a,b), 4.2, 4.4(a), 4.5, 4.6(a), and 4.7

Simulation:

1- Use the simulator to draw the logic circuit of the full-adder shown in the following figure.



2- Start the **functional simulation (Interactive Animation)** of the circuit by clicking, on the **d-DcS toolbar**, the command  . Now the three input switches **X, Y,** and **Z** can be toggled and the outputs S (sum) and C (carry) will be changed accordingly.

3- According to the simulation results, fulfil the following truth table. Compare the results before and after simulation.

X	Y	Z	S	C
0	0	0		
0	0	0		
0	1	1		
0	1	1		
1	0	0		
1	0	0		
1	1	1		
1	1	1		

4- Check the **timing simulation** of the circuit by clicking, on the **d-DcS toolbar**, the command  . The input values must be drawn directly on the timing diagram window. You should define the values versus time of the three inputs, such as all the possible combinations of **X, Y,** and **Z** are tested. Also draw the output resulted from the **timing simulation** on the on the following figure.



Reflection Question:

Student is required to submit the following:

- Answers to the exercises **4.7** above.
- Deadline: at the beginning of the next Lab session.

Student answer to the reflection question:

C

Lab 4b: Functions of Combinational Logic

Objectives

- To learn the different combinational logic circuits such as decoders/encoders, code converters, multiplexers/demultiplexers and parity generators/checkers.
- You are required to work individually and answer the following questions.

Requirements:

- A desktop or laptop computer.
- Download the Deeds (Digital Electronics Education and Design Suite) software.

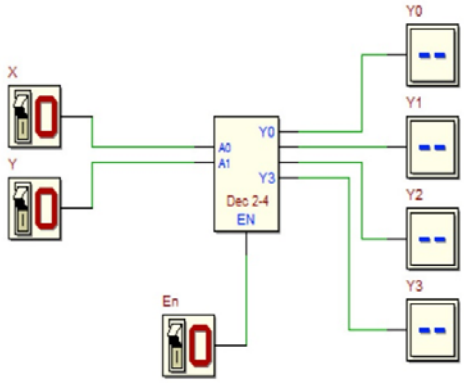
Background:

Try out the following exercises before your lab:
4.22, 4.23, 4.24, 4.27, 4.28(a), 4.31, 4.33, and 4.35.

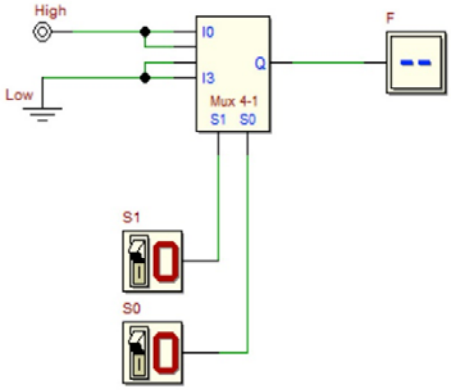
Simulation:

1- Use the simulator to draw each of the following logic circuits in a separate file:

a. A 2-to-4-line decoder with enable input.
b. A 4-to-1-line multiplexer.



a) 2-to-4 Decoder



b) 4-to-1 Multiplexer

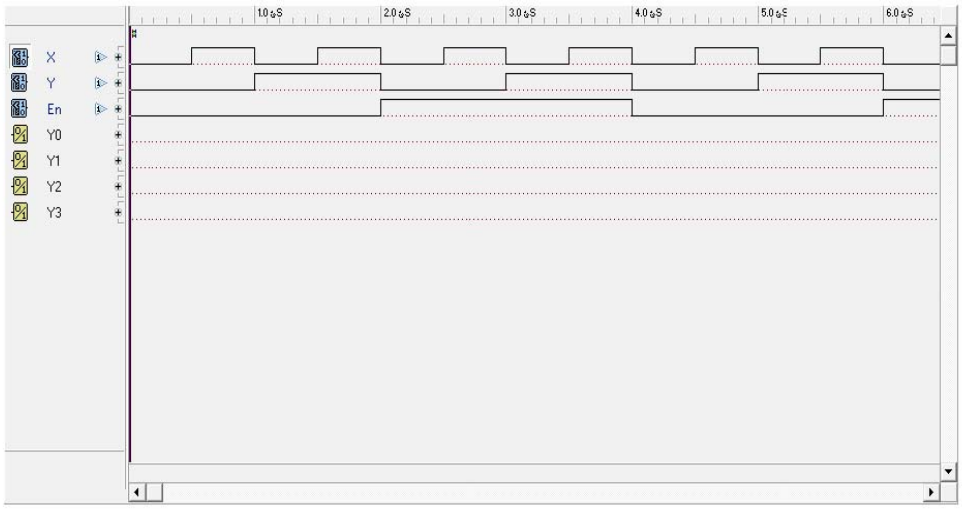
2- Start the **functional simulation (Interactive Animation)** of each circuit by clicking, on the **d-DcS toolbar**, the command  . Now the input switches of each circuit can be toggled and the corresponding outputs will be changed accordingly.

3- According to the simulation results of each circuit, fulfil each of the truth tables corresponding to each logic circuit.

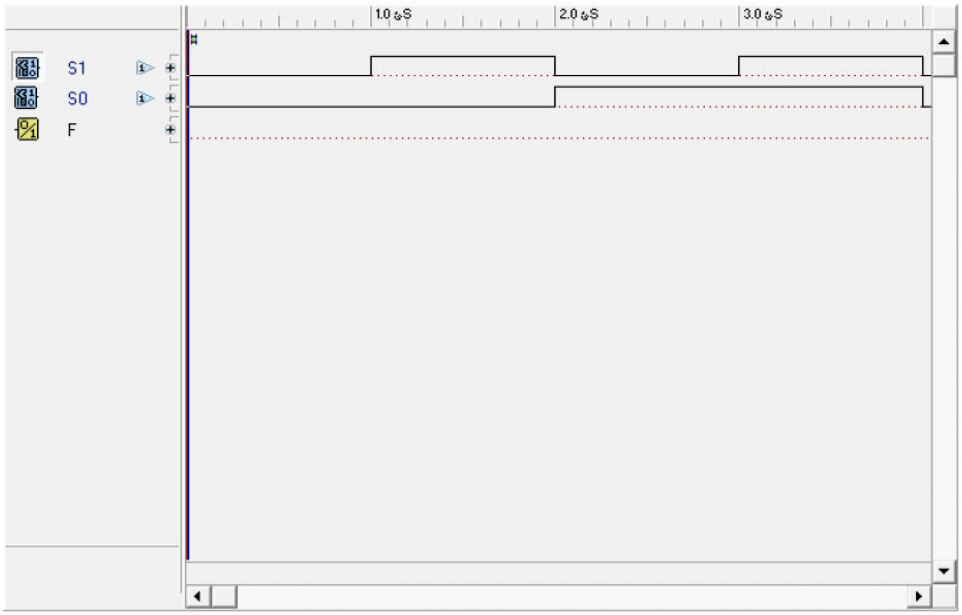
a) Decoder Truth Table						
En	X	Y	Y_0	Y_1	Y_2	Y_3
0	0	0				
0	0	1				
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				

b) Multiplexer Truth Table		
S_0	S_1	F
0	0	
0	1	
1	0	
1	1	

4- Check the **timing simulation** of each logic circuit by clicking, on the **d-DcS toolbar**, the command  . The input values must be drawn directly on the timing diagram window. You should define the values versus time of the inputs such as all the possible input combinations are tested. Also draw the output resulted from each **timing simulation** on the corresponding figure of the following figures.



a) Decoder timing diagram



b) Multiplexer timing diagram

Reflection Questions: Student is required to submit the following:

- Answers to the exercises 4.33 and 4.35 above.
- Deadline: The beginning of the next Lab session.

Student answer to the reflection question(if any)

LAB 5a: Synchronous Sequential Logic

Objectives

- To learn the basic principles of latches and flip-flops which make up the basic building blocks of sequential logic circuits.

Requirements:

- A desktop or laptop computer.
- Download the Deeds (Digital Electronics Education and Design Suite) software.

Background:

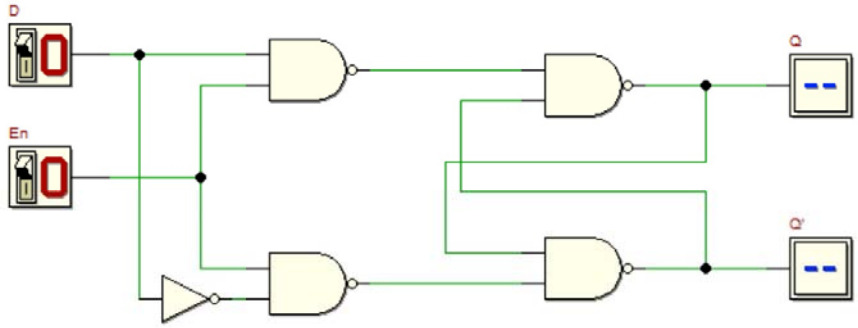
Try out the following exercises before your lab:
5.1, 5.4(a, b), 5.6, 5.7, 5.9 (a, b), and 5.10 (a, b, c)

Simulation:

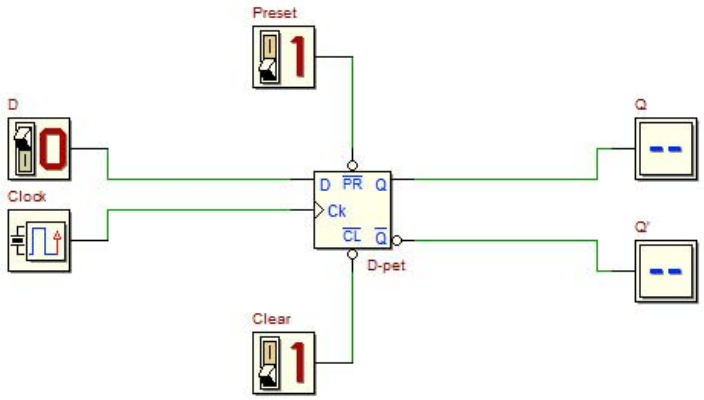
Simulate the following circuits. Write the State table for each flip-flop and Show the timing diagram for each.

a. D-Latch flip-flop
b. Positive edge-triggered D flip-flop
c. Positive edge-triggered J-K flip-flop.

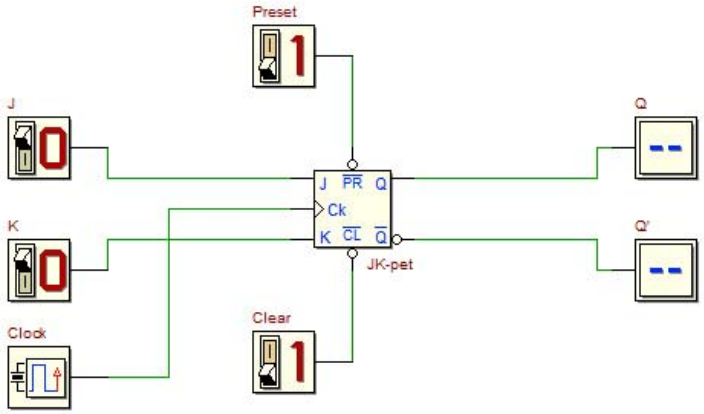
1- Use the simulator to draw each of the following sequential logic circuits in a separate file:



a) D-Latch flip-flop



b) Positive edg-triggered D flip-flop



c) Positive edge-triggered JK flip-flop

2- Start the **functional simulation (Interactive Animation)** of each flip-flop by clicking, on the **d-DcS toolbar**, the command  . Now the input switches of each circuit can be toggled and the corresponding outputs will be changed accordingly.

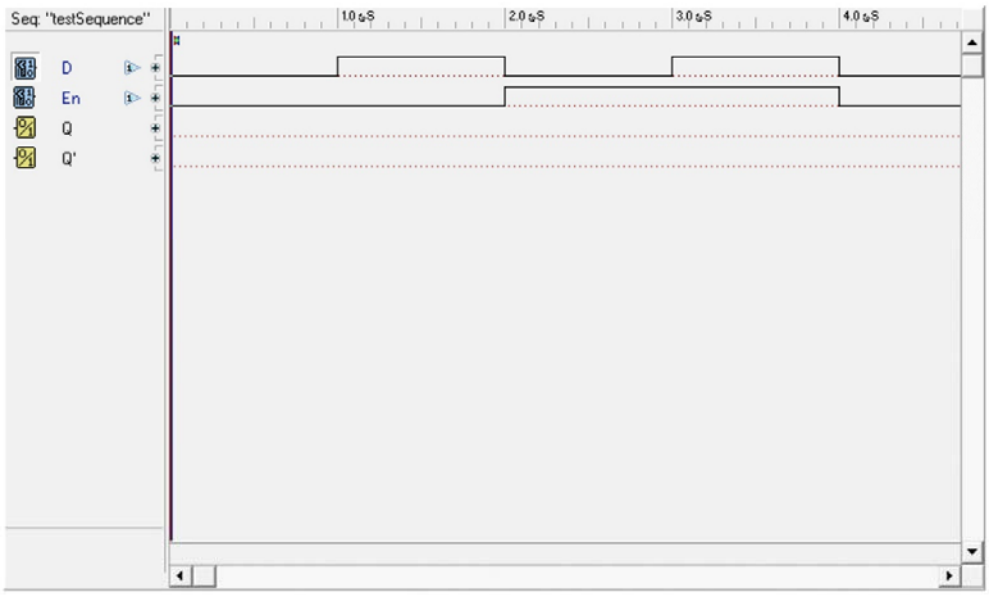
3- According to the simulation results of each flip-flop, fulfil the corresponding table of the following tables.

D-Latch flip-flop			
En	D	Q	Q'
0	0		
0	1		
1	0		
1	1		

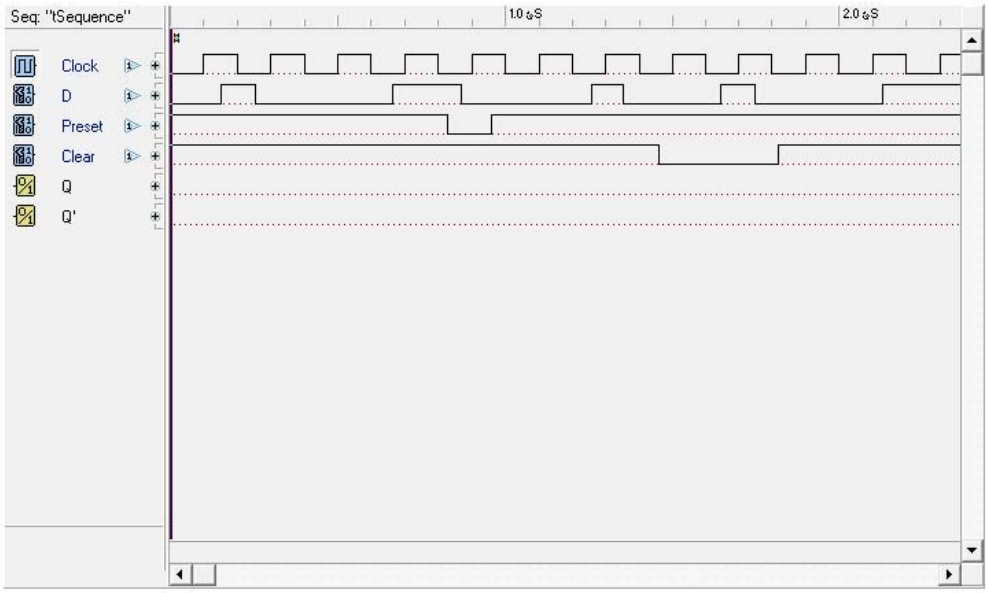
Positive edge-triggered D flip-flop					
D	Clock	Preset	Clear	Q	Q'
0	↑	1	1		
1	↑	1	1		
0	↑	0	0		
1	↑	0	0		

Positive edge-triggered JK flip-flop						
J	K	Clock	Preset	Clear	Q	Q'
0	0	↑	1	1		
0	1	↑	1	1		
1	0	↑	1	1		
1	1	↑	1	1		
0	0	↑	0	0		
0	1	↑	0	0		
1	0	↑	0	0		
1	1	↑	0	0		

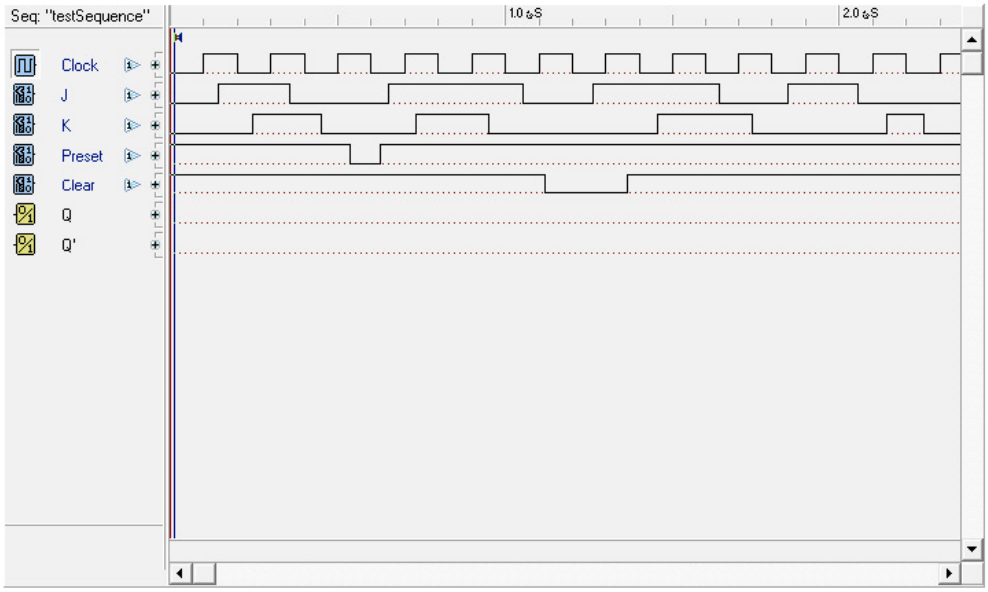
4- Check the **timing simulation** of each flip-flop by clicking, on the **d-DcS toolbar**, the command  . The input values must be drawn directly on the timing diagram window. You should define the values versus time of the inputs such as all the possible input combinations are tested. Also draw the output resulted from each **timing simulation** on the corresponding figure of the following figures.



a) D-Latch flip-flop timing diagram.



c) Positive edge triggered D flip-flop timing diagram.



c) Positive edge triggered JK flip-flop timing diagram.

Reflection Questions: Student is required to submit the following:

- Answers to the exercises **5.10 (a, b, c)** above.
- Deadline: at the beginning of the next Lab session.

Student answer to the reflection question(if any)

LAB 5b: Sequential Circuit Analysis

Objectives

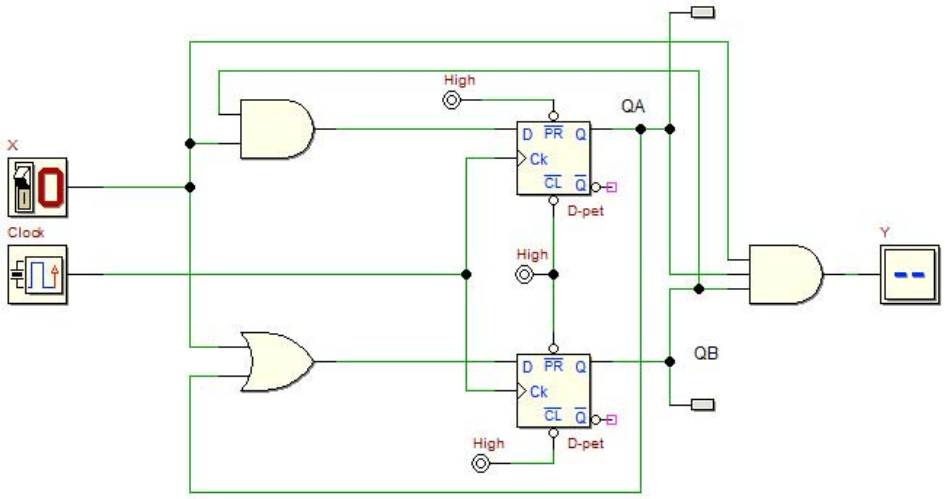
- To learn the basic principles of sequential circuit analysis.

Requirements:

- A desktop or laptop computer.
- Download the Deeds (Digital Electronics Education and Design Suite) software.

Simulation:

1- Use the simulator to draw the following sequential logic circuits.

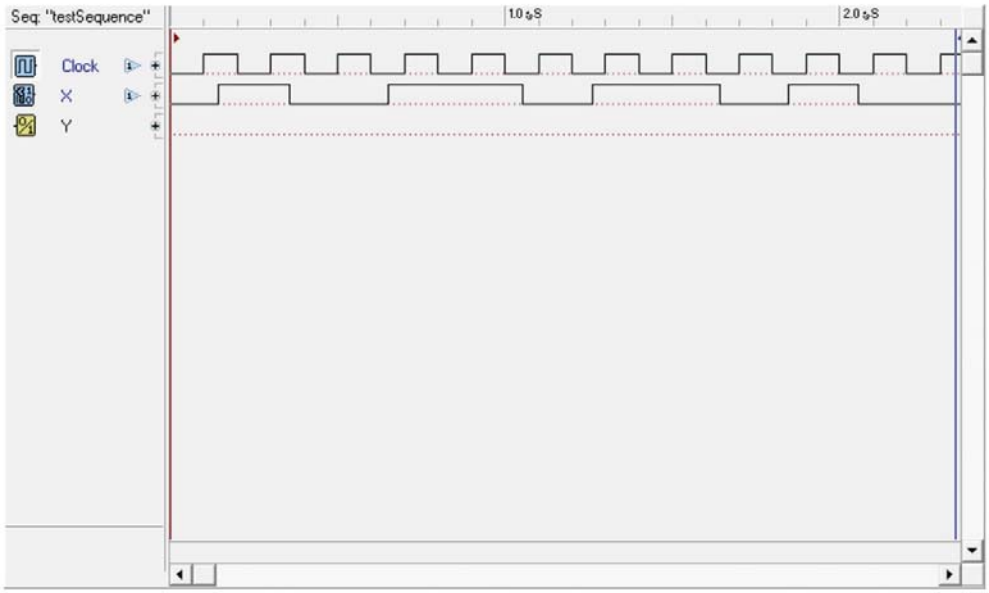


2- Start the **functional simulation (Interactive Animation)** of the sequential circuit by clicking, on the **d-DcS toolbar**, the command  . Now the input switch of the circuit can be toggled and the corresponding outputs will be changed accordingly.

3- According to the simulation results, fulfil the following state table.

Present-state		Input	Next-state		Output
QA	QB	X	QA	QB	Y
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

4- Check the **timing simulation** of the given sequential circuit by clicking, on the **d-DcS toolbar**, the command  . The input values must be drawn directly on the timing diagram window as shown in the following figure. Each student is required to draw the output resulted from the **timing simulation** on the figure below.



C

Lab 6a: Registers

Objectives

- To learn the different types of registers such as parallel load registers and shift registers.

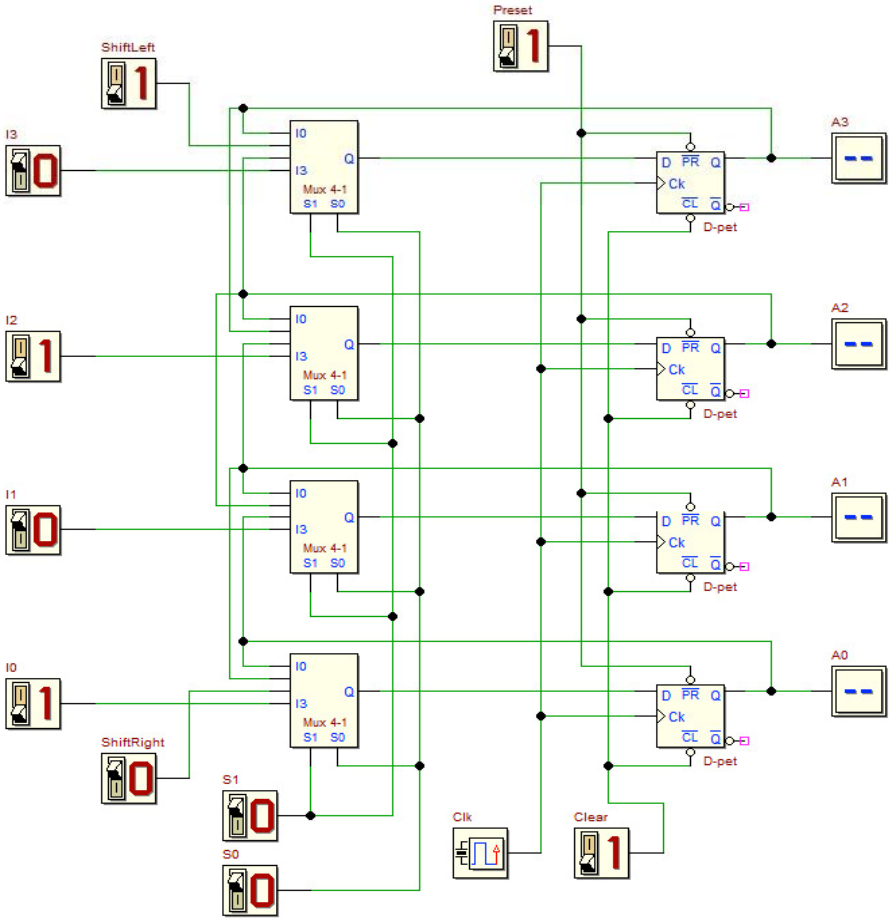
Requirements:

- A desktop or laptop computer.
- Download the Deeds (**Digital Electronics Education and Design Suite**) software.

Background: Try out the following exercises before your lab.
6.3, 6.4, 6.6, and 6.7

Simulation:
Simulate the Four-bit universal shift register shown in the figure below. Show the **timing diagram** and Write its function table.

1- Use the simulator to draw the following sequential logic circuits with the specified initial input values.



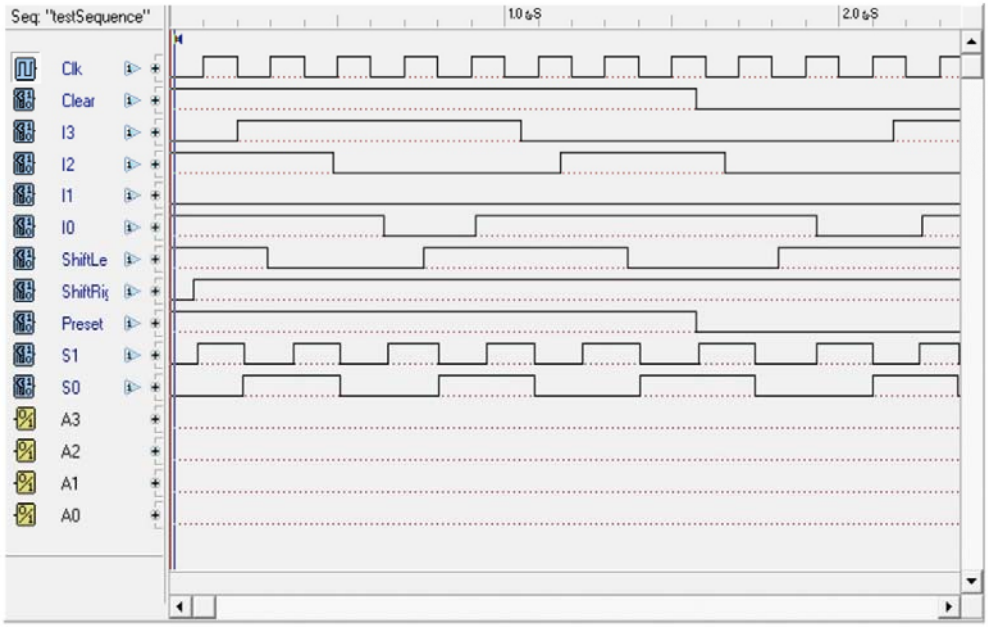
C

2- Start the **functional simulation (Interactive Animation)** of the sequential circuit by clicking, on the **d-DcS toolbar**, the command  . Now switch the inputs to their initial values and change only the selection switches.

3- After four clock cycles for each operation (i.e., selection mode), complete the following table .

Positive edge-triggered JK flip-flop									
S1	So	Clock	Preset	Clear	A3	A2	A1	Ao	Register Operation
0	0	↑	1	1					
0	1	↑	1	1					
1	0	↑	1	1					
1	1	↑	1	1					
0	0	↑	0	0					
0	1	↑	0	0					
1	0	↑	0	0					
1	1	↑	0	0					

4- Check the **timing simulation** of the universal shift register by clicking, on the **d-DcS toolbar**, the command  . The input values must be drawn directly on the timing diagram window as shown in the following figure. Each student is required to draw the output resulted from the **timing simulation** on the figure below.



Reflection Question:

Student is required to submit the following:

- Answers to the exercises 6.6 and 6.7 above.
- Deadline: The beginning of the next Lab session.

Student answer to the reflection questions:

Lab 6b: Counters

Objectives

- To learn the different types of counters such as asynchronous/synchronous and up/down counter, how to design synchronous counters, and counter decoding.
- You are required to work individually and answer the following questions.

Requirements:

- A desktop or laptop computer.
- Download the Deeds (Digital Electronics Education and Design Suite) software.

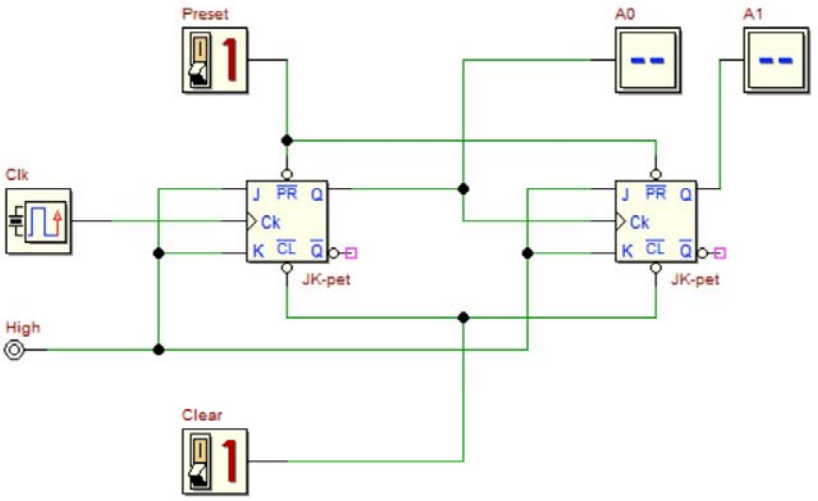
Background:

Try out the following exercises before your lab.
6.11, 6.13, 6.17, 6.24, 6.27

Simulation:

Simulate the following **Two-bit binary ripple counter** circuits. Show the **timing diagram**.

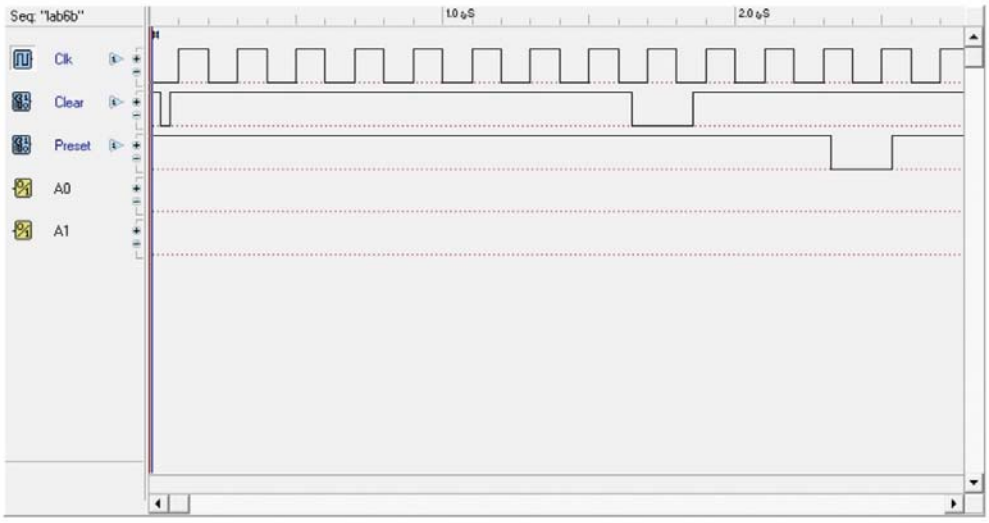
1- Use the simulator to draw the following sequential logic circuits with setting the values of *Preset* and *Clear* switches to 1's as shown in the figure below.



2- Start the **functional simulation (Interactive Animation)** of the sequential circuit by clicking, on the **d-DcS toolbar**, the command  . Then switch the *Clear* switch from 1 to 0 and return it back to 1. Observer the outputs and discuss the results.

.....
.....
.....
.....
.....
.....
.....
.....
.....

4- Check the **timing simulation** of the given sequential circuit by clicking, on the **d-DcS toolbar**, the command  . The input values must be drawn directly on the timing diagram window as shown in the following figure. Each student is required to draw the outputs **A0** and **A1** resulted from the **timing simulation** on the figure below.



Reflection Question: Student is required to submit the following:

- Answers to the exercises 6.11 and 6.13 above.
- Deadline: The beginning of the next Lab session.

Student answer to the reflection question (if any)

Lab 6c: Counters and Decoders

Objectives

- To learn how to use counters with decoders

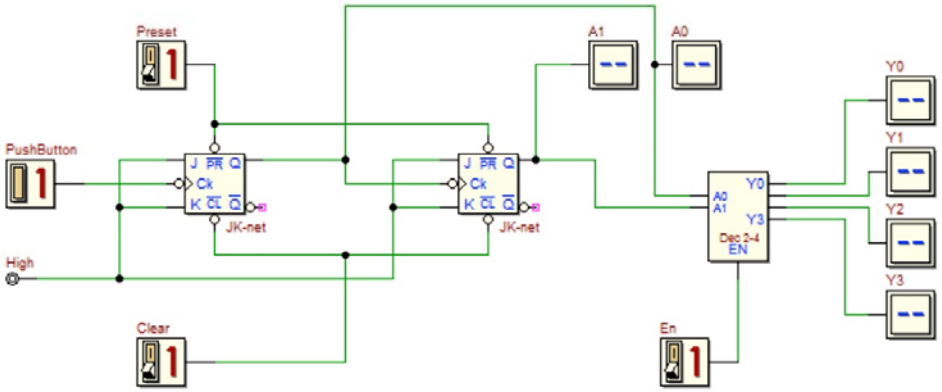
Requirements:

- A desktop or laptop computer.
- Download the Deeds (Digital Electronics Education and Design Suite) software.

Simulation:

Simulate the 2-bit counter with a 2-t0-4 decoder and show the **timing diagram.**.

1- Use the simulator to draw the following circuit with the specified initial values of the switches.



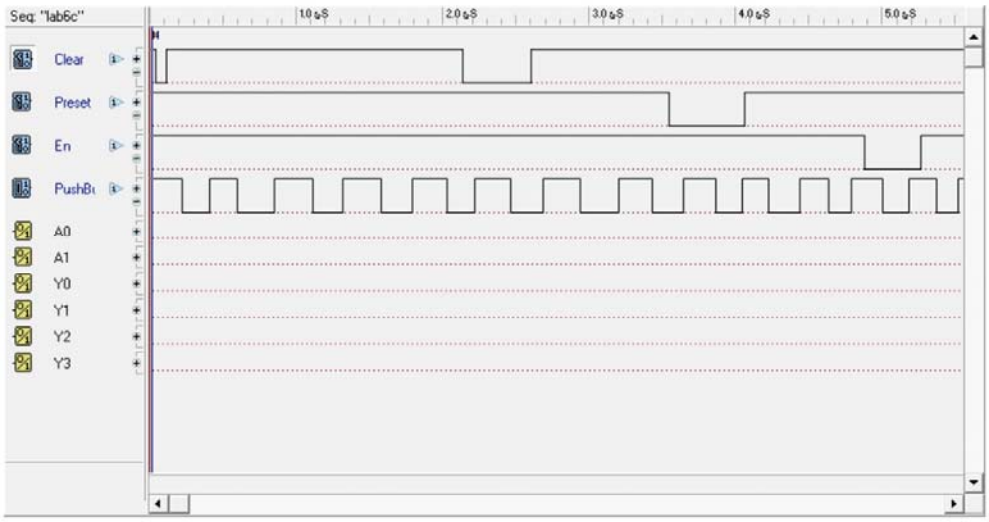
2- Start the **functional simulation (Interactive Animation)** of the sequential circuit by clicking, on the **d-DcS toolbar**, the command  .

3- Switch the *Clear* switch from 1 to 0 and then return it to 1.

4- Push on the button " PushButton" ten times and record the results in the following table.

PushButton	Clear	En	A0	A1	Y0	Y1	Y2	Y3
1	1	1						
2	1	1						
3	1	1						
4	1	1						
5	1	1						
6	1	0						
7	1	0						
8	0	0						
9	0	0						
10	0	0						

4- Check the **timing simulation** of the underlying circuit by clicking, on the **d-DcS toolbar**, the command  . The input values must be drawn directly on the timing diagram window as shown in the following figure. Each student is required to draw the output resulted from the **timing simulation** on the figure below.



5- Write here what you have learnt from the experiment.

LAB 7: Memory and Programmable Logic

Objectives

- To learn the basics behind the memory and programmable logic through a set of problems.

Background:

Try out the following exercises before your lab.
7.1, 7.9, 7.14, 7.15, 7.22, 7.25, 7.28

During the lab, the TA will answer the exercises 7.1, 7.9, 7.14, and 7.15.

Reflection Questions:

Student is required to submit the following:

- Answers to the exercises 7.22, 7.25 and 7.28 above.
- Deadline: Before the final lab test.

Student answer to the reflection questions:

References

1. M. Morris Mano and Michael D. Ciletti, Digital Design, 6th Edition, Pearson Education, 2018.

2. Thomas L. Floyd, Digital Fundamentals, 11th Edition, Pearson Prentice Hall, 2015.

3. Ronald J. Tocci, Neal S. Widmer, and Gregory L. Moss, Digital Systems: Principles and Applications, 12th Edition, Pearson, 2017.

4. Donald P. Leach and Albert Paul Malvino, Digital Principles and Applications, 8th Edition, McGraw-Hill, 2014.

5. Charles H. Roth Jr. and Larry L. Kinney, Fundamentals of Logic Design, 7th Edition, Cengage Learning, 2013.

6. M. Morris Mano, Computer System Architecture, 3rd Edition, Prentice-Hall, 1993.

7. R. P. Jain, Modern Digital Electronics, 4th Edition, Tata McGraw-Hill, 2010.

8. William Stallings, Computer Organization and Architecture: Designing for Performance, 11th Edition, Pearson, 2023.

9. Floyd, T. L., Digital Logic Fundamentals and Applications Using TTL Integrated Circuits, Pearson, 2020.

10. Online Resource: https://www.tutorialspoint.com/digital_electronics/ (for supplementary conceptual explanations).

11. John F. Wakerly, Digital Design: Principles and Practices, 4th Edition, Pearson Prentice Hall, 2005.

12. Stephen Brown and Zvonko Vranesic, Fundamentals of Digital Logic with VHDL Design, 3rd Edition, McGraw-Hill Education, 2009.

13. Stephen Brown and Zvonko Vranesic, Fundamentals of Digital Logic with Verilog Design, 3rd Edition, McGraw-Hill Education, 2014.

14. David Money Harris and Sarah L. Harris, Digital Design and Computer Architecture, 2nd Edition, Morgan Kaufmann, 2012.

15. Norman S. Nise, Engineering Digital Design, 2nd Edition, Wiley, 2000.

16. John M. Yarbrough, Digital Logic: Applications and Design, Thomson Learning, 1997.

17. Anil K. Maini, Digital Electronics: Principles, Devices and Applications, Wiley, 2007.

18. A. Anand Kumar, Fundamentals of Digital Circuits, 4th Edition, PHI Learning, 2016.

19. R. L. Tokheim, Digital Electronics: Principles and Applications, 8th Edition, McGraw-Hill Education, 2014.

20. Thomas L. Floyd, Digital Fundamentals, 11th Edition, Global Edition, Pearson Education, 2015.